

UNIVERSITY OF PADUA

DEPARTMENT OF INFORMATION ENGINEERING

Master's Degree in Computer Engineering

Modular Compiler Design for Emerging Compute-near-HBM Architectures

Supervisor:

Prof. Andrea Rigoni Garola

Candidate:

Alberto Bottari

Student ID: 2156382

Industrial supervisor:

Dr. J. A. H. Klein, imec Belgium

ACADEMIC YEAR 2025/2026

I would like to thank my academic supervisor, Prof. Andrea Rigoni Garola, for his guidance and support throughout this work, and for following my progress in a field that was new to me and often challenging.

I am grateful to imec for the opportunity to carry out this research within the TACOS team. I thank my industrial supervisor, Dr. J. A. H. Klein, and my colleagues at imec for their help, their time, and the discussions that supported the development of this thesis.

Finally, I thank my whole family for their support, patience, and encouragement throughout my studies and the completion of this work.

Abstract

Architectures such as Samsung's function-in-memory DRAM (FIMDRAM), which place computation near high-bandwidth memory (HBM), expose target-specific instruction, address, register, resource, and host-control semantics. Exploring architectural alternatives requires compiler support that preserves common program representations while isolating the decisions that change with the target. This thesis extends the Multi-Level Intermediate Representation (MLIR)-based Cinnamon infrastructure with a modular FIMDRAM backend for generating, checking, and comparing candidate configurations. The implementation introduces a FIMDRAM dialect, a selected-program representation, declarative target descriptions, exchangeable policies, pipeline recipes, and cumulative verification contracts. Samsung's original FIMDRAM address-aligned mode (AAM) design and Kukić's modified host-driven variant, called host-driven FIMDRAM (HD-FIMDRAM) in this thesis, serve as source-backed case studies. Both profiles share the representation and lowering flow while specializing instruction selection, physical addressing, register allocation, placement, binary encoding, and host-command generation. The evaluation covers six arithmetic workload families across five experiment groups. All 36 primary configurations compile successfully and pass the source-work and model-consistency checks. Replacing the register allocator preserves the eight checked aggregate counts while allowing physical register assignments to differ. Replacing the memory transaction mapping produces 32 successful compilations and four diagnosed rejections, demonstrating how AAM layout affects replay grouping and register legality within the same pipeline. Capacity sweeps show automatic partitioning of independent vector work and identify per-launch limits for reductions that the backend does not yet decompose. Separate matrix-multiplication experiments use generator-defined output tiles and show that tile shape can change both program size and address-derived register compatibility. These results demonstrate a modular backend within Cinnamon for generating and comparing FIMDRAM configurations. Compiler policies can change within a shared lowering structure while their effects on generated artifacts and compilation legality remain explicit. The evaluation establishes these properties through static artifacts rather than measured runtime performance. The resulting programs and host-command descriptions provide a basis for future automated search, functional validation, and timing simulation.

Contents

List of Acronyms **xvii**

1 Introduction **1**
1.1 Transformer Scaling and the Shift to Data Movement 1
1.2 The Memory Wall and High-Bandwidth Memory 2
1.3 Compute-near-Memory as an Architectural Response 2
1.4 The Programmability and Compilation Gap 4
1.5 Thesis Contributions and Outline 5

2 Motivation **9**
2.1 Workload Pressure in Autoregressive Inference 9
2.1.1 Prefill and Decode 9
2.1.2 Persistent State Beyond Model Weights 10
2.2 Why Compute Near High-Bandwidth Memory 11
2.3 Architectural Exploration Is Also a Compiler Problem 14
2.3.1 Locality Is Part of Correctness 15
2.3.2 Execution Is a Host–Memory Protocol 17
2.4 One Workload, Two FIMDRAM Execution Models 17
2.5 Why the Problem Belongs to a Modular Compiler 18
2.6 Summary 19

3 Background **21**
3.1 Architectural Foundations 21
3.1.1 DRAM Access and Address Structure 21
3.1.2 High-Bandwidth Memory 22
3.1.3 Samsung HBM-PIM and FIMDRAM 24
3.1.4 HD-FIMDRAM: Kukić's Modified Host-Driven Model 28
3.2 Compiler Infrastructure 31
3.2.1 MLIR's Extensible Intermediate Representation 31

3.2.2 Passes, Rewriting, and Progressive Lowering . . . 32
3.2.3 Instruction Selection, Ordering, and Registers . . . 33
3.2.4 Layout, Addresses, and Placement . . . 34
3.2.5 From Compiler Decisions to Output Artifacts . . . 34
3.3 Cinnamon and the CNM Compilation Model . . . 35
3.3.1 From Tensor Operations to Device Code . . . 35
3.3.2 Workgroups and Memory Visibility . . . 37
3.3.3 Buffers, Transfers, and Launches . . . 37
3.3.4 Target Selection and Device Extensions . . . 38
3.4 processing-in-memory (PIM) Related Work . . . 39
3.4.1 Programming Interfaces and Host–PIM Integration . . . 39
3.4.2 Compilation and Mapping for PIM . . . 40
3.4.3 Architecture Exploration and Evaluation Infrastructure . . . 41
3.4.4 FIMDRAM-Specific Compiler Lineage . . . 45
3.4.5 Positioning of This Thesis . . . 45

4 Methodology **47**
4.1 Research Design and Evaluation . . . 47
4.2 Source Traceability and Interpretive Boundaries . . . 50
4.3 Requirements for a Modular Backend . . . 50
4.3.1 Information Required by a FIMDRAM Backend . . . 51
4.3.2 Representation Requirements . . . 51
4.3.3 Separation of Facts, Policies, and Recipes . . . 52
4.3.4 Comparable but Target-Specific Lowering . . . 53
4.4 Verification Method . . . 54
4.5 Controlled Static Comparison . . . 56
4.5.1 Targets and Experimental Controls . . . 56
4.5.2 Workload Construction . . . 56
4.5.3 Measurements and Aggregation . . . 59
4.5.4 Reproducibility Procedure . . . 63
4.6 Validity and Scope of Inference . . . 63

5 Implementation **65**
5.1 Integration and Ownership Boundary . . . 65
5.2 Target-Facing Intermediate Representations . . . 66
5.2.1 Types and Launch-Level Operations . . . 66
5.2.2 Semantic and Selected Operations . . . 67

5.2.3 Selected-Program Attribute . . . 69
5.3 Declarative Target Model and Resolution . . . 69
5.3.1 Configuration, Traits, and Profiles . . . 70
5.3.2 Policy and Strategy Registries . . . 71
5.4 Shared Lowering Pipeline . . . 71
5.4.1 Conversion, Launch Formation, and Selection . . . 71
5.4.2 Scheduling and Synchronization . . . 73
5.4.3 Addressing, Registers, and Placement . . . 74
5.5 Source-Backed Target Specializations . . . 74
5.5.1 HD-FIMDRAM Host-Driven Path . . . 75
5.5.2 FIMDRAM AAM Path . . . 75
5.6 Final Artifact Materialization . . . 77
5.7 Example: Following a Vector Addition Through Lowering . . . 78
5.8 Verification and Test Implementation . . . 81
5.9 Extensibility and Implementation Trade-offs . . . 84

6 Experimental Evaluation **85**
6.1 Experimental Corpus and Validation . . . 85
6.2 Common Workloads Across Two Target Models . . . 89
6.3 Controlled Policy Substitution . . . 92
6.3.1 HD-FIMDRAM Register Allocation . . . 92
6.3.2 FIMDRAM Logical-to-Physical Mapping . . . 93
6.4 Resource-Sensitive Scaling and Boundaries . . . 94
6.4.1 Output Scaling and Launch Decomposition . . . 94
6.4.2 AAM Tile-Shape Legality . . . 95
6.4.3 Per-Launch Reduction Boundaries . . . 96
6.5 Evaluation Synthesis . . . 98

7 Discussion and Future Work **101**
7.1 Answers to the Research Questions . . . 101
7.2 Compiler-Driven FIMDRAM Design-Space Exploration . . . 103
7.3 Limitations and Threats to Validity . . . 103
7.4 Future Work . . . 104
7.4.1 Compiler–Simulator Evaluation Loop . . . 104
7.4.2 Diagnostic-Guided and Automated Exploration . . . 106
7.4.3 Lowering Extensions and Broader Validation . . . 106

8 Conclusion **109**
8.1 Thesis Outcome 109
8.2 Main Findings 110
8.3 Closing Perspective 110

Bibliography **111**

List of Figures

1.1 Terminology and selected architectural examples used in this thesis. PIM covers both computation within memory cells or arrays (CIM) and digital processing near them (CNM). The highlighted FIMDRAM-like branch identifies the architecture family addressed by the compiler backend. This is a simplified classification based on Khan et al. [5], [12]. 3

1.2 Scope of the FIMDRAM backend within Cinnamon's progressive lowering flow. 5

2.1 Prefill and small-batch decode impose different compute and memory demands. (a) Prefill processes prompt tokens together and reuses weights across them; decode generates one token per sequence at each step while reading and extending the KV cache. (b) Qualitative roofline for the principal linear layers: limited weight reuse places GEMV-like decode in the bandwidth-limited regime, while GEMM-like prefill benefits from higher arithmetic intensity. The points illustrate these tendencies, not measured throughput; batch size and operation shapes affect the balance. Author's schematic based on Gholami et al., Duan et al., and Heo et al. [1], [3], [24]. 12

2.2 Conventional host-side and bank-local execution. (a) Bank operands cross the external HBM interface to reach host arithmetic units, and results return to memory. (b) Processing units (PU) inside the HBM device compute on data from their paired banks, keeping local operand transfers within memory; the host still sends control commands and retrieves results. Dashed enclosures contain the memory-side interface, banks, and, in panel (b), processing units. B0–B3 denote banks, blue arrows show data transfers, and dashed gold arrows show control. The schematic illustrates the shortened operand path. Based on Samsung's architecture [11]. 13

2.3 Architectural changes can affect several compiler decisions at once. Instruction forms, resource capacities, and address fields constrain layout, register assignment, scheduling, and emitted artifacts through one resolved target profile. Solid arrows indicate shared constraints, not pass order. Separate copies of target facts, shown by the dashed red warning, risk inconsistent decisions when the architecture changes. 16

3.1 Organization of an HBM stack and the conventional operand path from a DRAM bank to host-side compute units. 23

3.2 Conventional HBM and Samsung FIMDRAM at die level (left) and bank level (right). Top: conventional bank organization, reproduced from the lower part of Lee et al.'s Figure 2. Bottom: processing units added between banks, with registers and arithmetic connected at the bank I/O boundary, reproduced from their Figure 1(a)–(b) [11]. The comparison highlights where bank-local computation replaces operand transfers to the host; the original detailed datapath panel is omitted. 25

3.3 Samsung FIMDRAM operating modes: (a) SB provides ordinary accesses to individual banks; (b) AB accesses corresponding locations across banks in parallel, without executing the microprogram; (c) AB-PIM uses memory commands to trigger CRF instructions across the processing units. The arrows distinguish the reserved `ACT`/`PRE` sequences for switching between SB and AB from writes to `PIM_OP_MODE` for enabling or disabling AB-PIM. `PIM CONF` exposes control and register-file areas through reserved addresses. Reproduced from Lee et al. [11, Fig. 3]. 27

3.4 Address-derived register selection in FIMDRAM AAM and compiler-selected command indices in HD-FIMDRAM. 29

3.5 Runtime access to device memory in Kukić's system. Solid arrows show the runtime using virtual mappings; dashed arrows show the mapping relationships established by kernel drivers through `mmap`. Cacheable data transfers and non-cacheable computation triggers reach the same reserved PIM region, while a separate mapping exposes the configuration space for mode control. 31

3.6 Structural elements of an MLIR operation and composition across dialect boundaries. 32

3.7 Cinnamon's progressive lowering from device-agnostic operations to shared CNM and CIM abstractions, device dialects, and host code generation. The pale device boxes indicate extension targets; the FIMDRAM path is proposed in the original paper and developed by this thesis. Reproduced from Khan et al. [5, Fig. 4]. 36

4.1 Separation of architectural facts, compiler decisions, target profiles, and ordered pipeline recipes. 52
4.2 Shared FIMDRAM lowering stages and cumulative pipeline-contract checkpoints, labeled by their main verification concern. Each checkpoint requires the state produced by preceding transformations and retains earlier guarantees. The lower boxes give examples of policies used across stages. Local operation and model checks are not shown. 54
4.3 Generation of identical target-neutral workloads and target-specific lowering inputs. 58

5.1 Ownership and integration boundary of the FIMDRAM extension inside Cinnamon. 66
5.2 Typed operation families and selection boundaries in the FIMDRAM dialect. . 68
5.3 Progressive enrichment and validation of a selected-program dictionary. 69
5.4 One declarative target description generates the cross-cutting tables used by the backend. 70
5.5 Main lowering stages and the representations they produce or update. 73
5.6 HD-FIMDRAM register allocation and low-order command-payload materialization. 76
5.7 FIMDRAM AAM address, register, replay, and host-command materialization. 77
5.8 Self-describing compiler artifacts attached to a completed FIMDRAM launch. . 79

6.1 Selected-program records and active binary words per source IR arithmetic operation for the largest primary instances. Grouped FIMDRAM records may represent multiple semantic instances; the ratios describe compiler artifacts, not throughput. 90
6.2 Target-specific selected-program composition at the largest primary sizes. FIMDRAM AAM broadcast grouping and HD-FIMDRAM host-driven movement produce different structures from the same source IR. 91
6.3 FIMDRAM mapping changes physical-field dispersion while preserving semantic work and the reported program, word, and command counts for dot product $n = 8$. 93

6.4 Legality of striped and direct FIMDRAM mappings across the primary corpus. AAM couples data layout to physical register identity, so policy substitution can alter legality as well as address dispersion. 94
6.5 AAM tile-shape design-space probe for fixed matmul $8 \times 8 \times 2$. Wider legal tiles reduce static artifacts, while adding a second row conflicts with the selected address-to-register mapping. 97
6.6 Current per-launch reduction envelope under resolved CRF, register, and command-buffer capacities. Independent work may be tiled; dependent reductions are not automatically decomposed. 98

7.1 Proposed compiler–simulator and search loop. Solid boxes denote implemented components; dashed boxes denote future work. Diagnostic feedback guides legal compilation, while evaluation feedback supports ranking among accepted candidates. 105

List of Tables

3.1 Compiler-visible binding in the two source-derived FIMDRAM models. 30
3.2 Roles of the main backend stages used in this thesis. The rows summarize responsibilities, not a universal pass order. 35
3.3 Responsibilities of Cinnamon's abstraction levels, summarized from Khan et al. [5]. 36
3.4 Scope and responsibilities of software systems related to the FIMDRAM backend. A filled mark denotes a principal responsibility, an open mark denotes partial or supporting coverage, and a dash denotes functionality outside the system's stated scope. 43

4.1 Experiments and observations used to answer the research questions. 49
4.2 Workload families and the computation patterns used to exercise the backend. . 57
4.3 Recorded compilation and validation outcomes. 61
4.4 Static measurements and their aggregation rules. 62

5.1 Ordered lowering stages, target-specific behavior, and cumulative contract boundaries of the two evaluated recipes. Rows group related passes. 72
5.2 Lowering stages and target-specific decisions for the 8-element vector addition. 80
5.3 Verification ownership and test coverage across the FIMDRAM implementation. Filled marks denote primary ownership and open marks denote supporting detection. 83
5.4 Implementation, integration, and verification requirements across compiler extension scopes. 84

6.1 Primary workload dimensions and source arithmetic counts. 86
6.2 Workload dimensions varied in the capacity study under fixed hardware budgets. 87
6.3 Resource and transaction settings of the evaluated profiles. 88
6.4 Variables and controls across the five experiment groups. 88
6.5 Outcome and invariant summary for the five experiment groups. 88

6.6 Selected-program records and active binary words per source IR arithmetic operation at the largest primary sizes. 89
6.7 Physical register assignments for vector addition at $n = 2$ under the two HD-FIMDRAM allocation strategies. 92
6.8 Launch counts at the largest sampled vector-add and matvec output sizes and across the matmul output sweep. 95
6.9 FIMDRAM AAM tile-shape outcomes for fixed matmul $8 \times 8 \times 2$. 95
6.10 First MAC trigger for each output in the first rejected 2×1 tile of matmul $8 \times 8 \times 2$. Offsets are relative to the left-hand input buffer; register indices follow the selected decoder. 96
6.11 Observed per-launch reduction boundaries and first-failure diagnostics. 97

List of Acronyms

AAM	address-aligned mode
AB	all-bank
AB-PIM	all-bank processing-in-memory
AI	artificial intelligence
API	application programming interface
CIM	compute-in-memory
CINM	Cinnamon compilation infrastructure
CNM	compute-near-memory
CPU	central processing unit
CRF	command register file
CSV	comma-separated values
DDR4	double data rate 4
DNN	deep neural network
DRAM	dynamic random-access memory
FIMDRAM	function-in-memory DRAM
FP16	16-bit floating point
FPU	floating-point unit
GEMM	general matrix–matrix multiplication
GEMV	general matrix–vector multiplication
GPU	graphics processing unit
GRF	general register file
GRF-A	general register file A
GRF-B	general register file B
HBM	high-bandwidth memory
HBM-PIM	high-bandwidth memory with processing-in-memory
HBM2	second-generation high-bandwidth memory
HBM2-PIM	second-generation high-bandwidth memory with processing-in-memory

HD	HD-FIMDRAM (abbreviated table heading)
HD-FIMDRAM	host-driven FIMDRAM
I/O	input/output
ID	identifier
IMP	in-memory processing
IR	intermediate representation
ISA	instruction set architecture
JEDEC	JEDEC Solid State Technology Association
JSON	JavaScript Object Notation
KV	key–value
LLM	large language model
MAC	multiply–accumulate
MAD	multiply–add
MLIR	Multi-Level Intermediate Representation
NMP	near-memory processing
NPU	neural processing unit
ODS	Operation Definition Syntax
PEI	PIM-enabled instruction
PIM	processing-in-memory
PIM-HBM	processing-in-memory in high-bandwidth memory
PU	processing unit
ReLU	rectified linear unit
SAXPY	scaled vector addition ($ax + y$)
SB	single-bank
SCF	structured control flow
SIMD	single-instruction, multiple-data
SoC	system-on-chip
SRF	scalar register file
SSA	static single assignment
TSV	through-silicon via
WSL2	Windows Subsystem for Linux 2

Chapter 1

Introduction

The recent progress of artificial intelligence (AI) is inseparable from the interaction between algorithms and computing systems. Improvements in model architecture have made it possible to learn increasingly expressive representations, while advances in accelerators and memory systems have made the resulting computations practical. This relationship is not static: an architectural innovation can remove one computational bottleneck only to expose another. Modern language models illustrate this dynamic particularly clearly. Their capabilities derive from scalable tensor computation, yet the cost of supplying that computation with data increasingly determines whether the models can be deployed efficiently [1].

1.1 Transformer Scaling and the Shift to Data Movement

Transformer inference provides a concrete example of the systems transition that motivates this thesis: transformers replaced much of the sequential computation characteristic of earlier sequence models with highly parallel tensor operations, making accelerator execution increasingly effective [2]. As these models scaled, however, their weights and persistent state grew, increasing the demands placed on the memory system. Consequently, efficient inference becomes a compiler and architecture problem rather than a question of peak arithmetic throughput alone [1]. Transformer inference serves here as a motivating workload, rather than as the subject of an end-to-end experimental evaluation. The thesis addresses the compiler infrastructure needed to explore near-memory architectural alternatives, with evaluation focused on generated programs and modeled target constraints. This pressure is not uniform across inference. Some phases can reuse loaded data across many operations, while others repeatedly transfer large amounts of data for comparatively little arithmetic. Chapter 2 examines this distinction through prompt processing and token generation. Here, the important consequence is that adding arithmetic capacity alone cannot resolve a bottleneck in supplying operands [1], [3], [4].

1.2 The Memory Wall and High-Bandwidth Memory

Conventional computers follow a processor-centric organization in which computation and storage are implemented by physically distinct structures. Processors operate on values held in registers and caches, whereas the bulk of an application's data resides in external dynamic random-access memory (DRAM). Cache hierarchies reduce average access time when locality permits reuse, but a value absent from on-chip storage must cross the memory channel before it can participate in computation. Results that outlive the local working set eventually travel in the opposite direction. For data-intensive applications, these transfers consume time, interconnect bandwidth, and energy without themselves advancing the arithmetic result [1], [5], [6], [7]. The term *memory wall* describes the growing disparity between the rate at which processors can consume data and the rate at which memory systems can supply it. Evidence from server hardware over a twenty-year period shows the scale of this divergence: peak floating-point throughput increased by roughly $3.0\times$ every two years, while DRAM bandwidth and interconnect bandwidth increased by approximately $1.6\times$ and $1.4\times$, respectively [1]. The bottleneck is therefore not a fixed property of an application. As compute throughput grows faster than communication capability, kernels that were once limited by arithmetic can become limited by data delivery.

Data movement also has an energy consequence. Accessing operands in DRAM and transporting them through the memory hierarchy can cost substantially more energy than the arithmetic operations that use them; the literature reports differences of orders of magnitude for representative DRAM accesses and floating-point operations [8]. The exact ratio depends on technology and access location, but the implication is that reducing computation is not sufficient when communication dominates the energy budget. This makes locality a first-class design concern for both high-performance and energy-constrained systems.

High-bandwidth memory (HBM) addresses part of this problem by providing a wide connection between stacked DRAM and the processor [9]. It increases the rate at which the host can obtain operands, but those operands must still travel from memory to the units that perform arithmetic. Reducing this remaining transfer cost motivates reconsidering where computation takes place [10], [11].

1.3 Compute-near-Memory as an Architectural Response

Reducing the cost of transporting operands between DRAM and a host processor motivates moving part of the computation closer to the data. Memory-centric architectures pursue this objective at different points in the memory hierarchy, with different degrees of coupling between processing and storage [6], [12], [13]. Because terminology varies across the literature, this thesis uses *compute-in-memory (CIM)* for computation that directly exploits memory-cell, array, or analog

electrical behavior [14], and *compute-near-memory (CNM)* for programmable digital processing units outside the cell array, connected to memory through local high-bandwidth paths. The broader term processing-in-memory (PIM) covers both categories [6], [12]. This thesis focuses on digital processing units coupled to HBM. Such units can perform selected operations near the stored data, reducing the operands that must cross the external memory interface. Samsung's function-in-memory DRAM (FIMDRAM) is the original architecture on which this work builds, implemented in an HBM2-PIM device also known as Aquabolt-XL [10], [11]. Its benefit depends on both the processing resources and the way the workload is distributed among them. Figure 1.1 locates FIMDRAM and related architectures within the CNM branch of the broader design space.

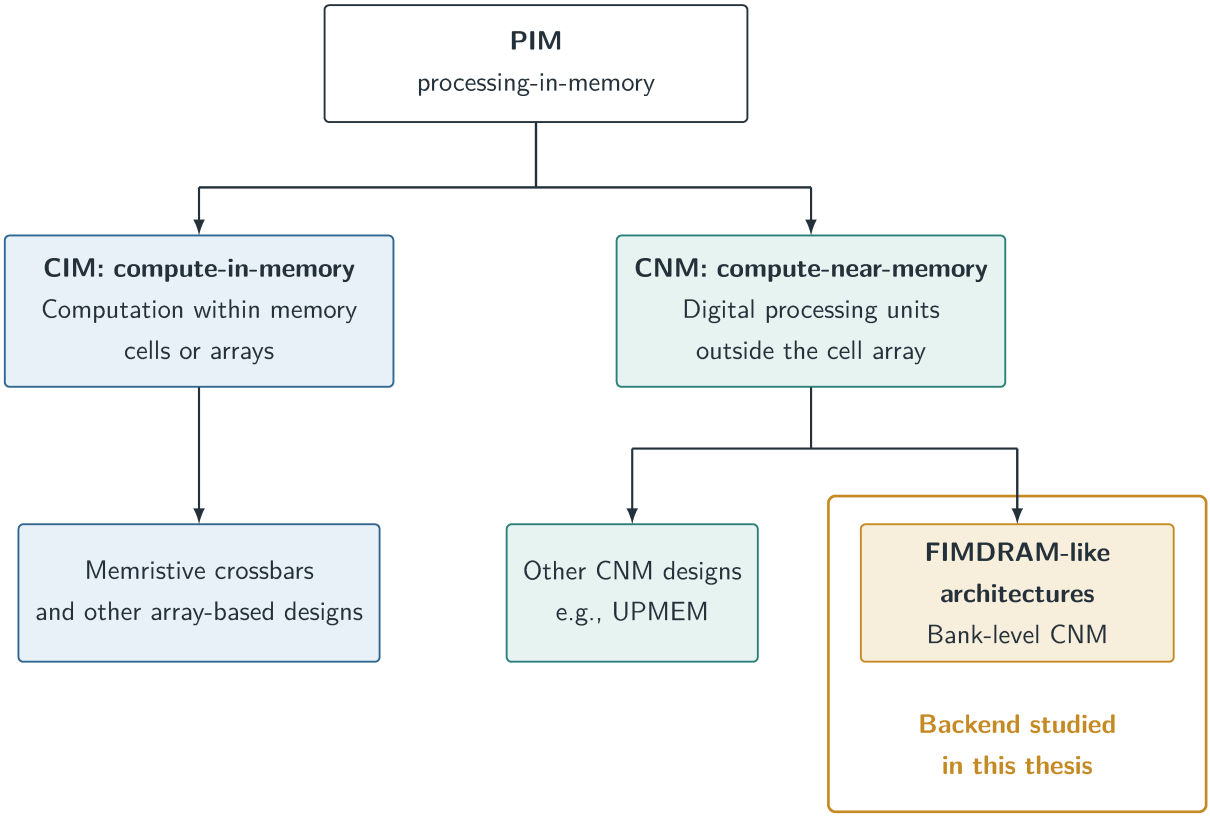


Figure 1.1: Terminology and selected architectural examples used in this thesis. PIM covers both computation within memory cells or arrays (CIM) and digital processing near them (CNM). The highlighted FIMDRAM-like branch identifies the architecture family addressed by the compiler backend. This is a simplified classification based on Khan et al. [5], [12].

Near-memory processing nevertheless introduces its own constraints. The limited resources available beside memory require trade-offs between processing capability, storage, area, and energy. Moreover, a workload mapping suited to one configuration may not make effective use of another. The research literature examines these coupled architectural and mapping choices through design-space exploration. Medina et al. study processing-near-bank architectures [15],

while Wang et al. introduce NicePIM to explore hardware configurations and workload mappings together [16]. The resulting design space calls for a systematic way to vary architectural resources, construct compatible workload mappings, and evaluate the resulting alternatives. For this thesis, the software requirement that follows is central: exploring a different configuration should not require rebuilding the workload implementation around that configuration by hand. A reusable compilation flow must make target variation manageable while preserving the relationship between operations, data placement, and available resources. This requirement motivates the programmability and compilation problem examined in the following section.

1.4 The Programmability and Compilation Gap

Moving computation close to memory also changes what software must manage. Data placement, the operations available near memory, and coordination with the host determine how a workload can execute. A program written for one device cannot assume that these choices remain valid on another [5], [17], [18]. For architectural exploration, this creates a practical difficulty: changing the machine may require changing the program used to evaluate it. A compiler can manage this dependence by preserving the intended computation while introducing the constraints of a selected target. The aim is to reuse the parts of compilation that remain valid and isolate the decisions that change with the hardware [5], [19], [20]. This is distinct from simplifying programming for one fixed device. Architectural exploration requires a way to vary the target without repeatedly constructing a separate compilation flow.

This thesis builds on *CINM (Cinnamon)*, an existing compilation infrastructure for heterogeneous CIM and CNM devices. Cinnamon uses the Multi-Level Intermediate Representation (MLIR) infrastructure to separate common computational abstractions from device-specific code generation [5], [19]. The present work extends its CNM flow with a backend, the part of the compiler that translates a shared representation into target-specific programs, for architectures such as FIMDRAM. Two published models provide concrete cases for the investigation. FIMDRAM address-aligned mode (AAM) follows Samsung's original design [11]. Host-driven FIMDRAM (HD-FIMDRAM) is the name used in this thesis for Kukić's host-driven variant, which modifies that design's instruction and addressing mechanisms [21]. Both designs are prior work; this thesis develops the shared compiler backend that supports them. Their differences matter because the same computation imposes different requirements on addressing, register use, and host control. Chapter 2 explains why this is a useful case for design-space exploration, and Chapter 3 develops the technical details. Figure 1.2 identifies the scope of the extension. Cinnamon supplies the compilation stages above the `cnm.launch` boundary, which represents work intended for near-memory execution. Below that boundary, the new backend derives programs

for the two target models through a shared compilation structure. The split into different output artifacts reflects their architectural differences, not two independent front-end compilers.

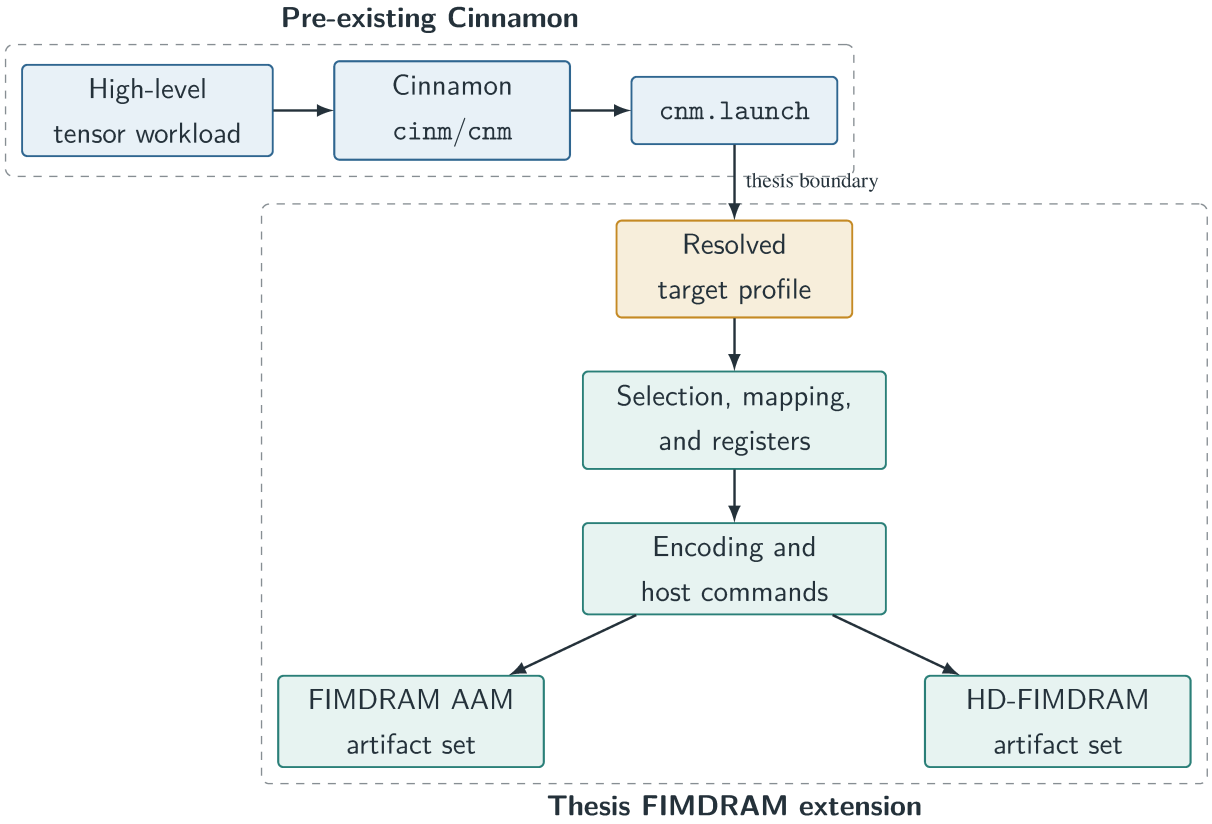


Figure 1.2: Scope of the FIMDRAM backend within Cinnamon's progressive lowering flow.

The research problem is therefore how to preserve a common compilation flow while making architectural variation explicit and inspectable. The backend must represent the differences that affect generated programs without distributing target-specific assumptions throughout otherwise reusable code. The following section states the questions and contributions through which the thesis addresses this problem.

1.5 Thesis Contributions and Outline

This thesis was developed in collaboration with the Technology–Application Co-Optimization for Systems (TACOS) team at IMEC. The team studies memory systems and computing accelerators to understand how emerging technologies affect application execution at system level. Within this collaboration, the project addresses the compiler support needed to explore FIMDRAM-like architectures. Such exploration requires more than changing hardware parameters: a different execution model can also change how the compiler assigns registers, places data,

and generates host commands [11], [21]. A reusable backend would let the team investigate these alternatives without developing a separate compilation flow for each one. The project grew from an initial investigation of host-memory interaction and synchronization in Kukić's earlier compiler. His evaluation reports that serialized accesses to an uncacheable PIM region dominate execution in the simulated system, limiting the runtime benefit of further tiling and fence optimizations [21, Section 6.2.2]. His proposed move towards cacheable accesses would also change the command-address encoding and require substantial changes to core compiler passes [21, Section 6.4.1]. Investigating that direction therefore raises a broader practical question: how can the compiler accommodate alternative execution and addressing models without rebuilding the lowering flow around each architectural choice?

With the approval of the industrial supervisor, Dr. Joshua Klein, the project consequently focused on a modular Cinnamon backend as a foundation for these investigations. Kukić already identifies Cinnamon as a possible higher-level framework for his backend, while Cinnamon defines shared CNM abstractions and extension points for device-specific lowering [21, Section 6.4.2], [5, Section 3.2.5]. For the collaboration, the intended value is a common infrastructure that makes architectural constraints and compiler decisions explicit, supports controlled changes, and supplies inspectable programs for subsequent evaluation. The thesis also assesses whether Cinnamon provides a suitable base for this form of FIMDRAM-like design-space exploration, rather than implementing the cache-interface redesign that initially motivated the investigation. To address this need, the thesis develops the backend below the shared compilation boundary introduced in Section 1.4. Its objective is to derive inspectable target programs from a common CNM workload, detect configurations that violate the modeled constraints, and allow selected compiler policies to vary without reconstructing the lowering flow. The original FIMDRAM AAM and modified HD-FIMDRAM models provide the two architectural cases for this investigation [11], [21]. They are used to examine both the structure that the backend can share and the target distinctions that it must preserve. Four research questions guide the investigation of compiler behavior within the modeled target subsets, rather than the relative execution speed of the devices:

RQ1: Can one Cinnamon backend compile the same workload for FIMDRAM AAM and HD-FIMDRAM, while keeping target-specific decisions separate from shared compiler stages?

RQ2: What errors can the backend detect during compilation, and what can its checks guarantee about the generated program?

RQ3: How do the generated instructions, register assignments, memory layouts, and host commands differ between the two target models for the same workload?

RQ4: Can compiler strategies, such as register allocation and memory mapping, be changed without modifying the rest of the compilation flow?

To answer these questions, the thesis makes the following contributions:

1. It introduces a common target-facing representation within Cinnamon, comprising a FIM-DRAM dialect and a representation of selected device operations. These retain the buffer, register, address, and execution information needed to lower a CNM workload to either modeled target.

2. It defines a declarative and policy-based target model. Named target profiles make architectural constraints and compiler choices explicit, while policy interfaces isolate decisions such as instruction selection, addressing, and register assignment. Pipeline recipes apply these choices through a shared sequence of lowering stages.

3. It implements two source-backed target paths in that common architecture. The HD-FIMDRAM path retains the modified model's host-driven command and addressing semantics; the FIMDRAM AAM path retains address-derived register selection and command-register-file replay. Their distinct instruction and host-command artifacts make these differences available for inspection and comparison.

4. It provides layered verification and a reproducible compiler-side design-space evaluation flow. Compatibility checks, pipeline contracts, and tests check modeled invariants and diagnose invalid configurations or intermediate states. The evaluation compares generated programs and target-specific static metrics, and varies register-allocation and logical-to-physical mapping policies in controlled ablations.

The scope of these contributions is deliberately compiler-visible. The backend generates and filters design candidates and exports artifacts suitable for subsequent functional or timing evaluation; it does not itself constitute a complete performance-oriented design-space optimizer. The goal is a reusable compilation structure, not exhaustive support for either target or an automatic search for a legal mapping and tiling for every workload. The thesis does not propose a new PIM device, provide a production Samsung runtime, or claim measured latency, bandwidth, energy, or thermal improvements. Its comparisons concern generated programs, target consistency, static costs, and the modularity of the compiler structure; model-specific command counts are not interpreted as execution time. This boundary separates demonstrated compiler behavior from claims that would require physical hardware or a validated timing model.

The remainder of the thesis develops this argument in stages. Chapter 2 examines the workload and data-movement pressures that motivate compute-near-HBM execution. Chapter 3 introduces the relevant memory architectures, MLIR and Cinnamon abstractions, and related PIM

compilation systems. Chapter 4 derives the source-traceability, representation, modularity, and evaluation requirements. Chapter 5 presents the dialect, target descriptions, policies, recipes, backend passes, and verification infrastructure. Chapter 6 applies the common static comparison flow to the source-backed targets and controlled ablations. Chapter 7 interprets the evidence, limitations, and opportunities for runtime and timing integration, before Chapter 8 summarizes the contribution and the main findings.

Chapter 2

Motivation

We have introduced the connection between memory-bound AI workloads, compute-near-memory hardware, and compiler infrastructure that motivates this thesis. This chapter develops that motivation more precisely. It first examines why autoregressive inference exposes a bandwidth bottleneck that conventional accelerators cannot remove by increasing arithmetic throughput alone. It then explains why compute-near-HBM is a relevant architectural response, but also why the hardware opportunity does not translate directly into a usable programming model. The discussion finally narrows to the central problem of this thesis: two closely related FIM-DRAM execution models can implement the same high-level operation while imposing different requirements on data placement, register assignment, instruction encoding, and host command generation. A compiler must preserve these differences without duplicating the complete lowering flow.

2.1 Workload Pressure in Autoregressive Inference

Transformer inference is not a uniform sequence of tensor operations. Its resource demands change between processing the input sequence and generating the output sequence, even though both phases evaluate the same model layers. This matters for architectural mapping because the shape of a matrix operation determines how often fetched data can be reused and, therefore, whether arithmetic throughput or memory traffic limits execution [1], [3].

2.1.1 Prefill and Decode

During *prefill*, the model processes the prompt tokens together. The linear projections in attention and the feed-forward network consequently operate on matrices of token activations and are commonly expressed as general matrix–matrix multiplications (GEMMs). A weight value

fetched from memory can participate in computations for multiple prompt tokens, allowing the accelerator to amortize the transfer over many arithmetic operations. Prefill therefore exposes substantial data reuse and can make effective use of the wide matrix engines in graphics processing units (GPUs) and neural processing units (NPUs) [3], [22].

Decode follows a different execution pattern: autoregressive generation produces one new token per sequence at each iteration, and the new token depends on the state generated by all preceding iterations. At a small batch size, the principal linear layers consequently approach general matrix–vector multiplication (GEMV): the system streams large weight matrices to process comparatively little activation data. The matrices still contain the same model parameters, but each fetched weight contributes to far fewer operations before the next layer or token step. With this limited weight reuse, small-batch decode can be constrained by how quickly memory supplies data to the compute units, rather than by their arithmetic throughput [1], [3], [23].

The roofline model provides a useful interpretation of this transition. If arithmetic intensity I denotes the number of arithmetic operations performed per byte transferred, attainable performance is bounded by the minimum of the platform's peak compute throughput and the product of I with memory bandwidth. GEMM can increase I through reuse across rows, columns, and tokens; GEMV offers much less reuse of the weight matrix. Adding more multiply–accumulate units does not remove the resulting decode bottleneck when the memory system cannot supply those units at a corresponding rate [1], [3]. This characterization is workload-dependent rather than absolute. Increasing the batch size converts some matrix–vector work into matrix–matrix work and can improve accelerator utilization, while request lengths and scheduling policies change the relative time spent in each phase. NeuPIMs, for example, uses sub-batch interleaving to overlap GEMM work on an NPU with GEMV work on PIM resources [24]. The relevant conclusion is thus not that all large language model (LLM) inference is memory-bound, but that a serving system contains regions with materially different compute and bandwidth requirements. Efficient execution requires the system to map each region to resources that match its operational characteristics.

2.1.2 Persistent State Beyond Model Weights

Here, *memory pressure* refers to the demands that inference places on memory capacity and bandwidth: how much data the system must retain and how quickly it must supply that data for computation [24], [25]. Model weights are not the only source of these demands. Attention reuses the key and value projections of preceding tokens, which inference systems retain in a key–value (KV) cache. During decode, every new token consults the accumulated state, so the cache creates both a capacity requirement and recurrent memory traffic that increase with the active sequence state [24], [25]. Paging and specialized placement can improve capacity utiliza-

tion, but they do not erase the underlying need to move or access the cached data. Weights and KV state also differ in lifetime and access pattern. Weights are read-only during inference and are reused across requests, whereas the KV cache grows dynamically and belongs to individual sequences. Architectures that accelerate only weight-intensive GEMV therefore address an important part of decode without automatically solving the complete serving problem. Recent heterogeneous proposals reflect this distinction by combining compute-enabled memory with conventional accelerators or with memory layers specialized for capacity and state access [23], [24], [25]. For this thesis, these systems establish the workload-level motivation for memory-side execution; they do not imply that the FIMDRAM backend should implement an entire LLM serving stack.

Figure 2.1 brings together the two sources of memory traffic discussed above. Panel (a) separates prompt processing from sequential token generation: prefill reuses weights across prompt tokens, whereas small-batch decode combines limited weight reuse with repeated access to the accumulated KV cache. Panel (b) relates the linear layers to the roofline model. The lower-intensity GEMV regime is constrained by memory bandwidth, while greater reuse in GEMM can bring execution closer to the compute limit. The positions are qualitative; batch size and operation shapes determine the actual balance [1], [3], [24].

2.2 Why Compute Near High-Bandwidth Memory

The limited weight reuse discussed in the preceding section makes the path between memory and arithmetic units central to small-batch decode. HBM addresses this pressure by widening and shortening the interface between DRAM and the processor [1], [9]. A wider interface supplies data at a higher rate, but the host must still fetch operands from their banks before performing arithmetic. Figure 3.1b shows this conventional path from an HBM bank to host-side compute units. For operations with little weight reuse, these transfers can still limit execution even when the host has unused arithmetic capacity [1], [10]. HBM therefore increases the bandwidth available for data movement without removing the need for that movement.

CNM reduces those transfers by moving selected arithmetic operations closer to the data. In a near-bank design, processing units read operands through internal DRAM paths and compute locally, so the host need not fetch every operand. Samsung's HBM-PIM architecture, also known as FIMDRAM, implements this approach in DRAM technology [11], [21]. A single command can trigger processing across multiple banks, allowing the processing units to use aggregate internal bandwidth beyond what the external HBM interface exposes to the host [11], [26]. CNM thus complements HBM: the wider interface increases the rate of external transfers, while bank-local computation reduces the traffic that must cross it. Matrix–vector products with

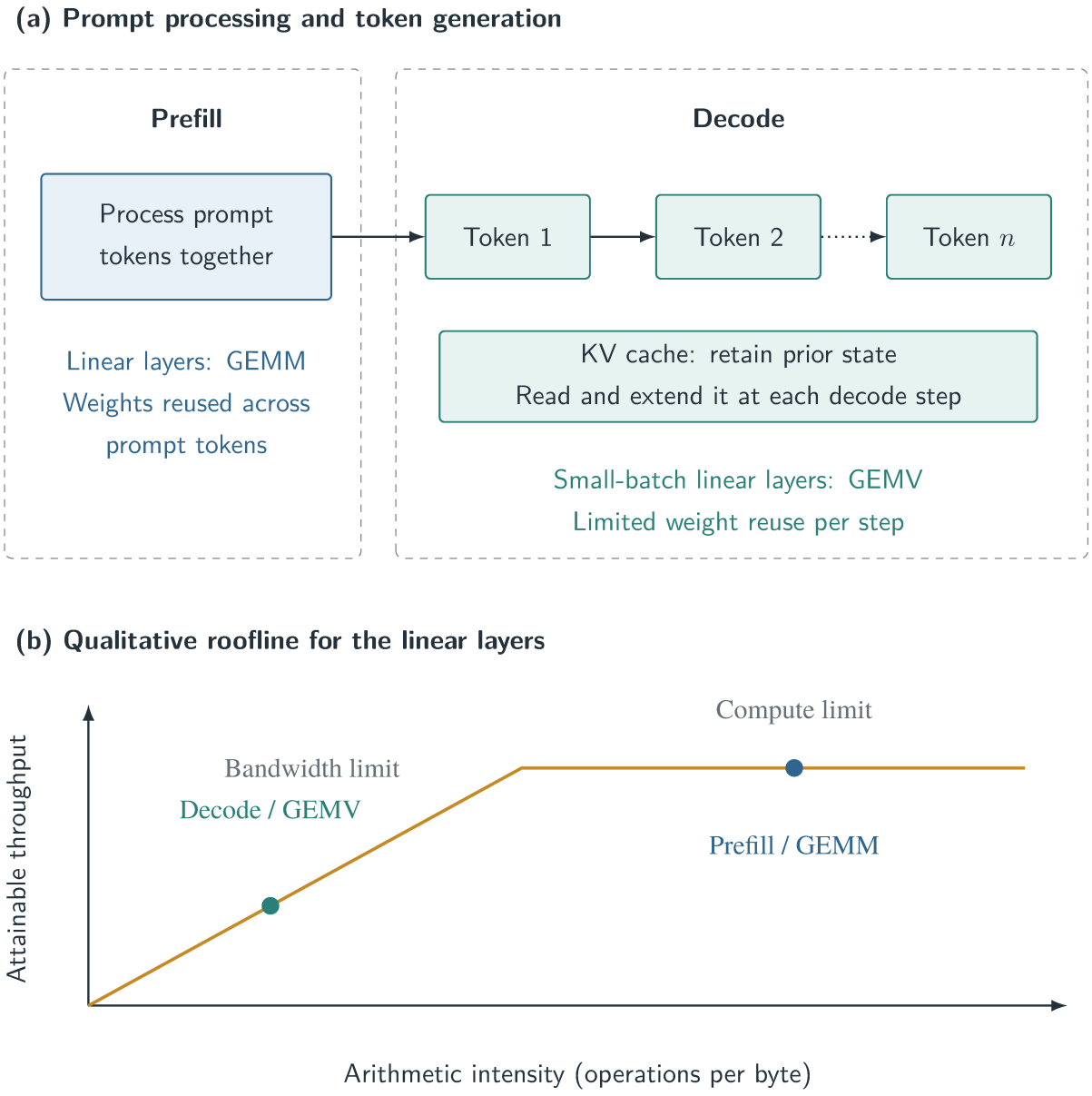


Figure 2.1: Prefill and small-batch decode impose different compute and memory demands. (a) Prefill processes prompt tokens together and reuses weights across them; decode generates one token per sequence at each step while reading and extending the KV cache. (b) Qualitative roofline for the principal linear layers: limited weight reuse places GEMV-like decode in the bandwidth-limited regime, while GEMM-like prefill benefits from higher arithmetic intensity. The points illustrate these tendencies, not measured throughput; batch size and operation shapes affect the balance. Author's schematic based on Gholami et al., Duan et al., and Heo et al. [1], [3], [24].

little weight reuse can benefit from this arrangement when the banks hold the weights needed by their local processing units [10], [11]. The shortened operand path and the retained host-control path are contrasted in Figure 2.2. In panel (a), operands and results from every bank cross the external HBM interface to host arithmetic units. In panel (b), paired banks feed local processing units, while the host path carries orchestration and final results rather than every intermediate operand. The comparison identifies the traffic that CNM can remove without implying that host involvement disappears.

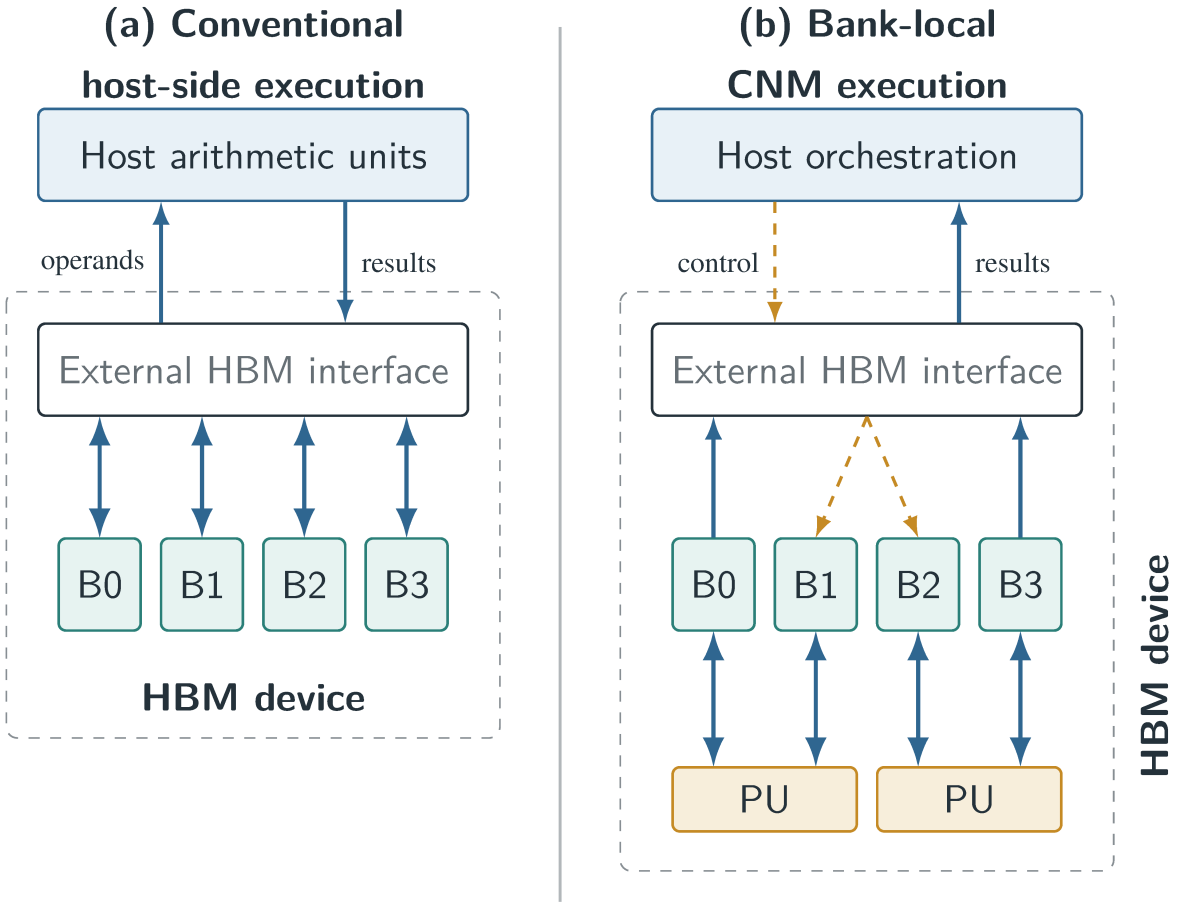


Figure 2.2: Conventional host-side and bank-local execution. (a) Bank operands cross the external HBM interface to reach host arithmetic units, and results return to memory. (b) Processing units (PU) inside the HBM device compute on data from their paired banks, keeping local operand transfers within memory; the host still sends control commands and retrieves results. Dashed enclosures contain the memory-side interface, banks, and, in panel (b), processing units. B0–B3 denote banks, blue arrows show data transfers, and dashed gold arrows show control. The schematic illustrates the shortened operand path. Based on Samsung's architecture [11].

The attraction of near-bank execution does not make it a universal replacement for the host. The processing units operate under tighter area, storage, and instruction constraints than a GPU or NPU, and compute-intensive GEMM can benefit more from the host's matrix engines. Non-

linear operations, control, and communication may likewise remain outside the memory device. PIM-GPT separates memory-side vector–matrix multiplication from nonlinear and control processing, while NeuPIMs and HPIM assign operators to different compute and memory resources according to their characteristics [3], [23], [24]. CNM should therefore be used as part of a heterogeneous system, for operations that suit its architecture and with efficient host interaction. This raises a software question: how should near-memory operations be expressed, prepared, and invoked? Once an operation moves into the memory system, ordinary tensor semantics are not sufficient to determine the generated program. The compiler must connect the operation to a physical data layout, the capabilities of bank-local processing units, and the command protocol through which the host controls execution.

2.3 Architectural Exploration Is Also a Compiler Problem

The central problem of this thesis is that changing a near-memory architecture can also change how a compiler must prepare a workload for execution. Design-space exploration compares alternative hardware configurations and the ways in which computations use them. These choices are related: NicePIM explores hardware configurations together with workload partitioning and data layouts, while Medina et al. examine how instruction storage, register capacity, and memory interfaces affect bank-level execution [15], [16]. In an exploration flow based on compiled workloads, each candidate needs a program that respects its constraints. The compiler must therefore adapt the same intended computation to the candidate architecture before its behavior can be evaluated.

Some changes affect resource limits; others change the meaning of a compiler decision. Consider a design in which the address of a memory request also selects the registers used by an arithmetic instruction. Choosing where to place an operand then also constrains which register the operation can use. A compiler cannot choose the address and register independently and assume that they will agree. In a design that carries register information explicitly in a command, it can instead assign registers according to the values needed by later operations and encode those assignments separately. Samsung's AAM and Kukić's modified host-driven model illustrate this distinction [11], [21]. The requested arithmetic can remain unchanged, yet the compiler must use a different relationship between data placement, register assignment, and command generation. Supporting the second design may therefore require a different compilation strategy, not just a different value for a hardware parameter. Figure 2.3 shows why these changes extend beyond one compiler stage. Instruction forms determine which operands an operation can use; register and command register file (CRF) capacities limit the data and instructions that can remain in the processing unit; address fields determine how memory requests identify data and, under AAM,

registers [11], [15], [21]. The lower group contains the compiler decisions that must respect these properties. For example, address mapping and register assignment must agree when an address selects a register, while scheduling and command generation must preserve the order required by the execution model. The figure connects them through one resolved target profile: an explicit description of the selected target and its compatible compiler choices. The arrows show shared constraints rather than the order of compilation. This dependence makes compiler organization part of the exploration problem. If several stages contain their own copies of a target's rules, an architectural change requires finding and updating each copy. Updating the address mapping without updating register assignment, for example, can produce a program whose chosen registers disagree with those selected by its commands. The red warning in the figure represents this risk. The backend requirements are therefore to make the selected constraints available consistently and to check that decisions made at different stages agree. The target descriptions and verification boundaries developed later in the thesis address these requirements.

The practical objective is to explore these alternatives without rebuilding the complete compilation flow for each one. The workload should retain its intended computation, while the backend changes the decisions required by the selected target. This also gives a common basis for examining the resulting programs: differences should follow from the selected architecture and compiler choices, rather than from unrelated compilation flows. The comparison in Section 2.4 develops this problem through the two FIMDRAM models. Before that comparison, the following subsections explain two reasons why near-bank execution exposes such dependencies: local data access and host-triggered execution.

2.3.1 Locality Is Part of Correctness

Near-bank processing obtains bandwidth by distributing work and data across the memory organization. A processing unit can directly consume only the operands reachable through its local memory path; the host-visible tensor layout must therefore be transformed into bank, row, and column placements that make the intended parallel operation possible. Samsung's software stack accordingly rearranges data for PIM GEMV and requires awareness of the physical address mapping [11]. Other systems reach the same problem through different mechanisms: FACIL and UM-PIM both treat the difference between host-friendly and PIM-friendly layouts as a central address-mapping concern [22], [27]. Placement affects more than performance. If corresponding operands map to incompatible banks or to addresses that select unintended hardware state, the emitted operation no longer has the requested tensor semantics. The compiler must consequently retain logical indexing, physical placement, and target addressing as related but distinguishable concepts. Collapsing them too early into one integer address makes it difficult to determine whether a later failure originated in tensor partitioning, bank assignment,

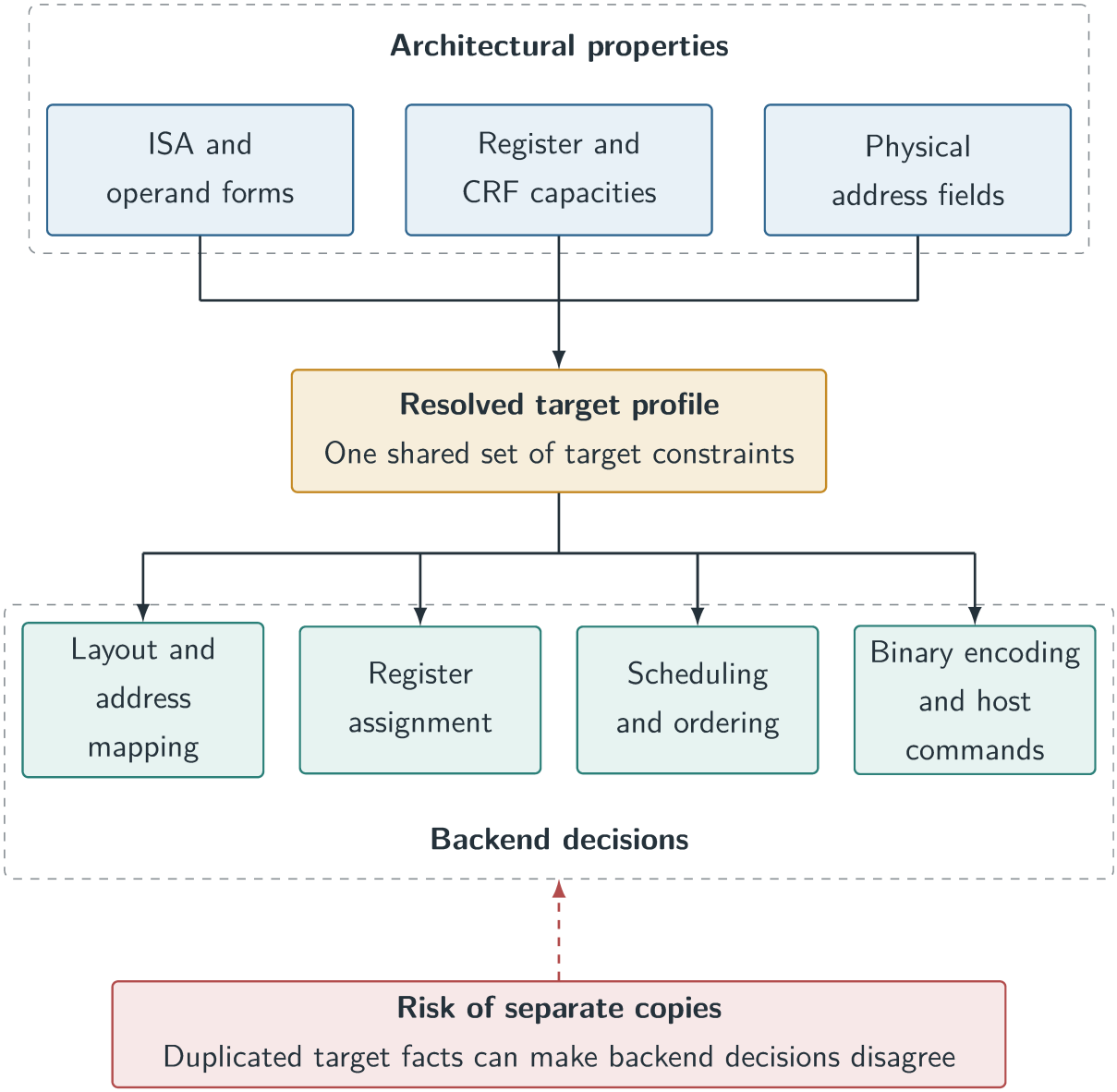


Figure 2.3: Architectural changes can affect several compiler decisions at once. Instruction forms, resource capacities, and address fields constrain layout, register assignment, scheduling, and emitted artifacts through one resolved target profile. Solid arrows indicate shared constraints, not pass order. Separate copies of target facts, shown by the dashed red warning, risk inconsistent decisions when the architecture changes.

physical encoding, or command generation.

2.3.2 Execution Is a Host–Memory Protocol

FIMDRAM does not execute an independently launched device kernel in the same sense as a GPU. The host configures the PIM mode, transfers instructions or control state, and issues memory commands whose arrival triggers operations inside the device. Correctness therefore depends on both the processing-unit instruction sequence and the order and addresses of the associated host commands [11], [21]. A backend that emits only device opcodes omits part of the executable behavior. Changing the execution model can therefore affect both device code and the host's interaction with memory. The compiler must preserve their correspondence, rather than treat host control as an unrelated step added after code generation [11], [21]. The operating modes and command mechanisms behind this requirement are described in Section 3.1.3; here, they establish why the compiler's responsibility extends beyond selecting arithmetic instructions.

2.4 One Workload, Two FIMDRAM Execution Models

Samsung's original FIMDRAM and the modified model developed by Kukić illustrate how an architectural choice affects compilation. Both place arithmetic near DRAM banks and rely on the host to trigger execution, but they bind memory access to register selection differently [11], [21]. The comparison connects this architectural difference to the changes needed in the compiler to prepare the same workload for either model. In Samsung's AAM, the address of a memory request also determines which registers an arithmetic operation uses. Data placement and register selection must therefore agree. Kukić's modified host-driven model separates those decisions by carrying compiler-selected command information alongside the memory address [11], [21]. The detailed encodings are presented in Section 3.1.3 and Section 3.1.4. For the motivation here, the important difference is whether the compiler can choose a register independently of the location of the data. Kukić's work also illustrates the compiler effort associated with exploring these alternatives. He develops a specialized FIMDRAM compilation flow on top of MLIR and Polygeist for his modified host-driven model. When discussing a possible transition to an AAM-like mechanism, he identifies the need to redesign core compiler passes [21, Section 6.4.1]. Testing an architectural alternative can therefore require changes to the compilation strategy before its effects can be evaluated. This does not mean that every new design requires an entirely new compiler; it means that support for one configuration does not automatically provide support for a different execution model. This example motivates the shared backend developed in the present work. The same workload should provide a common starting point, while the backend makes the decisions required by each model explicit. Separate implementa-

tions would make it harder to distinguish architectural effects from differences in the compiler code. Conversely, forcing both targets through one fixed strategy would ignore constraints that distinguish their execution. Design-space exploration needs both reuse and a way to change the parts of compilation that depend on the architecture.

2.5 Why the Problem Belongs to a Modular Compiler

The comparison above gives a concrete case of the compiler problem introduced in Section 2.3. The backend needs to change the strategy for address and register assignment while retaining the parts of compilation that both models can share. Cinnamon's separation of device-independent operations from device-aware lowering provides a starting point [5]; within the FIMDRAM backend, the same principle must also apply to variation between execution models. The backend must preserve a common semantic path for target-neutral CNM workloads while selecting target-specific policies only where the hardware models genuinely differ. For the targets considered here, those policies include operation legality, instruction selection, physical addressing, logical-to-physical mapping, register assignment, scheduling, binary encoding, and execution-packet formation. Section 3.2.3 introduces the instruction and register decisions, while Section 3.2.4 distinguishes indexing, address mapping, and placement. Treating these choices as an undocumented collection of command-line options would not be sufficient: incompatible combinations could reach late lowering stages and produce inconsistent programs.

The compiler architecture should therefore satisfy four needs:

1. It must retain the semantic intent of the workload until enough target information exists to make a valid selection.

2. It must make resolved target properties available through one coherent description rather than duplicating constants across passes.

3. It must define stage contracts that reject missing, premature, or contradictory state near the point where the inconsistency appears.

4. It must permit controlled policy changes so that architectural and compiler alternatives can be compared while the rest of the pipeline remains fixed.

These needs motivate the target profiles, policy interfaces, pipeline recipes, and layered verification developed in this thesis. They give concrete design requirements for the research questions in Section 1.5: preserving a common representation addresses RQ1, checking consistency addresses RQ2, and supporting controlled policy changes addresses RQ4. RQ3 concerns what can be learned from the programs generated by that infrastructure.

Modularity also determines what can be learned from evaluation. PIM studies often use different workloads, architectural assumptions, simulation depths, and metrics, which complicates direct comparison [8]. Lowering one target-neutral workload through shared compiler stages provides a narrower but reproducible basis for examining compiler-visible effects. It can reveal instruction composition, CRF use, host-command expansion, register pressure, address-field use, and target-specific feasibility conditions. This thesis therefore evaluates the compiler's ability to generate and compare programs for different target configurations, leaving their runtime performance to subsequent simulation or hardware evaluation.

2.6 Summary

Autoregressive inference motivates compute-near-HBM because small-batch decode repeatedly applies large weight matrices at low arithmetic intensity, while the growing KV state adds further memory pressure. Near-bank processing can exploit internal bandwidth for suitable operations, but it forms part of a heterogeneous system rather than replacing compute-oriented accelerators. Its usefulness depends on software that can place data, select legal device operations, manage scarce architectural state, and generate the host protocol that triggers execution. The comparison between FIMDRAM AAM and HD-FIMDRAM makes this software problem concrete. A change in the relationship between data location and register selection can require changes to the compilation strategy, not just a different hardware parameter. The two models therefore serve as architectural cases through which to investigate a shared backend. The aim is to preserve what remains common while making the consequences of each target choice observable. The next chapter explains the memory and execution mechanisms behind this example, then develops the MLIR and Cinnamon abstractions on which the compiler builds.

Chapter 3

Background

Chapter 2 established why FIMDRAM compilation depends on memory organization, device execution semantics, and host command generation. This chapter supplies the technical foundation for those dependencies without yet presenting the design introduced by the thesis. Section 3.1 reviews DRAM and HBM organization and describes Samsung's original HBM-PIM architecture alongside Kukić's modified host-driven model. Section 3.2 introduces MLIR's representation and rewriting mechanisms, then explains the backend decisions and artifacts used in the later chapters. Section 3.3 then develops Cinnamon's compilation flow, workgroups, and memory abstractions, establishing what the existing infrastructure provides at the entry to the new backend. Finally, Section 3.4 compares programming systems, compilers, and evaluation infrastructures that address complementary parts of the PIM software stack, before stating the precise boundary between prior work and this thesis.

3.1 Architectural Foundations

The terminology follows Chapter 1: CIM denotes computation that exploits memory-cell or array behavior, while CNM denotes programmable digital units outside the cell array. PIM covers the wider field. The architectures examined below belong to the CNM category; their compilation requires explicit instructions and host commands rather than a mapping onto analog computation [5], [11].

3.1.1 DRAM Access and Address Structure

Dynamic random-access memory partitions storage into banks and each bank into rows and columns. Bank groups further collect banks that share command or datapath resources [11]. Although a processor presents a flat physical address, the memory controller translates it into a

tuple of memory coordinates and issues the command sequence required by the selected DRAM organization [28]. An access begins by activating a row in a bank. Activation transfers the row's contents into the bank's row buffer, after which read or write commands select columns from the open row. A precharge closes the row before a different row can be activated in that bank. Reusing an open row avoids the precharge and activation sequence, whereas accessing a conflicting row requires both operations [23], [29], [30]. These commands and coordinates become compiler-visible in a host-driven PIM design because a memory request can identify both the data location and the device action that uses it.

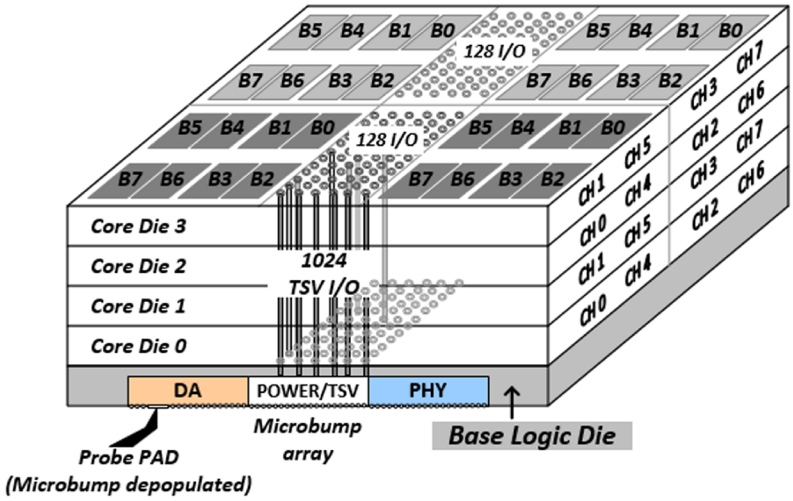
Address mapping also determines which requests can proceed independently. Bits assigned to channels, bank groups, and banks distribute accesses across parallel resources; row and column bits select locations within a bank. Changing the mapping can improve one access pattern while degrading another, and a PIM-oriented non-interleaved layout may conflict with the interleaving preferred by a host processor [22], [27]. A compiler targeting near-bank execution must therefore distinguish logical tensor indices, physical placement, and the final encoded address.

3.1.2 High-Bandwidth Memory

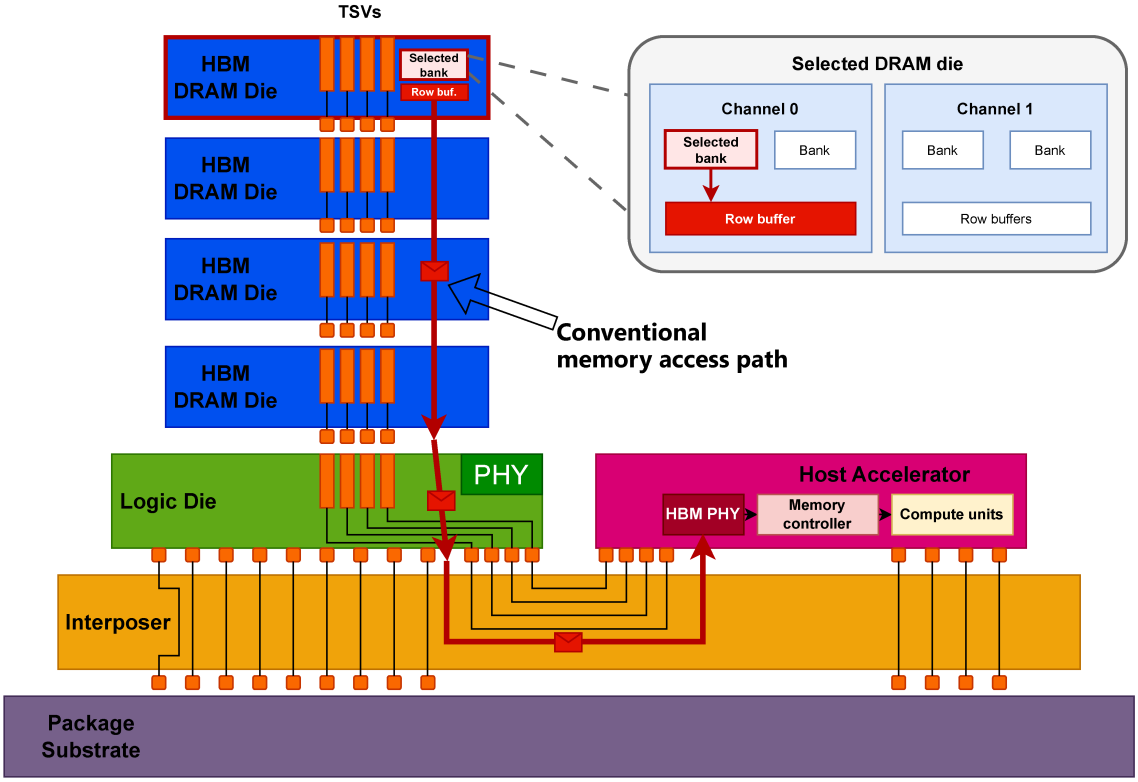
HBM follows an interface standardized by JEDEC, the organization that coordinates semiconductor-memory standards. Memory vendors and processor designers jointly developed this specification so that memory devices and host controllers could implement a common interface [9], [31]. The memory combines a base logic die with vertically stacked DRAM dies connected by through-silicon vias (TSVs), as shown in Figure 3.1a. The base die contains the physical interface and supporting logic, while wide links connect the stack to a nearby processor through a silicon interposer. This organization provides many data signals at moderate per-pin rates rather than relying only on a faster narrow interface [9]. The stack exposes multiple independent channels. Second-generation high-bandwidth memory (HBM2) further allows a channel to operate as two pseudo-channels with partially independent command interfaces, increasing command-level parallelism [9].

Within each pseudo-channel, conventional DRAM structures remain: banks hold cell arrays and row buffers, bank groups share portions of the datapath, and row and column commands select data. The wide internal path and replicated banks create substantial aggregate bandwidth, but a host can consume only the traffic carried by the external HBM interface. Near-bank PIM exploits the difference by attaching arithmetic to internal bank input/output (I/O) paths [9], [11]. Figure 3.1b traces the conventional path from a bank to host-side arithmetic. The bank, stack interface, and host memory interface remain distinct parts of this path even when they share a package. This is the transfer that near-bank execution seeks to shorten [9], [11].

The exact hierarchy is part of the architectural contract. A PIM program may need to identify



(a) HBM stacked-DRAM architecture, reproduced from Jun et al. [9, Fig. 1].



(b) Conventional operand path from a selected HBM bank to host-side compute units. Diagram drawn by the author using the Rambus illustration reproduced in Mutschler's article as a visual reference [32, Fig. 1].

Figure 3.1: Organization of an HBM stack and the conventional operand path from a DRAM bank to host-side compute units.

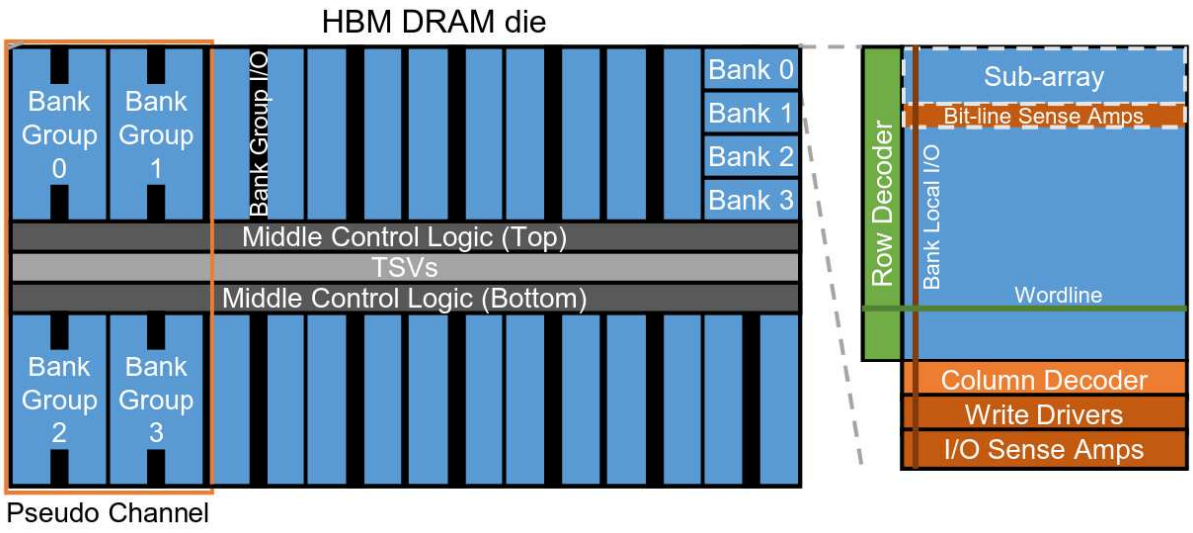
a pseudo-channel, bank group, bank parity, row, column, and byte offset, while the interpretation of those fields can change with the execution mode. For this reason, the backend cannot safely treat a target address as an opaque integer until placement and target-specific field selection have been resolved.

3.1.3 Samsung HBM-PIM and FIMDRAM

Samsung implemented its original PIM architecture using commercial HBM2 technology. In the discussion of this Samsung design, PIM-HBM, HBM-PIM, Aquabolt-XL, and FIMDRAM refer to the same architecture. More specifically, Aquabolt-XL is the name of the fabricated device that implements it [10], [11], [21]. This thesis uses *FIMDRAM* for the original architecture and its AAM target. It uses *HD-FIMDRAM* for Kukić's modified host-driven model, introduced below.

Compatibility with the standard interface is a central choice in Samsung's design. The memory controller resides on the host side, in a processor or accelerator, rather than in the HBM DRAM dies. A memory vendor can modify the device, but cannot assume corresponding changes to this controller [9], [11], [21]. Samsung therefore uses standard DRAM commands and timing (preserving compatibility with the JEDEC HBM2 interface [11], [31]) to trigger bank-local computation without redesigning the host controller. The authors demonstrate this approach by integrating their fabricated device with an unmodified commercial processor [11]. This compatibility also constrains execution: PIM must accommodate the commands and scheduling decisions of the existing controller [11]. Preserving this interface requires Samsung to express computation through commands designed for ordinary memory access. Reserved command sequences for mode transitions and address-derived register selection provide the additional control needed for PIM without extending the host controller's command set [11]. These mechanisms make integration with existing processors possible, but impose additional constraints on how software prepares and triggers computation. Figure 3.2 compares the conventional HBM organization with Samsung's bank-local design. The upper image shows the conventional die and bank structures; the lower image locates the added processing units between banks and shows their registers and arithmetic units at the bank I/O boundary [11, Figs. 2 and 1(a)–(b)]. The comparison follows the same two levels of organization in both images: the die on the left and one bank on the right. Unlike the host-side path in Figure 3.1b, this arrangement lets the processing unit consume bank operands without sending them through the stack interface to host arithmetic units [11, Section III-A]. For the compiler, the location of the data therefore determines which processing unit can use it.

Conventional HBM



Samsung FIMDRAM

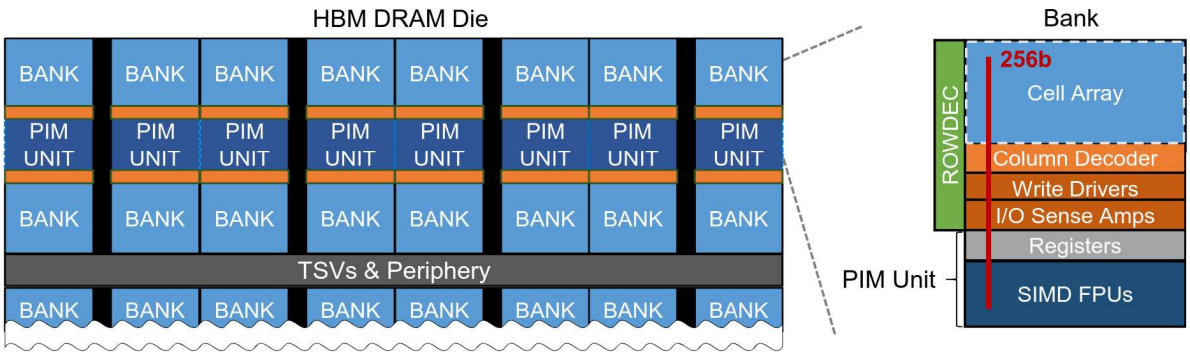


Figure 3.2: Conventional HBM and Samsung FIMDRAM at die level (left) and bank level (right). Top: conventional bank organization, reproduced from the lower part of Lee et al.'s Figure 2. Bottom: processing units added between banks, with registers and arithmetic connected at the bank I/O boundary, reproduced from their Figure 1(a)–(b) [11]. The comparison highlights where bank-local computation replaces operand transfers to the host; the original detailed datapath panel is omitted.

Processing Unit and Local State

One processing unit is shared by a pair of even and odd banks and can consume data from one bank of the pair at a time. Its arithmetic datapath contains 16 16-bit floating point (FP16) lanes, matching a 256-bit bank access. Separate multiplication and addition resources support vector arithmetic, while a controller selects operands and manages write-back [11], [26], [33]. The processing unit exposes three forms of local state. The general register file (GRF) contains 16 vector registers of 256 bits, partitioned as eight general register file A (GRF-A) and eight general register file B (GRF-B) entries between the even and odd banks. The scalar register file (SRF) supplies replicated scalar operands to the single-instruction, multiple-data (SIMD) lanes and is divided between multiplication and addition uses. The CRF holds 32 instructions, each encoded in a 32-bit word, and the controller advances a PIM program counter through this microprogram [26], [33]. These capacities are not merely descriptive hardware parameters: they bound register assignment, microprogram formation, and binary encoding.

The compact instruction set comprises control-flow operations (NOP, JUMP, and EXIT), arithmetic operations (ADD, MUL, MAC, and MAD), and data movement operations (MOV and FILL). Arithmetic instructions select operands from bank data, the GRF, or the SRF according to the instruction format. The restricted operation set is effective for multiply–accumulate patterns but does not natively cover every tensor operation; comparisons, general reductions, and data-type widening require decomposition or external execution [26], [33].

Operating Modes and Host Control

Samsung's three operating modes determine how the device interprets a host memory request: as an ordinary access, a parallel access across banks, or a trigger for computation. Figure 3.3 shows this progression. In *single-bank (SB)* mode, shown in panel (a), the device behaves as conventional HBM: the bank-selection fields identify one bank, and the host reads or writes its contents. In *all-bank (AB)* mode, shown in panel (b), a command accesses corresponding row and column locations across participating banks in the selected pseudo-channel. This parallel access supports initialization across banks, but does not itself execute the microprogram stored in the CRF [11, Section III-B] [26, Section 3]. *All-bank processing-in-memory (AB-PIM)* mode adds instruction execution to this bank-parallel access. In panel (c), a host read or write request produces a column command that triggers the current CRF instruction across the processing units. When the instruction uses bank data, the request also identifies the data location. The units perform the selected operation locally, and their program counters advance according to the microprogram's control flow. The host must therefore continue issuing the requests that drive execution; loading a microprogram into the CRF alone does not run it [11, Sections III-B and IV].

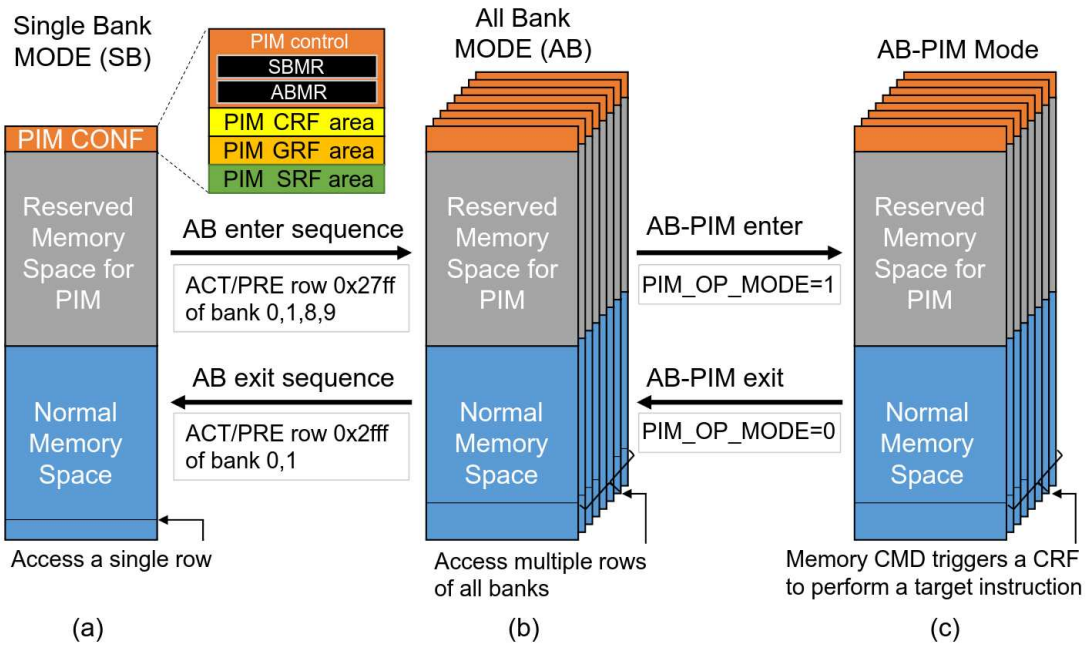


Figure 3.3: Samsung FIMDRAM operating modes: (a) SB provides ordinary accesses to individual banks; (b) AB accesses corresponding locations across banks in parallel, without executing the microprogram; (c) AB-PIM uses memory commands to trigger CRF instructions across the processing units. The arrows distinguish the reserved `ACT`/`PRE` sequences for switching between SB and AB from writes to `PIM_OP_MODE` for enabling or disabling AB-PIM. `PIM CONF` exposes control and register-file areas through reserved addresses. Reproduced from Lee et al. [11, Fig. 3].

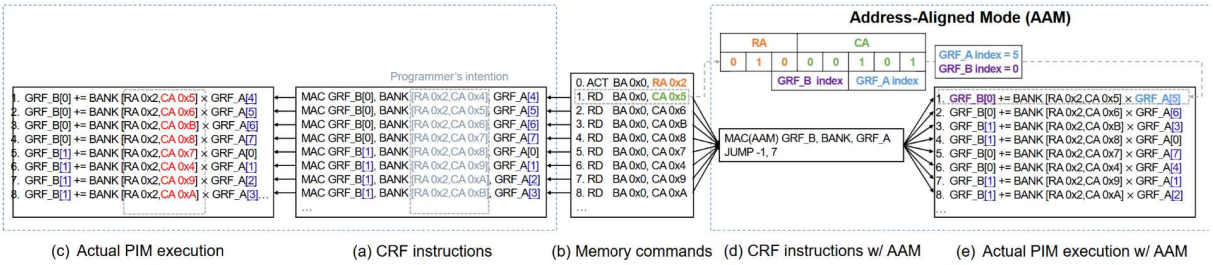
The arrows in Figure 3.3 show how the host changes this interpretation without introducing new DRAM commands. To enter AB mode from SB, it issues a reserved sequence of `ACT` and `PRE` commands to configuration addresses in `PIM CONF`. Returning to SB also closes the open bank rows. From AB, setting the mapped `PIM_OP_MODE` register to one enables AB-PIM; setting it to zero returns to AB. Samsung uses these memory-access sequences rather than privileged mode-register commands to avoid a kernel context switch for each transition [11, Section III-B and Fig. 3]. To execute a kernel, host software prepares its data and loads the device microprogram, enters AB-PIM through AB, issues the required computation triggers, and leaves computation mode before retrieving the results [26, Sections 4–5]. A FIMDRAM program therefore combines device instructions with the host transactions that supply operands and trigger them; the compiler must preserve this correspondence when scheduling and encoding the program [11, Section V-B]. Samsung's driver reserves PIM memory and supplies physically contiguous allocations, while its runtime manages that memory, prepares microkernels, and invokes execution [11, Section V-A]. Section 3.1.4 explains how runtime calls use driver-established mappings to control the device in Kukić's system.

Address-Aligned Mode

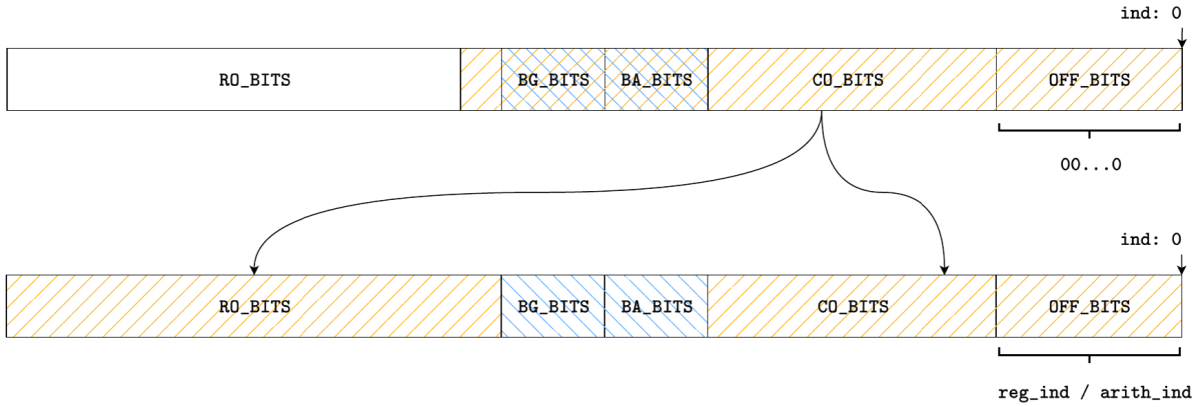
AAM is an instruction-level choice during PIM execution, rather than a fourth device operating mode. Samsung enables it through a flag on arithmetic instructions [11, Section IV-C]. An unmodified host controller can reorder memory requests to improve ordinary DRAM access. In conventional CRF execution, however, each request supplies the bank operand for the next instruction in the microprogram. If the controller changes the request order, an instruction can receive the wrong operand. Memory fences can preserve the required order, but they also restrict the controller's scheduling freedom and can reduce performance [11], [21], [26]. Address-Aligned Mode changes the binding for arithmetic instructions. An AAM instruction ignores its explicit register-index fields; instead, the device derives the effective source and destination register indices from selected row and column address fields of the triggering request. The operation and its register selection therefore remain associated even if requests are reordered [11], [26]. For suitable data-parallel kernels, AAM thus tolerates some request reordering without changing the host controller. The compiler must instead make register assignment agree with physical address selection, while retaining the ordering required by data movement operations [11], [21]. Figure 3.4a illustrates the relationship between memory commands, CRF instructions, and effective register indices. Its comparison of execution with and without AAM shows why the address becomes part of operand selection, rather than identifying only the bank data [11, Fig. 5].

3.1.4 HD-FIMDRAM: Kukić's Modified Host-Driven Model

Kukić developed a compiler-oriented generalization of FIMDRAM in which the host remains responsible for triggering each PIM operation, but several instruction set architecture (ISA) and addressing choices differ from FIMDRAM AAM. The modified arithmetic model uses the data selected by the triggering memory command as an implicit bank operand and represents the remaining operand choices explicitly. It also separates compiler register allocation from Samsung's address-derived AAM selection [21]. The associated command encoding begins from an aligned DRAM address. Low-order offset bits, which are zero in the aligned data address, carry a register identifier for data movement or a static CRF index for arithmetic. The row and column location of the bank operand remains distinct from this payload. The scheme gives the compiler a direct binding between a host request and the selected register or instruction, but only when the available offset width can represent the payload without violating alignment [21]. Figure 3.4b shows how the compiler maps a logical access to physical row and column fields and uses the low-order offset for a register or arithmetic instruction index. Unlike the AAM example, this path retains an explicitly selected command index alongside the data location [21, Fig. 4.10]. This payload also constrains how the host accesses memory. A computation trigger must reach



(a) FIMDRAM instruction execution and address-derived register selection in AAM. Reproduced from Lee et al. [11, Fig. 5].



(b) Logical-to-physical address transformation and encoding of register or arithmetic instruction indices in the low-order offset bits of the host-driven model. Reproduced from Kukić's Master's thesis [21, Fig. 4.10].

Figure 3.4: Address-derived register selection in FIMDRAM AAM and compiler-selected command indices in HD-FIMDRAM.

the device rather than be satisfied by a host cache. Kukić uses non-cacheable trigger accesses for this purpose; in his ARM system simulated with gem5, these requests execute serially and limit the number of memory requests serviced concurrently [21, Section 6.2.2]. He also explains why using cacheable triggers would require a different encoding: cache-line requests do not preserve the low-order byte-offset distinctions that identify registers and instructions in his model. His proposed alternative uses an AAM-like mechanism and requires changes to address-aware tiling and register assignment [21, Section 6.4.1]. Changing the host-access mechanism therefore changes the compiler decisions needed to prepare the device program, not just the way the host sends commands.

Table 3.1 summarizes the compiler-visible difference. Both models use host-issued memory transactions and bank-local SIMD execution, yet they assign architectural meaning to different address fields. This difference propagates into placement, register assignment, instruction selection, feasibility checks, and command encoding. Reading across the table, the HD-FIMDRAM model exposes compiler-selected register identities and low-order payload fields, whereas FIM-

DRAM AAM derives register selection and broadcast behavior from structured physical addresses. The shared rows therefore identify reusable stages; the differing rows identify the policies and checks that cannot be collapsed.

Table 3.1: Compiler-visible binding in the two source-derived FIMDRAM models.

Property	**FIMDRAM**	**HD-FIMDRAM**
Arithmetic trigger	HBM column command advances or replays a CRF instruction	Host memory command selects a static CRF entry
Register binding	Derived from structured row and column address fields	Selected by the compiler and carried in low-order command bits for data movement
Address relationship	Register choice is coupled to physical data placement	Command payload is separated from the aligned row/column location
Principal static check	Address fields select the required registers	Register and CRF identifiers fit the available offset payload

Runtime and Driver Support

Kukić's ARM-based full-system simulation provides a concrete example of how compiled accesses reach device memory. The system reserves a physical PIM region outside ordinary operating-system allocation. Figure 3.5 shows how character device drivers expose this region in the application's address space through `mmap`. The two upper paths create a cacheable mapping for data transfers and a non-cacheable mapping for computation triggers; both refer to the same physical memory. A third driver maps the device's configuration space, through which host software controls the operating mode [21, Section 3.4]. The application therefore uses driver-established virtual mappings rather than choosing arbitrary physical addresses. Because the two data mappings have different cache attributes, software must explicitly flush cached data when necessary to make it available to the device [21, Section 3.4].

The runtime allocates buffers within the reserved region, arranges their data in the layout required by the kernel, loads the device program into the CRF, invokes the compiled kernel, and retrieves the results [21, Section 5.1.4 and Fig. 5.3]. Kukić's compiler also lowers control operations marking the load, compute, and store phases to runtime calls that select the device mode [21, Section 4.5.8]. These calls use the mapped configuration space, so changing mode does not require a kernel context switch on every transition [21, Section 3.4]. The compiler and runtime must agree on the physical layout behind the mappings, since memory-address fields identify both data and device actions [21, Sections 3.3–3.4]. This separation connects compiled programs to execution without making the compiler responsible for the process's virtual-memory mappings. Section 5.6 defines the artifact boundary used by the backend developed in this thesis.

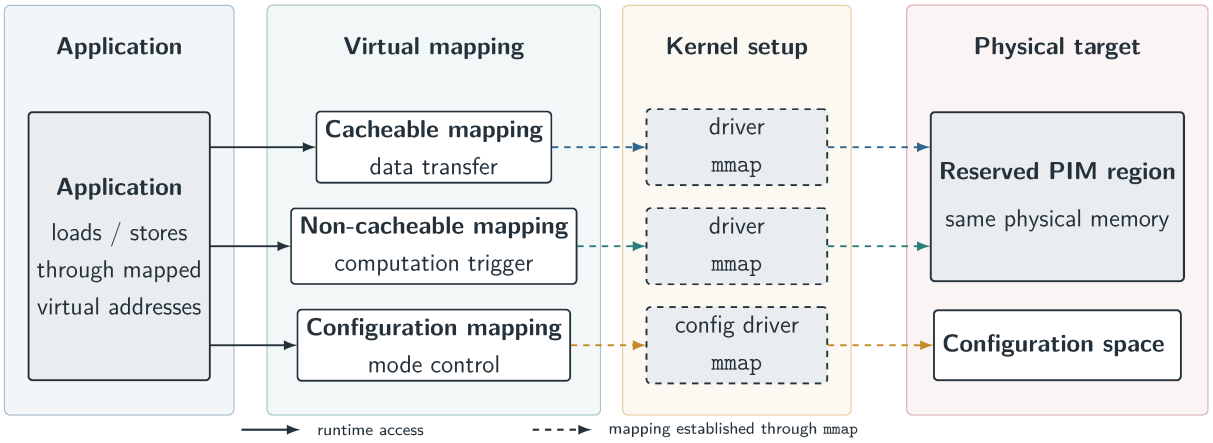


Figure 3.5: Runtime access to device memory in Kukić's system. Solid arrows show the runtime using virtual mappings; dashed arrows show the mapping relationships established by kernel drivers through `mmap`. Cacheable data transfers and non-cacheable computation triggers reach the same reserved PIM region, while a separate mapping exposes the configuration space for mode control.

The two descriptions are sources for this thesis, not contributions of it. Samsung defines the original device and AAM behavior; Kukić defines the modified host-driven model and its compiler strategy. The contribution developed in later chapters is their source-traceable integration as selectable targets within a shared Cinnamon backend.

3.2 Compiler Infrastructure

The architectural model above combines tensor semantics, explicit memory placement, device instructions, and a host execution protocol. No single low-level representation is well suited to all four concerns. This section introduces the multi-level compiler infrastructure used to preserve the appropriate information at each stage and to lower it only when the required target decisions are available.

3.2.1 MLIR's Extensible Intermediate Representation

An intermediate representation (IR) represents a program for analysis and transformation at a chosen level of abstraction, from tensor operations and structured loops to target instructions [19]. MLIR is a compiler infrastructure for defining, transforming, and connecting intermediate representations at different abstraction levels. It standardizes a static single assignment (SSA)-based structural model and supplies common infrastructure for parsing, printing, diagnostics, source locations, passes, and pattern rewriting. Its central extension mechanism is the *dialect*: a namespace containing operations, types, and attributes whose semantics are defined

for a domain or target [19]. An MLIR operation can consume and produce typed SSA values, carry immutable attributes, and contain regions composed of blocks and nested operations. Types describe compile-time properties of values; attributes attach compile-time information to operations. This distinction is useful for accelerator backends: a buffer can remain an SSA value while an address-space choice, target identifier, or encoded field is represented as compile-time metadata [5], [19].

MLIR permits operations from different dialects to coexist in one module. Each operation, type, and attribute belongs to one dialect, but values and regions can cross dialect boundaries. A compiler can therefore retain a high-level tensor operation while introducing lower-level memory or control operations around it, then remove the high-level construct when its semantics have been fully transferred [19], [34]. This mixed-dialect model supports progressive lowering without requiring one monolithic IR to encode every abstraction simultaneously. Figure 3.6 identifies the structural elements that make this mixed-dialect composition possible. The example operation combines a dialect-qualified name, typed SSA operands and result, attributes, and a source location; its optional region contains blocks and further operations. The lower arrow shows the key composition mechanism: a typed SSA value produced in one dialect can be consumed by an operation in another, so lowering need not replace an entire module at once.

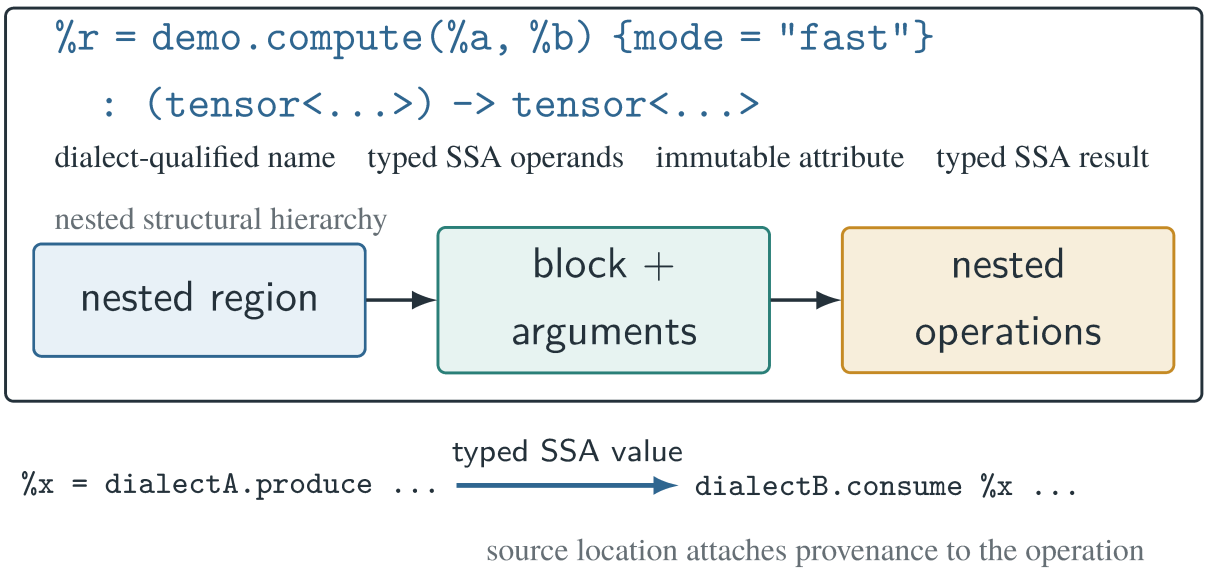


Figure 3.6: Structural elements of an MLIR operation and composition across dialect boundaries.

3.2.2 Passes, Rewriting, and Progressive Lowering

MLIR transformations are organized as passes and rewrite patterns. A pass analyzes or transforms an operation hierarchy, while a pattern describes a local replacement subject to semantic

and structural conditions. Dialect conversion coordinates sets of such rewrites and distinguishes operations that are legal in the target state from those that must be eliminated. These mechanisms allow one stage to lower only the concepts for which it has enough information while preserving unrelated dialects [19], [29]. These mechanisms separate the representation of a computation from the point at which each hardware decision is made. They do not themselves define the semantics of a PIM device: those belong to the dialects and conversions built on top of MLIR [5], [19]. Lowering progressively replaces a representation with one that makes more implementation decisions explicit. Selecting an instruction, assigning a register, and encoding that register into a binary word are therefore different decisions, even when they concern the same source operation. Progressive lowering preserves the information needed for each decision instead of encoding everything at once [19].

3.2.3 Instruction Selection, Ordering, and Registers

Instruction selection determines how a computation uses the operations offered by a target. Its input describes the intended arithmetic and available operands; its output specifies a supported instruction and operand form. An operand form identifies where an instruction obtains its inputs and places its result, not merely whether the operation is an addition or multiplication. For example, an input may come from a register or from the bank access that triggers execution. Two instructions that perform the same arithmetic can therefore impose different register and memory requirements [11], [21]. Scheduling determines the order of the selected work. A data dependency requires an operation that produces a value to precede the operation that consumes it; memory ordering imposes further restrictions. A scheduler can move operations only while preserving the required dependencies. Synchronization complements this ordering: a memory fence establishes an ordering boundary. This matters especially for host-triggered execution, where an instruction order alone does not ensure that memory requests reach the device in the required order. The instruction reordering and fence passes in Kukić's compiler address these related but separate concerns [21]. Memory barriers constrain the ordering of memory operations, while thread barriers synchronize participating threads. Kukić's `cnm.fence` belongs to the former category; Cinnamon's UPMEM path also provides thread synchronization [5], [21]. In this thesis, fences in the selected program record required ordering between phases or operations. Section 5.4.2 describes their target-specific treatment.

Register allocation assigns instruction values to a finite set of physical registers. A value remains live while a later operation still needs it. Values with overlapping lifetimes cannot freely share one register, whereas the allocator may reuse a register after its previous value is no longer needed. Register pressure concerns this demand (and not the total number of values produced over the whole program). Scheduling affects that demand by changing the distance between def-

initions and uses. Allocation must also respect tied operands, which must share an assignment, and register-bank constraints, which restrict the banks available to an operand. Kukić's allocator groups tied values and uses their live ranges to construct the interference graph for register assignment [21]. Section 5.4.3 explains the assignment strategies implemented in this thesis. The live-group and physical-register measurements are defined separately in Section 4.5.3.

3.2.4 Layout, Addresses, and Placement

A tensor index identifies an element in the computation, but does not by itself specify a memory bank or processing unit. A layout describes how tensor elements are arranged in storage; logical indexing uses that arrangement to derive a linear position from tensor coordinates. A buffer-relative offset specifies a displacement from the start of a buffer; the buffer identifier records which buffer that displacement belongs to. Cinnamon's distributed buffers and local memory views preserve the distinction between the data visible to a computation and its eventual device-specific organization [5]. The buffer and launch abstractions are developed in Section 3.3.3. In the backend studied here, address assignment connects logical accesses to the target's address representation. Logical-to-physical mapping chooses the transactions used for those accesses, and address decomposition identifies their row, column, bank, and other target fields. Placement assigns execution coordinates, identifying the processing unit and local bank without changing the assigned physical address. Here, "physical mapping" concerns modeled device address fields rather than operating-system page translation; addresses remain buffer-relative. The runtime and driver roles described in Section 3.1.4 connect compiled accesses to memory mapped into an application's address space. Section 5.4.3 describes the interfaces, while Section 5.5.2 explains how mapping constrains register identities under AAM.

3.2.5 From Compiler Decisions to Output Artifacts

Program materialization constructs structured records of selected instructions and their operands in this backend. Later passes add addresses, registers, and placement before binary encoding packs the required fields into device words. Host-command generation produces the separate commands needed to supply data and trigger execution. Neither a selected instruction record nor a host command necessarily corresponds to one encoded device word. Table 3.2 separates these inputs and outputs; the example in Section 5.7 follows a vector addition through the actual target-specific representations. Each stage also needs checks on the representation it receives and produces. Section 4.3.3 explains how profiles, policies, and recipes organize these decisions in the proposed backend, while Section 4.4 defines its verification layers. Metric definitions remain in Section 4.5.3. The following section explains how Cinnamon supplies the shared computation

and memory abstractions from which this backend starts.

Table 3.2: Roles of the main backend stages used in this thesis. The rows summarize responsibilities, not a universal pass order.

Stage	Input	Decision	Output
Instruction selection	Computation and operand roles	Supported operation and form	Selected operations
Program materialization	Selected operations	Record instructions and value references	Structured program
Scheduling and fences	Program and dependencies	Legal order and phase boundaries	Ordered program
Address assignment	Logical accesses and layout	Target mapping and address fields	Addressed accesses
Register assignment	Values, lifetimes, constraints	Physical register bindings	Assigned operands and results
Placement	Addresses and target topology	Execution coordinates	Processing-unit and local-bank fields
Artifact emission	Planned program and target rules	Encode and form host commands	Device words and host artifacts

3.3 Cinnamon and the CNM Compilation Model

CINM, referred to as Cinnamon in this thesis, uses MLIR to compile programs for heterogeneous compute-in-memory and compute-near-memory devices. Its purpose is to separate the computation from the device-specific code needed to allocate resources, move data, and execute operations. The paper demonstrates this approach with memristor-based CIM accelerators and UPMEM CNM systems, while defining extension points for other devices [5]. Understanding those shared abstractions is necessary to identify what the FIMDRAM backend receives and what it must add.

3.3.1 From Tensor Operations to Device Code

The left-to-right flow in Figure 3.7 starts from operations expressed through existing MLIR abstractions such as `linalg`, `tosa`, and `torch`. Conversions bring supported computations into the `cinm` dialect: it represents operations such as matrix multiplication, element-wise arithmetic, and reductions before committing them to a particular implementation. Operations that cannot be converted remain on the host [5, Sections 3.1–3.2.2]. The central branches separate the two

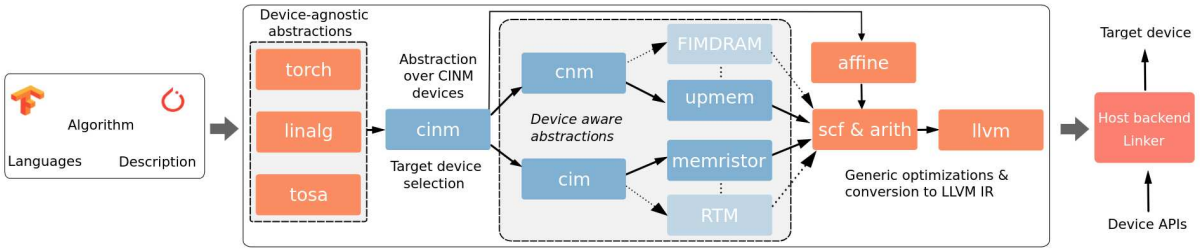


Figure 3.7: Cinnamon's progressive lowering from device-agnostic operations to shared CNM and CIM abstractions, device dialects, and host code generation. The pale device boxes indicate extension targets; the FIMDRAM path is proposed in the original paper and developed by this thesis. Reproduced from Khan et al. [5, Fig. 4].

memory-centric execution models. The `cim` dialect describes resource management and execution for compute-in-memory devices, while `cnm` describes distributed memory and computation for near-memory units. Device dialects then translate these shared operations into the instructions or runtime calls required by a concrete target. For example, the UPMEM path ultimately uses its device runtime rather than treating a generic CNM operation as an instruction that the hardware can execute directly [5]. The `affine` branch and the lower-level `scf`, `arith`, and `llvm` dialects connect this device-aware lowering to the host compilation flow and device interfaces shown on the right [5, Fig. 4 and Section 3.1]. The pale FIMDRAM box marks an extension envisaged by the original paper, rather than an implemented backend in that work. This thesis develops that device-specific path below the shared `cnm` boundary [5, Section 3.2.5]. Table 3.3 summarizes the responsibilities of these levels. Their separation lets the compiler transform the computation before deciding how a particular device will execute it, without assuming that all devices expose the same memory or control interface [5].

Table 3.3: Responsibilities of Cinnamon's abstraction levels, summarized from Khan et al. [5].

Level	**Responsibility**
`cinm`	Represent supported computations and provide the point at which target selection takes place.
`cnm`	Express workgroups, distributed buffers, transfers, and near-memory execution independently of a device API.
`cim`	Express resource acquisition, data movement, and execution for compute-in-memory devices.
Device dialect	Resolve shared operations into device-specific representations and code generation.

3.3.2 Workgroups and Memory Visibility

The central abstraction in `cnm` is the workgroup. It describes a logical organization of processing units and memory resources rather than a fixed collection of physical banks. Memory forms a tree, with processing units at its leaves and global memory at the root. A unit can access the memory spaces on its path to the root, but not arbitrary memory belonging to another branch [5, Section 3.2.3]. For a workgroup indexed by (i,j,k), the paper illustrates memory private to one unit, memory shared by units with the same (i,j) coordinates, and a larger space shared by units with the same i coordinate. This organization makes data visibility part of the representation before the compiler maps logical units onto hardware. A buffer shared by a group and a buffer private to one unit therefore have different roles even when neither has a final physical address [5, Fig. 7]. This separation allows the compiler to transform the logical workgroup independently of the device's physical resource dimensions. For independent processing units, the compiler can split, combine, or interchange workgroup dimensions while preserving each unit's computation. Such transformations can change where buffers are shared and how much memory or data movement the resulting organization requires. Tiling then divides work into pieces that fit the device resources [5, Section 3.2.3]. The workgroup thus connects parallel computation to memory distribution without requiring the application to name physical banks. The paper illustrates this with a tensor contraction that, writing the reduction explicitly, has the form

$$x_{ijk} = \sum_{r} A_{ir} B_{rjk} + C_{jk}.$$

The output indices (i,j,k) define independent workgroup coordinates. Each logical processing unit needs the row $A_{i,:}$, the slice $B_{:,j,k}$, and the scalar C_{jk} to compute its result. The reduction over r stays within that unit, rather than introducing a dependency between units. Changing the workgroup organization changes how these inputs are distributed or shared, while retaining the computation associated with each output [5, Section 3.2.3 and Fig. 8]. This is the distinction between describing the available parallel work and selecting the physical resources that will execute it.

3.3.3 Buffers, Transfers, and Launches

Cinnamon separates host-side management of distributed storage from the computation that accesses it. At the shared CNM level, allocation provides opaque buffers rather than ordinary memory references that the host can index directly. Transfers use `cnm.scatter` and `cnm.gather`; inside a workgroup's `cnm.launch`, the buffers become device-local memory references available to the computation. The dialect also provides `cnm.wait` for synchronization [5, Section 3.2.3

and Table 2]. This distinction gives a launch more meaning than a request to call a function. It connects a computation to a workgroup and to the data visible within that workgroup. Outside the launch, the compiler handles allocation and transfers through the distributed-buffer abstraction. Inside it, the computation uses the corresponding local views. These two representations allow the compiler to change the distribution of storage without embedding physical addresses throughout the source computation [5, Section 3.2.3].

The convolution example in the paper shows this separation in a complete lowering: the CNM representation creates a workgroup, arranges the buffers and transfers, and launches computation on the distributed data. Tiling changes the amount of work assigned to the units without replacing this host–device structure [5, Fig. 6(a)]. The example is useful here because it distinguishes what the common CNM representation already expresses from what a device backend must still decide: the logical organization is present, but the device's register and command encodings are not.

3.3.4 Target Selection and Device Extensions

Cinnamon also separates reusable transformations from the device information that guides them. Its generic tiling interface can be invoked by lower-level dialects, which select tile sizes using architectural constraints. The transformation is shared, but the choice of tile is device-aware [5, Section 3.2.6]. Reuse therefore does not mean removing hardware decisions; it means giving them an explicit place in the flow. Cinnamon provides interfaces through which device implementations can participate in lowering and supply cost models. In the version described by the paper, target selection uses user choices or analysis-based rules; the development of general cost models and search mechanisms remains outside that work's scope. The existence of an interface for comparing devices therefore should not be confused with an implemented optimizer that finds the best mapping for every workload [5, Sections 3.2.2 and 3.3].

A new target normally adds a device dialect and conversions from the shared abstractions. The paper uses FIMDRAM as an example: its device representation would need arithmetic operations with the appropriate operand sources, control operations, and barriers, together with a conversion from `cnm`. Because the corresponding computations are already expressible at the higher level, this extension need not replace the shared `cinm` and `cnm` abstractions [5, Section 3.2.5] (this is an extension strategy described by the paper, not the FIMDRAM backend contributed by the present thesis). The backend studied here takes a target-neutral `cnm.launch` as its entry point. At that boundary, Cinnamon has a representation of the buffers and computation intended for near-memory execution. The new extension must translate them into a representation that can resolve the register, address, instruction, and host-command requirements of either FIMDRAM AAM or HD-FIMDRAM. The shared abstraction and the target-specific low-

ering therefore have separate responsibilities: Cinnamon describes the CNM work; the backend determines how the selected FIMDRAM model realizes it. Chapter 4 develops the requirements for that extension.

3.4 PIM Related Work

PIM programmability involves several software layers. A programming interface can hide device allocation and communication without generating a hardware microprogram; a compiler can map tensor operations without solving cache coherence; a simulator can execute generated commands without providing the abstractions that produced them. For this reason, the following works are grouped according to their main software role.

3.4.1 Programming Interfaces and Host–PIM Integration

PIM-enabled instructions (PEIs) integrate simple PIM operations into the host instruction set and allow hardware to choose host or memory-side execution according to data locality. Restricting an operation to one cache block lets the proposal reuse existing virtual-memory and coherence mechanisms while preserving a conventional sequential programming model [35]. PEIs address transparent invocation and placement of fine-grained operations; they do not target the bank-pair microprogram and host-command formation used by FIMDRAM.

SimplePIM raises the programming level for UPMEM by managing PIM-resident arrays and exposing host-invoked map, reduce, and zip iterators. Its communication interface provides scatter, gather, broadcast, all-gather, and reduction-style primitives between the host and PIM cores or among PIM cores [36]. This design demonstrates the value of separating array management, communication, and computation, but its core abstraction is a distributed array framework for programmable UPMEM cores rather than a compiler backend for command-driven HBM-PIM.

Other systems focus on memory sharing between the host and PIM. FACIL supports multiple physical-to-DRAM address mappings so that PIM-optimized placement can coexist with contiguous virtual access by a system-on-chip (SoC) processor [22]. UM-PIM similarly combines interleaved central processing unit (CPU) pages and non-interleaved PIM pages through dual-track memory management and dynamic address mapping [27]. PIM-SHERPA addresses a related LLM deployment problem in software: prefill and decode prefer different memory attributes and weight layouts, so it uses buffering or online rearrangement to bridge them [37]. These systems make host-visible layout and address translation first-class concerns, complementing the compile-time placement and address generation studied in this thesis.

Cache coherence forms another system-level layer. LazyPIM executes PIM kernels speculatively, records compressed signatures of their read and write sets, and checks conflicts after a

kernel completes. On conflict, the system writes back relevant processor data and re-executes the PIM kernel; otherwise it commits the speculative updates [38]. This mechanism illustrates why a complete host–PIM system requires support beyond command generation. The FIMDRAM backend can emit ordering records and fences, but it does not implement a coherence protocol or shared virtual-memory subsystem.

3.4.2 Compilation and Mapping for PIM

PIMCOMP provides an end-to-end path from deep neural network (DNN) descriptions to pseudo-instructions for a configurable PIM accelerator template. Its multi-level optimizer handles model mapping, resource allocation, scheduling, and memory orchestration, and the pseudo-instruction layer can be translated to hardware-specific primitives [39]. PIMCOMP consequently addresses broad DNN deployment across an abstract accelerator family. The present thesis studies a narrower backend question at greater target detail: how two FIMDRAM command and addressing models share lowering infrastructure while retaining distinct binary and host-protocol semantics. The data-centric compiler by Yang et al. maps the PIM memory hierarchy into a compute hierarchy, constructs tensor and loop partitions together, and co-optimizes data rearrangement with compute code. Its backend abstraction supports different near-bank PIM systems, while backend-specific optimizers refine candidate schedules [40]. This work reinforces two principles relevant here: data distribution and computation cannot be optimized independently, and common abstractions must still leave explicit extension points for target-specific behavior.

IMTP and ATiM integrate tensor-level lowering with search-based code generation for UPMEM. Their search spaces include both host and PIM-kernel programs, and hardware constraints filter infeasible candidates before evaluation [41], [42]. Search is appropriate when many legal schedules and transformations require empirical ranking. The FIMDRAM work in this thesis instead asks whether source-derived target choices are represented correctly and comparably; it supports policy ablations but does not introduce an autotuner or claim to select a performance-optimal configuration.

SongC uses hierarchical graph and hardware abstractions for a hybrid near-memory/in-memory many-core architecture. Its composer, arranger, orchestrator, and mixer stages separate algorithm representation, spatiotemporal allocation, primitive generation, and scaling across the system [43]. SongC demonstrates how distinct abstraction levels can decouple otherwise coupled mapping problems. Its target is a hybrid many-core neural-network accelerator, whereas this thesis concentrates on instruction, address, register, and host-command policies within one FIMDRAM-oriented backend.

AME-PIM approaches HBM-PIM from the host ISA boundary by mapping matrix extension

instructions onto Aquabolt-XL microprograms. Its analysis shows that the compact FIMDRAM ISA expresses multiply–accumulate patterns but lacks native support for operations such as general reductions, comparisons, and data-type widening [33]. AME-PIM is directly relevant to instruction selection and operand routing, although its objective is to realize a matrix ISA on one HBM-PIM model rather than to compare alternative FIMDRAM execution semantics.

C4CAM provides another example of extending Cinnamon with a device-specific path, in this case for content-addressable-memory accelerators [44]. It complements the extension mechanism described in Section 3.3. The FIMDRAM work follows that general approach, but studies how distinct execution models can coexist below one CNM-facing boundary.

3.4.3 Architecture Exploration and Evaluation Infrastructure

Compiler decisions are meaningful only with respect to an architectural model. NicePIM jointly explores hardware parameters and DNN mapping because capacity allocation, topology, placement, and scheduling interact across a large design space [16]. Bank on Compute-near-Memory similarly studies processing-near-bank organizations and exposes how internal bandwidth, area, and workload mapping affect one another [15]. These works optimize or compare architecture points using performance models; the target profiles in this thesis instead provide executable compiler contracts for two fixed, publication-derived models and selected policy ablations.

Simulation frameworks cover different levels of detail, from functional operation models to full-system timing [8], [45]. This diversity makes results dependent on the simulator's modeled processor, memory system, workloads, and output metrics [8]. PIMSys specifically combines gem5 with DRAMSys to execute compiled host binaries against a Samsung PIM-HBM memory model and provides a Rust software library for PIM operations [26]. It supplies a route toward system-level validation of generated FIMDRAM programs, but it is not integrated as the timing model of the current thesis. The absence of that integration constrains the evaluation claims made later. The backend can compare generated instruction records, CRF occupancy, host commands, fences, register pressure, address fields, and static feasibility. It cannot infer latency or energy solely from those counts because the two execution models package work into different command structures and because memory scheduling contributes behavior not represented by a static total.

Table 3.4 compares the responsibilities of the reviewed systems and makes the narrower scope of this backend explicit. Filled and open marks distinguish principal from supporting responsibility rather than ranking the systems. The rows show that runtime-oriented systems cover allocation, invocation, coherence, or execution, while compiler and mapping systems more often own layout and device-program generation. Cinnamon provides the shared compiler substrate, and PIMSys provides a possible future execution environment; the thesis contribution occupies

the target-backend gap between them.

Table 3.4: Scope and responsibilities of software systems related to the FIMDRAM backend. A filled mark denotes a principal responsibility, an open mark denotes partial or supporting coverage, and a dash denotes functionality outside the system's stated scope.

System	Principal abstraction	Target class	Layout / address	Device program	Host / runtime	Execution model	Relation to this thesis
PEIs [35]	Host ISA operations with locality-based placement	Fine-grained PIM operations	○	–	●	○	Transparent invocation and coherence-compatible placement
SimplePIM [36]	Distributed arrays and data-parallel iterators	Programmable UPMEM cores	●	○	●	–	Productive allocation, communication, and kernel invocation
FACIL / UM-PIM [22], [27]	Shared address-space and mapping mechanisms	SoC–PIM systems	●	–	●	○	Host-visible physical mapping and page organization
LazyPIM [38]	Speculative coherence protocol	Cache-coherent host–PIM system	–	–	●	●	System support beyond compiler-generated fences
PIMCOMP [39]	DNN graph mapping and scheduling	Configurable PIM accelerator template	●	●	○	○	Broad mapping flow with hardware-specific translation
Yang et al. [40]	Tensor/loop partitioning and backend optimization	Near-bank PIM families	●	●	○	○	Data-centric common abstraction with target optimizers

Table 3.4: Scope and responsibilities of software systems related to the FIMDRAM backend (continued).

System	Principal abstraction	Target class	Layout / address	Device program	Host / runtime	Execution model	Relation to this thesis
IMTP / ATiM [41], [42]	Search-based tensor program generation	UPMEM	●	●	●	●	Autotuning of host and PIM programs
SongC [43]	Hierarchical graph and hardware composition	Hybrid NMP/IMP many-core accelerator	●	●	○	○	Separation of mapping and orchestration levels
AME-PIM [33]	Matrix-extension to microprogram mapping	Aquabolt-XL HBM-PIM	○	●	○	○	Directly relevant instruction selection and operand routing
Cinnamon [5]	Hierarchical memory-centric MLIR dialects	CIM and CNM device families	○	○	○	–	Shared compiler substrate extended by this thesis
NicePIM [16]	Joint architecture and DNN mapping exploration	Configurable PIM accelerators	●	○	–	●	Coupled target and mapping design space
PIMSys [26]	Full-system virtual prototype and library	Samsung PIM-HBM	●	○	●	●	Potential execution and timing validation environment

3.4.4 FIMDRAM-Specific Compiler Lineage

Samsung's architecture and software stack define the original processing unit, operating modes, instruction set, host-command interface, and AAM semantics [11]. Kukić subsequently developed an MLIR-based compiler framework for generalized CNM processing units and a modified FIMDRAM-like execution model. That work contributes the host-driven ISA, logical processing-unit abstraction, compiler-selected register allocation, low-order command encoding, host/device code generation, and a multi-stage verification strategy [21]. These ideas are prior work and remain attributed to their respective sources. The implementation studied in this thesis is not a merge of an existing Samsung backend and Kukić's repository. It is a new FIMDRAM target extension inside Cinnamon, informed by both publications. It re-expresses the relevant architectural semantics in Cinnamon's dialect and pass infrastructure, places the two models behind shared target-facing representations, and adds explicit profiles, policy interfaces, pipeline recipes, compatibility checks, and static comparison artifacts.

3.4.5 Positioning of This Thesis

Prior work establishes several complementary solutions: productive application programming interfaces (APIs) hide distributed PIM management; memory-system proposals reconcile host and PIM addressing; coherence mechanisms regulate shared data; tensor compilers search mapping and scheduling spaces; hierarchical infrastructures separate high-level operations from device code; and simulators execute architectural models. No single layer makes the others unnecessary. The problem addressed here lies at a specific boundary between Cinnamon's target-neutral CNM representation and FIMDRAM-specific executable artifacts. The backend must lower a common workload through shared stages according to a selected target profile and pipeline recipe, reject incompatible configurations before emission, and expose static differences without treating target-specific command counts as measured performance, with FIMDRAM AAM and HD-FIMDRAM serving as two concrete instances of this design. This scope is narrower than a complete PIM software stack but deeper than a generic device descriptor: it includes the coupled semantics of instructions, registers, addresses, placement, and host-triggered execution. Chapter 4 translates that boundary into concrete design and evaluation requirements.

Chapter 4

Methodology

The preceding chapters explained why FIMDRAM design-space exploration needs a compiler that can support different target models. Shared stages should reduce implementation effort without hiding differences in instructions, addresses, registers, or host commands. This chapter defines how the backend is designed and evaluated using FIMDRAM AAM and HD-FIMDRAM as the two cases. It connects the research questions in Chapter 1 to compiler requirements, verification checks, and controlled experiments.

The methodology deliberately separates three kinds of evidence. Structural evidence concerns whether one backend can express both target models through shared representations and explicit policy boundaries. Model-consistency evidence concerns whether target descriptions and intermediate compiler states satisfy their declared contracts. Comparative evidence concerns measurable properties of verified compiler artifacts generated from common workloads. Together, they address whether the compiler can generate, check, and compare candidates under the two implemented target models. Section 4.6 defines the limits of the conclusions drawn from this evidence.

4.1 Research Design and Evaluation

The investigation proceeds by constructing a FIMDRAM backend inside Cinnamon and subjecting that artifact to controlled target and policy changes. The artifact is evaluated as a compiler for design-space exploration: it must construct legal candidate programs, preserve target semantics, reject infeasible configurations, and isolate selected decisions for ablation. MLIR is appropriate for this purpose because its dialect mechanism permits target-specific semantics to coexist with reusable infrastructure, while progressive lowering allows each representation to retain information until a later decision can resolve it [19]. Cinnamon supplies the target-neutral CNM boundary and the surrounding compiler flow [5]; Samsung FIMDRAM and the HD-FIMDRAM

compiler model supply the two architectural cases [11], [21]. The backend is not treated only as an implementation result: we examine its structure, verifier behavior, and generated artifacts to answer the research questions. Each research question is therefore linked to a specific observation strategy:

Representation (RQ1).
The analysis examines the common target-facing representation, unified lowering stages, declarative compatibility data, and explicit policy choices reconciling the two source-backed profiles.

Contracts and model consistency (RQ2).
The evaluation combines backend static checks, pipeline contracts, model verifiers, and positive, negative, boundary, and integration tests. A separate comparison check verifies operation counts and the presence and consistency of the required target artifacts. It does not establish numerical equivalence.

Architectural consequences (RQ3).
The same generated workloads are compiled through the HD-FIMDRAM and FIMDRAM profiles. The analysis compares program-record composition, scheduling, synchronization, register pressure, address-field use, binary coverage, and host-command structure.

Design-space modularity (RQ4).
Controlled ablations replace one policy while holding the source workload, source-derived architectural model, and ordered pipeline stages fixed. A HD-FIMDRAM register-allocation ablation and a FIMDRAM logical-to-physical mapping ablation test whether these decisions can be exchanged without reconstructing the backend.

Table 4.1 maps each research question to its controlled variation, observations, checks, and the chapters that present the analysis. The final column states the conclusion supported by these observations, linking the research questions to the claims made in Chapters 6 and 7.

Table 4.1: Experiments and observations used to answer the research questions.

RQ	Controlled variation	Observations	Checks	Chapter	Supported conclusion
RQ1	FIMDRAM AAM vs. HD-FIMDRAM profile; common source launch and stage order	Target-facing IR, policy selections, program and target models	Compatibility table and cumulative contracts	Chs. 5–6	One backend structure can retain target-specific semantics
RQ2	Valid, malformed, boundary, and incompatible inputs	Rejections, verified artifacts, source-specific consistency fields	Backend contract, six checkpoints, positive/negative tests, experiment consistency gate	Chs. 5–6	Implemented invariants detect modeled inconsistencies
RQ3	Same generated workloads under the two source-backed profiles	Instruction, schedule, fence, register, address, binary, and command metrics	Shared schema plus target-specific feasibility checks	Ch. 6	Compiler-visible consequences can be compared without conflating metric semantics
RQ4	One allocator or mapping policy replaced; architecture and recipe held fixed	Changed policy model and downstream static artifacts	Full-recipe contracts and ablation tests	Chs. 5–6	Implemented policy boundaries support controlled substitutions

4.2 Source Traceability and Interpretive Boundaries

The two targets do not originate from interchangeable specifications. Samsung's publication defines the original HBM-PIM organization, ISA, CRF-controlled execution, host interface, and AAM behavior [11]. Kukić's work defines a modified host-driven model, including its reduced instruction set, implicit memory operands, low-order command payloads, and compiler-side allocation and scheduling procedures [21]. Cinnamon defines the pre-existing compiler framework into which these behaviors are integrated [5]. The implementation is a re-expression of the relevant behavior inside Cinnamon. The following terms distinguish what comes from the sources, what the compiler adapts, and what the experiments choose:

Source-defined. A publication or prior implementation directly specifies the behavior. Examples include Samsung's AAM register selection and the HD-FIMDRAM use of aligned low-order address bits for command metadata.

Ported behavior. The Cinnamon implementation preserves the intended semantics of prior work while expressing them through different operations, attributes, passes, or interfaces. For example, the HD-FIMDRAM path adapts the prior compiler's register-allocation constraints to the Cinnamon backend.

Compiler interpretation. The available source leaves a detail underspecified, and compilation requires a precise interpretation. Such a decision is named, recorded in the target configuration or output artifact, and never presented as a new hardware fact. The implemented decoder for Samsung's Figure 5(e) is one example: its address-field interpretation is recorded explicitly, rather than treated as an independently validated device specification.

Evaluation choice. The experiment defines the workload dimensions, normalization rule, ablation, or visualization. These choices characterize the study rather than either FIMDRAM architecture. Examples include the workload dimensions and output tile shapes listed in Table 6.1 and Table 6.2.

4.3 Requirements for a Modular Backend

The backend must retain the information needed by later passes and separate the decisions that depend on the target. MLIR supports small, composable abstractions and explicit validation [19], while PIM compilers connect hardware-independent representations with architecture-specific

mapping, scheduling, and resource management [16], [39], [40]. The following requirements cover representation, policy composition, and target-specific lowering.

4.3.1 Information Required by a FIMDRAM Backend

The combination of MLIR and Cinnamon determines how a backend can be structured, while the architecture determines what it must represent. Four classes of information recur throughout FIMDRAM lowering:

Semantic computation. Memory transfers and arithmetic intent, without prematurely fixing an opcode whose legality or operand form depends on the target.

Memory organization. Mapping logical tensor indices to pseudo-channels, bank groups, banks, rows, and columns. FIMDRAM AAM also uses address fields to select registers, whereas the HD-FIMDRAM model reserves low-order command bits for explicit payloads.

Device program state. Selected instructions, register assignments, CRF records, schedules, and control flow. Validity depends on target-specific operand rules and finite register and CRF capacities.

Host execution state. Mode transitions, instruction and register initialization, command packets, fences, and trigger addresses that realize the device program.

These classes become valid at different lowering stages: physical address fields cannot be checked before address assignment, register indices cannot be encoded before allocation, and host packets cannot be finalized before the execution model is known. The backend must preserve semantic information for later decisions while recording each resolved choice explicitly.

4.3.2 Representation Requirements

The backend accepts the same target-neutral `cnm.launch` for both profiles. Target choice occurs after this common boundary so that comparisons do not begin from different source programs. The target-facing representation must preserve semantic distinctions needed by later passes: memory movement and arithmetic, streaming versus stationary operand roles, register classes, and abstract host triggers. Once instruction selection occurs, the selected-program model keeps instruction identities, dependencies, logical and physical address metadata, register assignments, placement, command records, and binary coverage inspectable. The unit of backend compilation is one launch. Values are not assumed to remain resident in FIMDRAM

registers across launch boundaries. Multi-launch workloads are therefore aggregated explicitly rather than analyzed as a single device program.

4.3.3 Separation of Facts, Policies, and Recipes

Figure 4.1 separates hardware facts from compiler choices and shows how they form a resolved target profile. The architectural description on the left records legal opcodes, operand forms, register and CRF capacities, address fields, binary rules, and execution capabilities. The policies on the right specify how the compiler performs decisions such as instruction selection, logical indexing, scheduling, physical addressing, register assignment, placement, binary encoding, or command materialization. A target profile selects a compatible set of facts and policies. Below it, the pipeline recipe orders the transformations and verification checkpoints that produce the target artifacts.

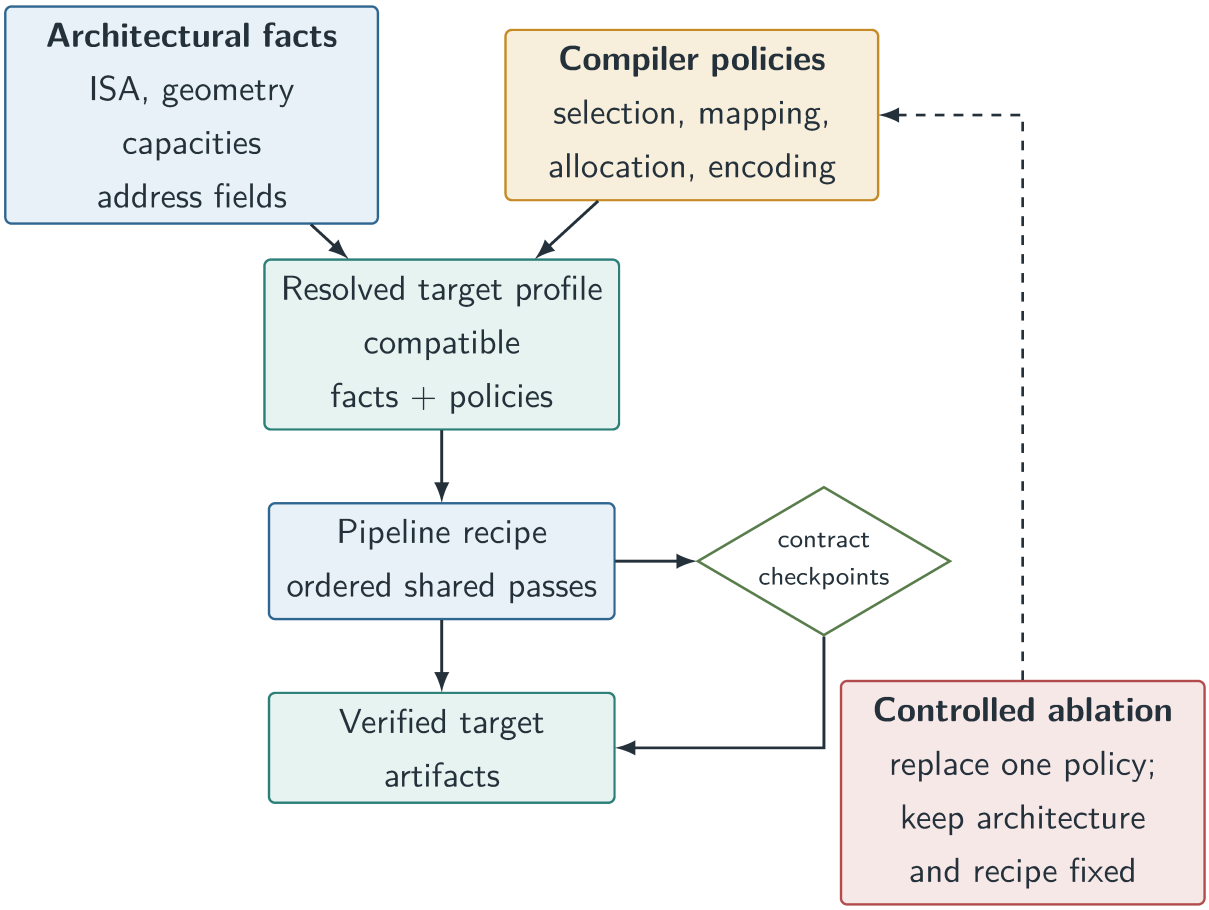


Figure 4.1: Separation of architectural facts, compiler decisions, target profiles, and ordered pipeline recipes.

Each decision has an explicit owner and result, allowing later passes to check that state without reconstructing how the policy chose it. This separation makes invalid compositions observ-

able: a binary policy cannot encode an opcode excluded by the selected ISA, and an execution model requiring host packets cannot pair with a profile lacking packet rules. The dashed return path in Figure 4.1 enables the ablations for RQ4: replacing one named policy while keeping the architectural facts, other policies, and recipe fixed, whereas comparing FIMDRAM AAM and HD-FIMDRAM replaces the complete source-backed profile. The allocation and mapping experiments follow this approach, measuring which downstream artifacts remain invariant and which change under substitution.

4.3.4 Comparable but Target-Specific Lowering

A fair comparison keeps the sequence of compiler decisions and verification checkpoints fixed while allowing target-dependent behavior within those stages. The backend preserves the distinction introduced in Chapter 3 between HD-FIMDRAM's explicit command payloads and FIMDRAM's address-derived register selection [11], [21]. The same inputs and shared stages need not produce the same instructions, register assignments, or host commands. A modular stage keeps its inputs and invariants stable while a policy supplies the target-dependent decision. Later contracts check the result without depending on how that policy works. Figure 4.2 follows the shared stages and names the main concern of each cumulative checkpoint. After target resolution, the target-profile check confirms that the recorded configuration matches the resolved profile. After instruction selection, the selection-rules check confirms that the selected forms and rules agree with that target. Program materialization provides the records needed to check the program schema, opcode legality, and operand forms. Address assignment enables validation of the address model, including required fields and target-specific layouts. The two lower checkpoints guard emission: the emission-readiness check validates register assignments, placement models, and encoder dependencies, while the artifact-coverage check validates the final binary words and host commands.

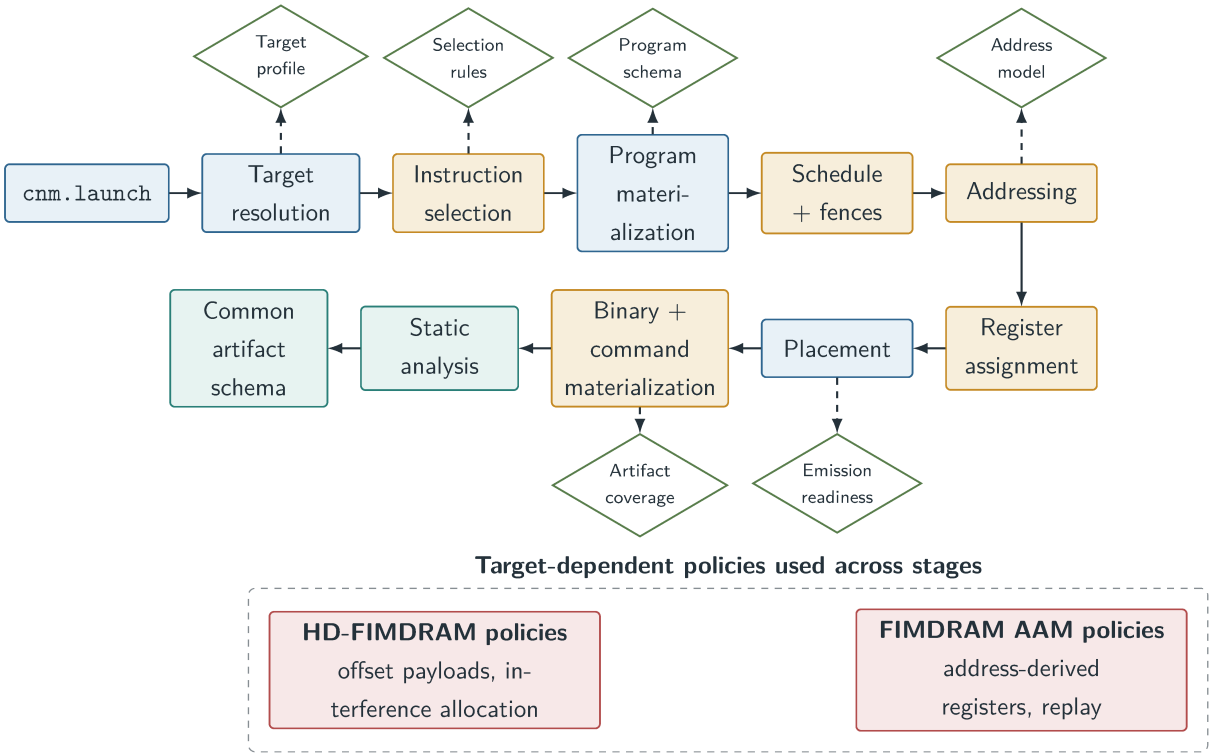


Figure 4.2: Shared FIMDRAM lowering stages and cumulative pipeline-contract checkpoints, labeled by their main verification concern. Each checkpoint requires the state produced by preceding transformations and retains earlier guarantees. The lower boxes give examples of policies used across stages. Local operation and model checks are not shown.

These checkpoints validate stages cumulatively rather than individual passes; for example, checking emission readiness after placement covers both register assignment and placement, while earlier guarantees remain in force. Local operation and model checks also run within the stages. Section 4.4 describes the verification layers, and Section 5.8 explains the checks at each boundary. The lower boxes list examples of target policies used across the pipeline. They do not attach to individual stages: address, register, and command decisions can depend on the same target rules. Chapter 5 describes these dependencies, including the FIMDRAM hooks in Section 5.5.2.

4.4 Verification Method

An extensible compiler can fail through malformed input, inconsistent target data, an invalid transformation, or an incomplete final model. MLIR's own design emphasizes that verification becomes more important as independent dialects and transformations are composed [19]. Kukić's compiler work similarly uses staged verification to isolate faults across a FIMDRAM compilation and execution flow [21]. Because this thesis has no integrated physical runtime or

simulator, the checks concern compiler structure, modeled constraints, and artifact completeness rather than numerical execution. Different checks use different information:

1. **Dialect verification** checks typed operation structure and local semantics, including launch regions, operand and block-argument agreement, static shapes where required, register-block bounds, and result types.

2. **Configuration verification** checks scalar geometry, policy names, value ranges, address-field widths, and profile-specific constraints before lowering begins.

3. **Program-schema verification** checks that the planning representation contains the information required at its current stage. Requirements accumulate as decisions become final, so a later check must retain the obligations established by earlier stages.

4. **Backend rule contracts** validate the target database without an input program. They check uniqueness and resolvability of compatibility combinations of target components, consistency of target geometry, opcode and operand-form agreement, complete binary-rule coverage, execution capabilities, and recipe composition.

5. **Pipeline contracts** check the relationship between a transformation's output and the resolved target. They detect missing artifacts and stale decisions, including metadata belonging to another profile. Section 5.8 identifies the concrete checkpoints and the checks performed at each one.

6. **Target-model verification** checks properties that are local to an execution or encoding model, including packet schemas, packed-field representability, FIMDRAM replay and command progression, and the presence of target-specific completion markers.

7. **Experiment consistency checking** rejects compiler output that is structurally successful but unsuitable for comparison. It checks semantic operation counts, final instruction legality, HD-FIMDRAM alignment and payload fit, fallback bank decisions, FIMDRAM model completeness, and AAM register/address agreement.

The final layer is intentionally not an independent hardware verifier. Compiler contracts answer whether an artifact satisfies the modeled backend semantics; the experiment gate answers whether that artifact contains the evidence required by the experiment. Keeping these questions separate avoids making benchmark policy part of general compiler legality.

Tests are grouped by the behavior they check. Positive tests establish accepted forms and expected transformations. Negative tests provide malformed profiles, illegal opcodes, missing

metadata, incompatible policies, and invalid transitions to ensure the compiler raises explicit errors instead of silently masking failures. Boundary tests exercise finite register, CRF, command-buffer, and address-field capacities. Integration tests compile representative launches through complete recipes and check their checkpoints and final artifacts. This combination is necessary because a local pass test cannot detect target-table drift, while an end-to-end success may fail to expose an invariant that was accidentally bypassed.

4.5 Controlled Static Comparison

The comparative study measures compiler artifacts using only static modeling: the method reports instruction, register, address, and command properties.

4.5.1 Targets and Experimental Controls

The primary comparison includes only the two source-backed targets, `hd-fimdram` and `fimdram`, invoked through `hd-fimdram-pipeline` and `fimdram-pipeline`, respectively. Two policy substitutions test whether the backend can change one decision without changing the surrounding compilation flow. The HD-FIMDRAM allocator experiment replaces interference-based allocation with linear allocation. It keeps the instruction set, address encoding, workload, and recipe fixed. This comparison tests the allocation interface and exposes changes in register assignment. The FIMDRAM mapping experiment compares direct transaction mapping with an AAM-aligned striped mapping. The policies place logical elements at different physical addresses, while the hardware's address-to-register decoder stays fixed. This isolates the effect of data layout on register constraints, generated records, and compilation success. Two more studies examine scaling and tiling. The capacity study increases output dimensions or reduction length while keeping each target's resource limits fixed. It identifies where the backend splits work into launches or rejects a configuration. The separate tile-shape study holds matmul dimensions at $8 \times 8 \times 2$ and compares 1×1, 1×2, 2×1, and 2×2 output tiles. It tests how tiling affects address-derived register constraints. Table 6.4 summarizes the variables and controls for all five experiment groups, and Table 6.3 gives the target resource settings.

4.5.2 Workload Construction

The comparison starts from the same source program for both targets, so that differences in the output reflect backend decisions rather than different inputs. Figure 4.3 shows how the workload generator constructs this common input before target-specific lowering begins.

Generating common inputs. A manifest lists the configurations to generate. Each entry specifies the workload dimensions, layout, operation-count metric, and any explicit output tiles. The generator uses these values to construct an MLIR module with fixed buffer sizes, loop bounds, and reduction lengths. The supported workloads are vector addition, SAXPY, dot product, matrix–vector multiplication, matrix multiplication, and explicit multiply–accumulate. Table 4.2 summarizes their computations and the compiler behavior each pattern exercises. Table 6.1 in Chapter 6 gives the actual dimensions and source arithmetic counts used in the primary comparison. These small, fixed-size cases allow individual lowering decisions to be inspected; the separate capacity study tests larger demands on the same target resources. All primary inputs use logical row-major storage. Both branches in Figure 4.3 receive the same target-neutral `cnm.launch` input; the selected backend policies then map its logical elements to physical addresses. The generator therefore does not build AAM striping into the FIMDRAM input, which would introduce a layout difference before the comparison reaches the backend. All generated kernels use 16-bit integer IR operations, as our evaluation focuses on compiler artifacts (such as instruction selection and resource allocation) rather than runtime numerical precision. Consequently, workload names such as SAXPY denote the underlying computational pattern rather than a floating-point implementation.

Table 4.2: Workload families and the computation patterns used to exercise the backend.

Workload	Computation	Compiler aspect exercised
Vector addition	$z_i = x_i + y_i$	Independent additions with two input loads and one output store.
SAXPY	$z_i = \alpha x_i + y_i$	Scalar reuse across outputs and selection of multiply–add patterns.
Dot product	$s = s^{(0)} + \sum_{i=0}^{n-1} x_i y_i$	A dependent accumulation chain producing one output.
Matrix–vector multiplication (matvec)	$y_i = y_i^{(0)} + \sum_{j=0}^{k-1} A_{ij} x_j$	Independent output rows, each containing a reduction; vector reuse across rows.
Matrix multiplication (matmul)	$C_{ij} = C_{ij}^{(0)} + \sum_{\ell=0}^{k-1} A_{i\ell} B_{\ell j}$	Two-dimensional output tiling, reductions, and multiple launches.
Multiply–accumulate	$z_i = a_i + x_i y_i$	Element-wise accumulation from three input arrays, without a cross-element reduction.

α is a scalar input; superscript (0) denotes an initial accumulator supplied as input. The formulas describe logical computations; the generated kernels use 16-bit integer arithmetic. Table 6.1 lists the evaluated dimensions and source arithmetic counts.

Dividing outputs into launches. For matrix multiplication, the generator divides the output matrix into tiles and places each tile in a separate `cnm.launch`. The primary corpus uses the generator's default 2×2 output tiles. The output-scaling experiment instead uses 1×2 tiles, which compile for the sampled output sizes. This allows the study to examine launch growth without a tile-shape rejection stopping the comparison. Section 6.1 lists the complete dimensions and experimental controls. As the red tiling note in Figure 4.3 indicates, this division separates output elements but leaves each reduction intact. Splitting a reduction across launches would require partial results and a separate step to combine them, which the current backend does not implement. A long reduction can therefore exceed the target's register or CRF capacity, in which case the harness records a compilation failure.

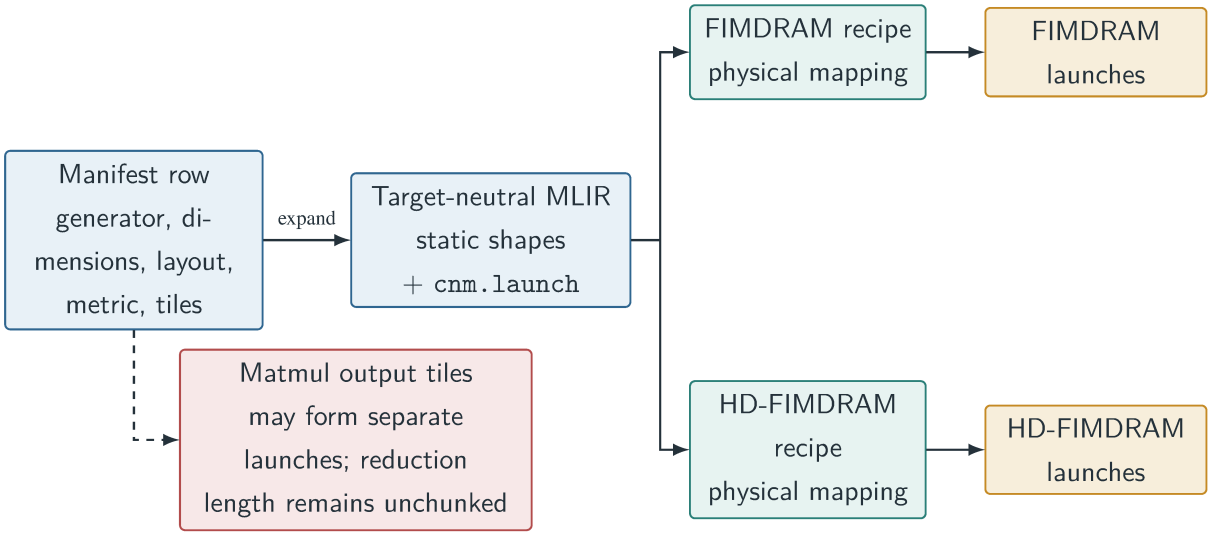


Figure 4.3: Generation of identical target-neutral workloads and target-specific lowering inputs.

Counting the work represented. The comparison distinguishes the computation requested by the source from the records generated to express it. It retains four counts:

- *Source arithmetic operations:* ADD and MUL operations in the source, with MAC and MAD each contributing two arithmetic operations.
- *Represented semantic instances:* the arithmetic instances covered by the selected operations, including all members represented by a grouped record. A fused multiply–accumulate counts as one such instance but as two source arithmetic operations.
- *Selected arithmetic records:* the arithmetic instruction records in the generated program.
- *Total program records:* arithmetic records together with movement, fence, control, and pseudo-records.

These counts need not agree. For vector addition with eight elements, the source contains eight additions. The FIMDRAM output represents those eight instances with one grouped arithmetic record and seven program records in total. The HD-FIMDRAM output retains eight arithmetic records. Thus, fewer records do not necessarily mean less computation. Section 5.5.2 explains replay construction, and Section 5.7 illustrates the distinction with a vector addition compiled for both targets. The next subsection explains how these counts and the other measurements are combined and compared.

4.5.3 Measurements and Aggregation

The measurements answer a practical question: what changes in the generated program when the target profile or one compiler policy changes? The analysis first collects results for each launch, then combines launches belonging to the same workload and target. Table 4.4 lists the metric families, their units, and the rules used to combine and normalize them.

What is measured. The shared analysis schema records three aspects of the compiler output:

- **Generated program and host commands.** Program-record counts describe the arithmetic, data movement, synchronization, and control records produced by lowering. Scheduling measurements record the selected strategy and how many program records it moves. Host-command counts describe the corresponding host work, using packets for HD-FIMDRAM and expanded commands for FIMDRAM AAM. These remain distinct units, even though both appear in the same result schema. Active binary words count the device words actually encoded, excluding padding. This differs from both the program-record count and the padded CRF image emitted by HD-FIMDRAM. Section 5.6 describes these outputs.

- **Register use and address mapping.** Register measurements show how many physical names the compiler assigns, how many allocation groups are live at once, and whether names are reused or spills occur. Address measurements record the number of address entries and the row, column, and bank values used by the mapping. Each address retains its buffer identifier and buffer-relative offset.

- **Target configuration and consistency.** Transaction size and broadcast width describe the selected target. Additional checks establish whether generated addresses and commands satisfy its rules: HD-FIMDRAM checks alignment and whether register and CRF identifiers fit the command payload, while FIMDRAM AAM checks agreement between addresses and register indices.

An allocation group contains values that must share a register assignment, such as a tied MAC input and destination. Reusing that assignment within the group is different from recycling the physical register for another group whose lifetime does not overlap. The analysis counts these cases separately. Peak live-group count measures the largest number of allocation groups live at the same point in a launch. Distinct physical-register count instead records how many different register names appear over the entire launch. The first describes simultaneous demand; the second records the names used even if their lifetimes do not overlap. Address-field counts likewise describe distinct values within individual fields, not complete physical locations: the same bank-field value can occur in different bank groups. Section 3.2.4 distinguishes the indexing, mapping, and placement decisions behind those addresses.

How launches are combined. The aggregation rule depends on what a measurement counts. Program-record, command, active binary-word, and address-entry counts are summed across launches. Peak register use takes the largest launch value rather than the sum, because the measurement describes the highest per-launch requirement. Average live-register use is weighted by each launch's program-record count. For address fields, the analysis takes the union of the observed values, counting a repeated row, column, or bank value only once within its field.

After aggregation, additive counts can be divided by the workload's number of output elements or source IR arithmetic operations. This normalization allows different workload sizes to be compared, for example through the number of generated program records per source operation. Peaks, averages, Boolean checks, and hardware properties retain their original values. Table 4.4 specifies which normalization applies to each metric family.

How missing results are handled. Only successful runs contribute numerical measurements. A field that does not apply to a target is omitted, while an applicable measurement can be zero: a successful compilation with no spills reports `spill_count = 0`. A failed allocation instead retains its diagnostic and is not plotted as zero register use. When register assignment fails, the backend rejects the configuration rather than generating spill code. Table 4.3 defines the outcome labels used throughout the evaluation. These labels distinguish recorded failure categories; `unsupported` and `compile_failed` can both describe an expected resource or mapping rejection. The term *rejection* refers to the compiler declining a configuration, while the tables retain the harness's recorded status.

Table 4.3: Recorded compilation and validation outcomes.

Status	Meaning
`success`	Compilation completes and the required comparison checks pass.
`unsupported`	A compiler rejection classified as unsupported, including the recorded AAM collisions and CRF overflows.
`compile_failed`	Another unsuccessful compilation, including the recorded register exhaustion and early tiling-budget diagnostics.
`contract_failed`	A backend or pipeline contract fails, or required analysis output is missing or malformed.
`model_consistency_failed`	Compilation completes, but the comparison checks detect inconsistent semantic counts or target artifacts.

Table 4.4: Static measurements and their aggregation rules.

Metric family	Measurements	Unit	Across launches	Normalization	Scope
Program accounting	Source operations, represented semantic instances, arithmetic and total program records	operations / instances / records	Sum each count separately	Per output element or source operation	Both targets; counts remain distinct
Binary output	Active encoded words, excluding padding	words	Sum	Per output element or source operation	Both targets; not the padded image size
Scheduling	Selected strategy and program records moved	records	Sum moved records; strategy must agree	Per output element or source operation	Both targets
Host commands	Execution packets or expanded AAM commands	packets / commands	Sum within each target	Per output element or source operation	Different units for the two targets
Register use	Peak and average live groups; reuse, aliases, and spills	groups / events	Maximum peak; record-weighted average; sum events	Event counts only	Both targets; interference edges only for HD-FIMDRAM
Address mapping	Address entries and observed row, column, and bank values	entries / values	Sum entries; union of values for each field	Entry counts only	Buffer-relative addresses, not runtime allocations
Target settings	Transaction size and broadcast width	bytes / lanes	Settings must agree	None	Resolved target configuration
Payload fit	Required bits, alignment violations, overflows, and fit checks	bits / events	Maximum required bits; sum violations; all fit checks must pass	Violation counts only	HD-FIMDRAM
Register/address agreement	Matches and mismatches between assigned and decoded registers	events	Sum	Per output element or source operation	FIMDRAM AAM

Only successful runs contribute measurements. Inapplicable fields are omitted; zero denotes an applicable measurement with no occurrences.

4.5.4 Reproducibility Procedure

The procedure follows each experiment from its input configuration to the reported results. Manifest expansion first generates the target-neutral modules and records their resolved dimensions. Recipe compilation then processes each module for the selected targets with all pipeline contracts enabled. The third stage checks model consistency and eligibility for comparison, retaining the compiler output and classifying the outcome. Successful runs proceed to analysis, which extracts JavaScript Object Notation (JSON) and tabular measurements, combines launches, and produces plots using the rules in Section 4.5.3. Unsupported configurations and compilation or verification failures instead retain their diagnostics; they remain visible in the results without contributing numerical measurements. The procedure retains the expanded manifest, resolved target configuration, compiler output, and diagnostics. These records allow each plotted value to be traced to its compiler output and each unsuccessful case to its reported cause.

4.6 Validity and Scope of Inference

The method controls compiler-side variables but does not remove every source of uncertainty: four limitations bound the interpretation of its results.

First, the backend reimplements the two target models rather than reproducing their original software stacks. Since the published paper [11] does not disclose every hardware and software detail, our model may exhibit minor discrepancies compared to the actual device; the AAM checks therefore ensure internal consistency with our model rather than exact hardware and software parity. For HD-FIMDRAM, the implementation adapts Kukić's compiler behavior to Cinnamon rather than retaining the original Polygeist-based compilation flow [21]. The different representations and passes mean that identical binary output is not guaranteed. The port also leaves out components beyond the shared-backend comparison studied here, including the original frontend and automatic tiling based on the Z3 constraint solver. Second, the study evaluates compiler output, not execution on a complete system. The study compares generated program records, register use, and address mappings, but does not measure execution time or energy. The generated addresses remain relative to individual buffers, and the host commands describe the required actions without an integrated runtime to execute them. Assessing functional behavior and numerical correctness requires execution and validation of the generated programs. Performance claims additionally require validated timing and energy models or hardware measurements. PIMSys illustrates how host, cache, and DRAM models can be integrated for system-level evaluation [26]. Third, the workload suite emphasizes small, statically expanded kernels that fit the current launch-oriented lowering flow. It covers several arithmetic and reduction patterns, but it does not represent the diversity of complete applications, dynamic shapes, run-

time scheduling, or large-model execution. Output tiling does not partition long reductions, and tile-shape search is not automatic, which can make register or CRF capacity the first observed limit. Finally, modularity is evaluated through two deliberate policy substitutions and through the shared realization of two profiles. This shows that the chosen register allocation and mapping boundaries are interchangeable, though supporting future FIMDRAM variants may still require extending the interfaces.

Within these limits, the method provides a controlled answer to the thesis questions. It keeps source and evaluation choices traceable, requires the same logical inputs for both targets, checks intermediate states before analysis, preserves failures instead of converting them to favorable measurements, and restricts conclusions to compiler-visible evidence. Chapter 5 next presents the concrete realization of this design; Chapter 6 applies the comparison protocol defined here.

Chapter 5

Implementation

Chapter 4 defined the requirements for a modular and verifier-backed FIMDRAM compiler intended to generate and analyze design candidates. This chapter presents their realization inside Cinnamon. The implementation starts at Cinnamon's target-neutral `cnm.launch` boundary [5], introduces a target-facing FIMDRAM dialect, and progressively constructs a selected device program together with the address, register, placement, binary, and host-command models required by the resolved target. FIMDRAM AAM and the HD-FIMDRAM model obtain their distinguishing semantics from their respective sources [11], [21]. They use the same ordered recipe but specialize the decisions at which those semantics differ.

The presentation follows ownership rather than listing source files in directory order. Section 5.1 defines the boundary with Cinnamon. Section 5.2 describes the typed dialect and mutable selected-program representation. Section 5.3 explains how declarative facts, configuration, profiles, and registries are connected. Section 5.4 follows the shared lowering mechanics, after which Section 5.5 examines the two source-backed specializations. A worked vector-addition example connects the representations to the emitted artifacts. The final sections describe verification, testing, and the extension points used by the policy ablations.

5.1 Integration and Ownership Boundary

Cinnamon already provides the `cinm-opt` driver, dialect and pass registration, target-neutral `cinm` and `cnm` abstractions, and the conversions that produce a `cnm.launch` [5]. Figure 5.1 separates these existing components on the left from the thesis-owned extension on the right. The extension contains the FIMDRAM dialect, lowering passes, target model, recipes, tests, and tools; the existing Cinnamon framework remains outside the contribution. The Samsung publication and Kukić's earlier compiler supply architectural requirements [11], [21]. The backend implements these requirements within Cinnamon rather than reproducing the original soft-

ware stacks, for the reasons discussed in Section 4.6. The dashed arrow therefore points to architectural references, not code included in the extension. The central conversion in the figure marks the implementation boundary: it transforms `cnm.launch` into `fimdram.launch`. A FIMDRAM launch owns a workgroup, input and output buffers, optional kernel information, and a single-block region. It is `IsolatedFromAbove`: every external value required by the device region appears through a launch operand and corresponding block argument. This property makes the launch a self-contained compilation unit on which target configuration and all derived backend models can be attached. It also defines a deliberate limitation: physical register residency is not carried between launches. The registration block connects the extension to the existing `cinm-opt` driver by registering its dialect, passes, and public pipelines. Pipelines such as `--hd-fimdram-pipeline` and `--fimdram-pipeline` serve as wrappers around predefined target recipes, while low-level passes remain available for focused tests.

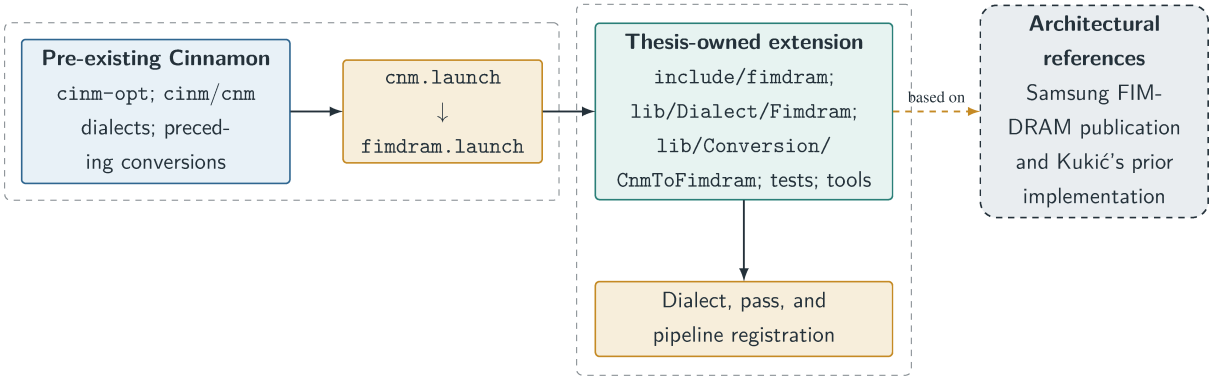


Figure 5.1: Ownership and integration boundary of the FIMDRAM extension inside Cinnamon.

5.2 Target-Facing Intermediate Representations

The implementation uses two complementary representations. Typed MLIR operations preserve source-level structure, types, and SSA dependencies while semantic lowering and instruction selection are still active. A dictionary array attached to each launch then serves as the mutable planning representation for scheduling, addressing, physical allocation, placement, encoding, and analysis. This division avoids forcing every evolving target record into a new operation class, but it requires a centralized schema verifier once compilation leaves the typed operation layer.

5.2.1 Types and Launch-Level Operations

The FIMDRAM dialect defines workgroup, buffer, and register types. A workgroup type records backend-owned geometry and exposes its element count. A buffer type retains logical shape, ele-

ment type, workgroup shape, and memory level, and implements MLIR's shaped-type interface so that verification and lowering can compute element counts and byte sizes. Register values remain typed before physical assignment: distinct types represent GRF-A, GRF-B, SRF, and a contiguous SRF block. Preserving the element type allows loads, arithmetic, and stores to reject mismatched data before register numbers or instruction words exist. Lifecycle operations mirror the CNM workgroup, allocation, and release operations at the target level. Buffer-oriented operations `set_zero`, `scatter`, `gather`, and `accumulate` act as temporary placeholders. They preserve transfer or compute intent while the input still has buffer-level structure, then lower into register movement and arithmetic. Their verifiers check static shape requirements, element-type agreement, workgroup compatibility, and transfer windows.

5.2.2 Semantic and Selected Operations

Figure 5.2 separates typed operations from the program attribute attached to a launch and distinguishes semantic arithmetic from selected target-specific forms. Three semantic register operations, `add_reg`, `mul_reg`, and `mac_reg`, represent arithmetic that is target-neutral within the FIMDRAM family. The emphasized arrow marks instruction selection, which replaces these operations with forms supported by the resolved ISA. HD-FIMDRAM uses independent-destination ADD and MUL forms while retaining a tied accumulator for MAC; and FIMDRAM uses AAM forms in which the BANK operand and effective register indices are associated with the trigger address. Transfer and synchronization operations remain shared. GRF loads and stores carry a logical index, an SRF-block load represents contiguous scalar materialization, and an extract pseudo-operation refers to an element of that block. Additional operations express zero formation, direct host-value movement, rectified linear unit (ReLU) writeback, and fences. FIMDRAM control operations such as NOP, JUMP, and EXIT are introduced later as program records rather than standalone typed operations because they arise during CRF finalization rather than from the original launch body. The lower program-attribute group in the figure distinguishes these control instructions and the arithmetic MAD record from standalone typed operations. Operations eligible for program materialization implement a common encoding interface. The interface supplies a canonical mnemonic, assembly shape, and operation-specific fields such as logical index or SRF-block extent. Instruction-stream emission can therefore consume a shared interface instead of maintaining a type switch over every selected operation. Adding a new typed form requires implementing the interface, while opcode-wide properties remain in the target description.

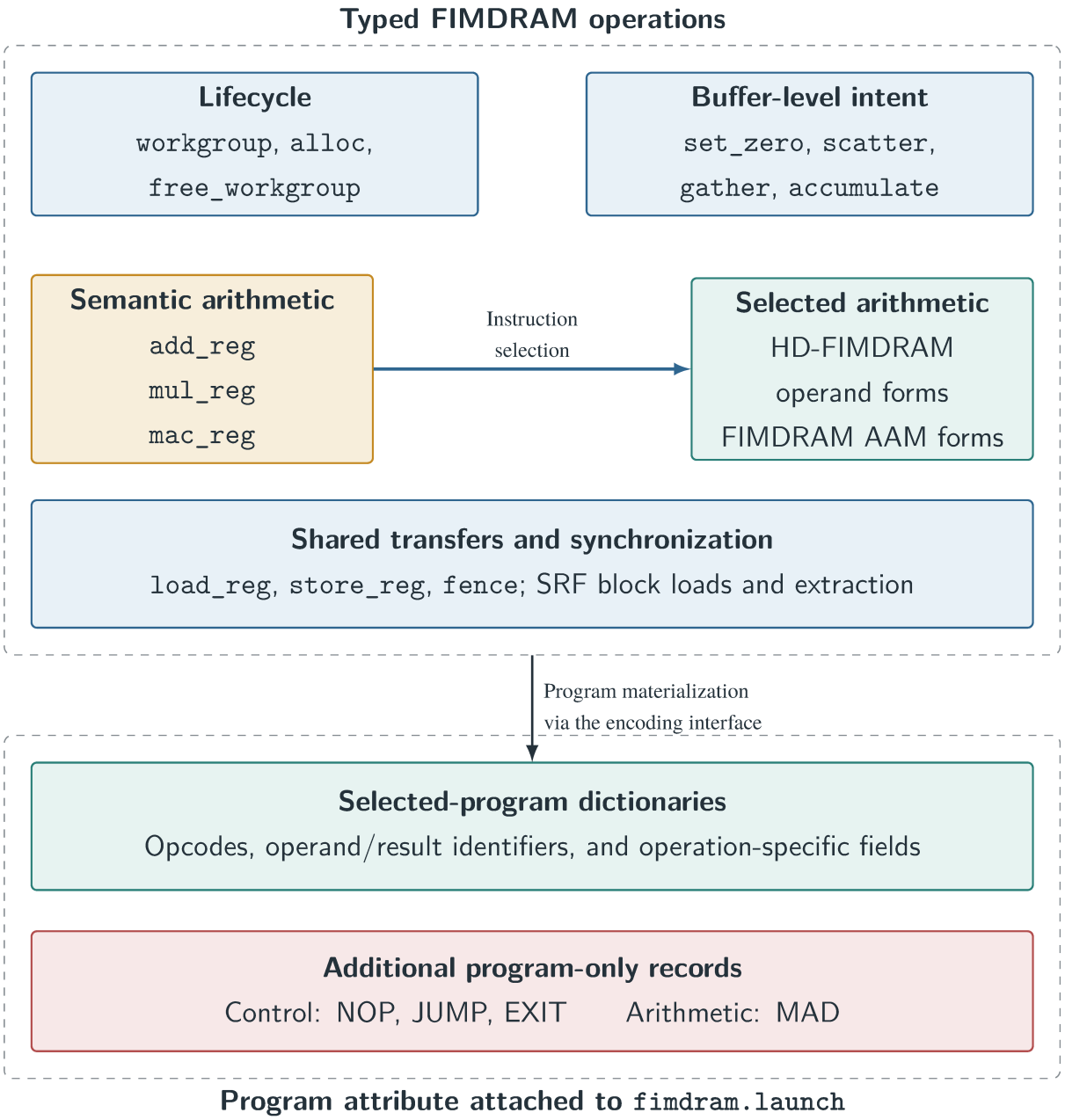


Figure 5.2: Typed operation families and selection boundaries in the FIMDRAM dialect.

5.2.3 Selected-Program Attribute

After instruction selection, the instruction-stream pass walks encoding operations and attaches a `program` array to the launch. Each dictionary initially contains a canonical opcode, integer operand and result identifiers that preserve SSA-like dependencies, and fields required by the operation, such as a logical index, selector, block range, or control immediate. Figure 5.3 starts from this `Emitted` state and follows the validation requirements as later passes enrich the same record with logical and physical addresses, trigger information, physical registers, processing-unit placement, and local bank-group identity. The typed selected operations remain in the launch body, while the attribute program becomes the mutable backend plan. The implementation centralizes its opcode enumeration, traits, parsing, field access, integer-width checks, trigger helpers, and stage validation in `FimdramProgramAttrs`. The states shown in the figure add requirements cumulatively: checking physical registers at `Registered`, for example, does not replace the address checks required at `Addressed`. The final states check that the program is ready for encoding and analysis. Passes use these helpers instead of opening dictionary fields directly, which provides one migration point if a future implementation replaces the dictionaries with typed ISA operations. This representation is intentionally flexible but not weakly checked. Opcode traits define operand and result counts, tied values, phase classification, assembly form, and semantic class. Readers validate that integer attributes are representable rather than truncating them, and command-buffer checks apply only when the selected target declares a meaningful limit. The dictionary thus remains inspectable while retaining explicit stage contracts.

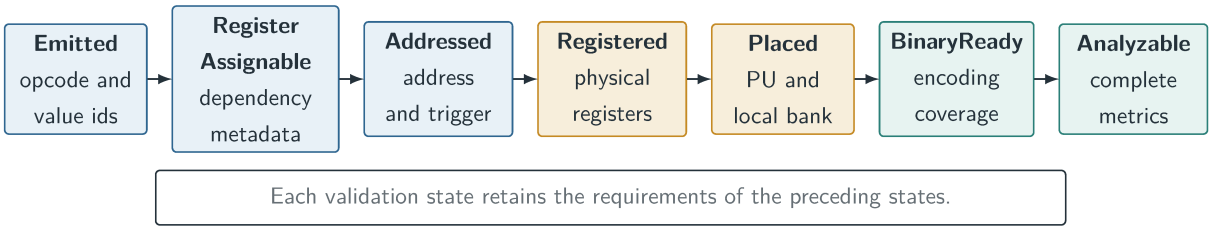


Figure 5.3: Progressive enrichment and validation of a selected-program dictionary.

5.3 Declarative Target Model and Resolution

Repeated target facts are stored in `FimdramTargetDescription.def`, an X-macro data file included by multiple C++ owners. Figure 5.4 shows this file as the common source of the backend's generated tables. Inside it, row families describe program opcodes, ISA legality and operand forms, ISA-wide capabilities, supported compatibility tuples, FIMDRAM operand selectors and geometry, HD-FIMDRAM geometry and offset fields, instruction-selection rules, binary actions and encoder bindings, execution models, and packet rules. Different translation

units generate the typed tables required by selection, encoding, packet construction, program traits, and verification from this single description. The lower group in the figure contains the code that consumes these tables: generation centralizes the facts, but developers still implement the algorithms and validation checks separately.

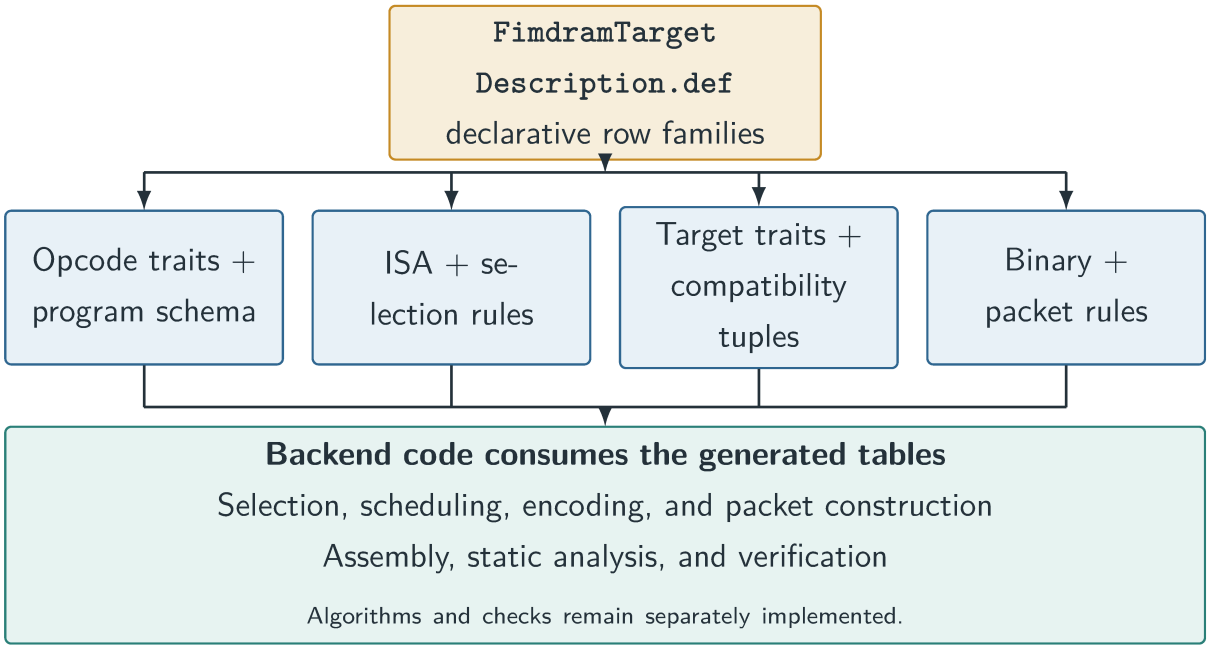


Figure 5.4: One declarative target description generates the cross-cutting tables used by the backend.

5.3.1 Configuration, Traits, and Profiles

`FimdramTargetConfig` stores machine geometry and policy selectors without prescribing pass order. Geometry includes processing-unit and bank topology, register capacities, address dimensions and burst size, command-buffer limits, and any address fields reserved for command payloads. Selectors name the ISA, binary policy, physical-address policy, execution model, logical indexing, instruction selection, placement, and register assignment. Typed FIMDRAM and HD-FIMDRAM trait views expose these model fields directly to subsequent passes. FIMDRAM traits define HBM geometry, CRF replay parameters, and default striping alongside modeled capacities: eight GRF-A, eight GRF-B, 16 combined SRF, and 32 CRF entries. HD-FIMDRAM traits define double data rate 4 (DDR4) address geometry, burst size, and target register limits. Hardware properties such as the AAM decoder remain fixed architectural traits, whereas decisions like logical striping remain configurable compiler policies. A resolved `FimdramTargetProfile` combines this configuration into a validated tuple attached to the launch. Rather than overriding explicit user settings, recipes populate unspecified fields with

coherent target defaults, such as pairing the HD-FIMDRAM profile with its packet execution model and interference allocator, or FIMDRAM with its AAM encoder and address-aligned allocator. A subsequent validation check verifies the resolved profile, maintaining a strict separation between metadata mutation and consistency checks.

5.3.2 Policy and Strategy Registries

Modular compiler stages decouple algorithmic choices via dedicated registries. Logical indexing currently provides row-major linearization. Instruction selection offers scalar and FIMDRAM AAM strategies. Placement includes logical division/modulo, physical bank-group, HBM physical-bank, and round-robin approaches. Register assignment supports linear, bank-aware, HD-FIMDRAM interference, and AAM address-aligned allocators. Finally, physical-address policies implement identity and debug mappings, HD-FIMDRAM offset packing, interleaved layouts, and the structured HBM AAM policy. Each selected strategy records its identifier in the launch model. This metadata enables downstream contract verification and allows the evaluation to isolate individual policy ablations from broader architectural profile changes.

5.4 Shared Lowering Pipeline

The two evaluated recipes use a common sequence of lowering stages and contract checkpoints. Target-gated FIMDRAM hooks appear in that sequence for both profiles and become no-ops when their predicate does not match. This keeps pass order constant across the primary comparison, while policy dispatch changes the behavior that must differ. Figure 5.5 relates the main stage groups to their outputs, from a typed launch to program dictionaries and materialized artifacts. The dictionaries become the working representation for physical planning; the typed operations remain in the launch body.

Table 5.1 follows these stages in order and identifies the six cumulative contract boundaries shown in Figure 4.2. The two FIMDRAM AAM hook groups remain in the shared sequence but make no change under HD-FIMDRAM; its host packets are produced during binary emission instead. A dash in the checkpoint column means that the group has no additional cumulative checkpoint, not that its operations are unchecked. Later checkpoints retain the earlier requirements, as described in Section 5.8.

5.4.1 Conversion, Launch Formation, and Selection

The conversion stage on the left of the figure lowers CNM constructs (workgroups, buffer lifecycles, and launch terminators) into FIMDRAM equivalents while preserving legal host dialects.

Table 5.1: Ordered lowering stages, target-specific behavior, and cumulative contract boundaries of the two evaluated recipes. Rows group related passes.

Order	Stage group	Role and target behavior	Contract boundary
1	Backend validation	Check target tables, rule coverage, and recipe compatibility before processing launches.	Separate backend-table check
2	Target resolution and launch formation	Apply the profile, convert `cnm.launch`, lower placeholders, and complete the body where needed.	Target profile
3	Instruction selection	Select legal target forms and resolve explicit or implicit operand roles.	Selection rules
4	Program materialization	Construct program records with opcode, operand, and result identifiers.	Program schema
5	Scheduling and fences	Order records under dependencies, then insert or retain phase boundaries.	–
6	AAM control-flow hook	Form replay groups and `JUMP`/`EXIT` control for FIMDRAM; no-op for HD-FIMDRAM.	–
7	Address assignment	Derive target address fields; reserve payload bits for HD-FIMDRAM or decode register indices for AAM.	Address model
8	Register assignment	Allocate registers under liveness and bank constraints for HD-FIMDRAM; enforce address-derived assignments for AAM.	–
9	AAM trigger and command hooks	Form triggers, then expand the FIMDRAM host-command trace; both hooks are no-ops for HD-FIMDRAM.	–
10	Placement and prologue	Assign processing-unit coordinates and attach setup metadata required by emission.	Emission readiness
11	Binary and packet emission	Encode device words and coverage; also construct host packets and a padded image for HD-FIMDRAM.	Artifact coverage
12	Static analysis	Measure the verified program and its address, register, binary, and host artifacts.	–

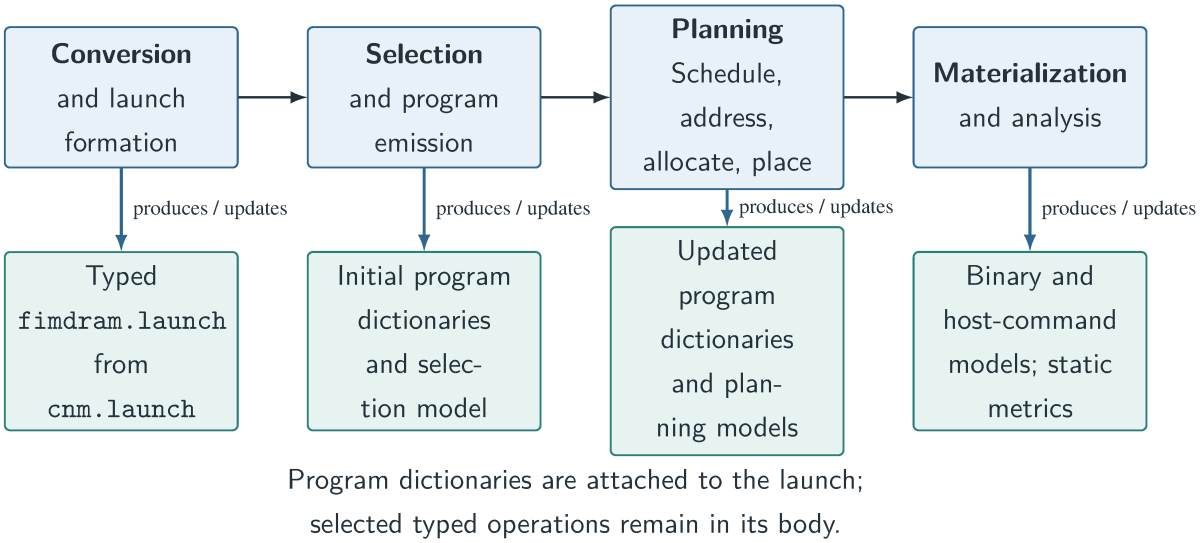


Figure 5.5: Main lowering stages and the representations they produce or update.

Launch-body lowering processes static affine and structured control flow (SCF) loops, linearizes memory accesses, and matches arithmetic patterns such as ADD, MUL, and MAC into register-level operations. To prevent static expansions from exceeding modeled register or command-buffer limits, this stage applies bounded unrolling and tiling. Logical indexing remains decoupled from this lowering, ensuring multidimensional access patterns are resolved before physical address packing. Instruction selection then maps generic operations to the target ISA based on semantic operation, strategy, and operand geometry. For host-triggered instructions, selection can fold single-use memory loads into implicit `BANK` operands. This decision is guided by operand roles: streaming data is prioritized for trigger-supplied sources, whereas stationary data is allocated to registers. The selected rules and operand behaviors are recorded in `instruction_selection_model` before emitting program dictionaries.

5.4.2 Scheduling and Synchronization

The shared scheduler constructs a dependency graph from operand identifiers and memory barriers. Both primary targets use a stable topological sort that prioritizes loads, followed by computation and extraction, and finally stores. Because of this FIMDRAM does not retain source order; it shares the same reordering strategy as HD-FIMDRAM, while downstream passes enforce its specific AAM and replay constraints. The scheduler records the number of reordered program records in the launch metadata. Synchronization is handled by a subsequent fence-insertion pass. Operations are classified into load, compute, store, or existing barrier phases, allowing the pass to insert necessary boundaries and prune redundant ones. For HD-FIMDRAM, fences between successive MAC waves are eliminated only when operand readiness and bank availability

guarantee safe execution; otherwise, conservative boundaries are preserved.

5.4.3 Addressing, Registers, and Placement

Address assignment maps each operation's logical index to a logical address, delegating physical decomposition to the active policy. The normalized representation tracks the originating buffer identifier and buffer-relative offset across all downstream triggers, packets, and commands. Depending on the target, the physical representation resolves raw addresses into row, column, bank, bank-group, and AAM register-index fields, logging the resulting configuration in an `address_mapping_model`.

Register allocation applies the lifetime, operand-tie, and register-bank constraints introduced in Section 3.2.3 to the scheduled, addressed program. The backend provides linear, bank-aware, interference-based, and address-aligned strategies for these different constraints. For HD-FIMDRAM, the default interference allocator first groups values that must share a register. It then builds a graph whose edges connect groups with overlapping lifetimes. Greedy coloring assigns each group a register in an allowed bank, preventing connected groups from sharing that register. The alternative linear allocator makes assignments in a deterministic order and reuses registers according to value lifetimes. Both strategies must obey the target's bank constraints, but their assignment choices can differ: a register chosen for a flexible value may later be needed by a value restricted to that bank. Section 6.3.1 compares these strategies without changing the surrounding pipeline. For FIMDRAM AAM, by contrast, the allocator cannot freely choose register indices: it must use those derived from the assigned addresses and reject incompatible live values.

During this stage, FIMDRAM lowering bounds resources across an eight-processing-unit parallel budget during AAM tiling, while final CRF emission independently enforces the 32-entry architectural limit. Placement is decoupled from address decomposition: a global physical bank group does not map trivially to a processing unit's local bank. While HD-FIMDRAM routes accesses through physical bank groups, FIMDRAM flattens global HBM bank coordinates to derive the specific processing unit and local bank position. Placement annotates these execution coordinates without mutating the underlying physical address, allowing mappings to be evaluated across alternative placement strategies.

5.5 Source-Backed Target Specializations

The shared pipeline establishes structure; target profiles determine the semantics at its extension points. Each target profile resolves its own source-backed ISA, address policy, allocation constraints, binary policy, and host-execution model.

5.5.1 HD-FIMDRAM Host-Driven Path

The HD-FIMDRAM profile targets the `hd_fimdram_v1` ISA. It excludes device-side control flow and MAD, relies on an implicit `BANK` operand for arithmetic, assigns independent destinations to ADD and MUL, and enforces accumulator ties for MAC [21]. Streaming and stationary operand roles map directly to hardware-mandated GRF-A/GRF-B bank constraints. As illustrated in Figure 5.6, command formation decouples address calculation from payload resolution. The physical-address policy models logical indices as DDR4-oriented transactions, deriving row and column coordinates while reserving the six low-order offset bits. In parallel, register assignment groups tied values and resolves allocation via graph coloring under bank constraints, supplying register identifiers independently of the DRAM address. The interference and linear strategies described in Section 5.4.3 provide alternative assignments within this same recipe. Command and packet materialization merges these streams, packing instruction metadata into the reserved low-order address bits once alignment is verified. The flow branches based on operation type: memory operations embed the allocated register identifier, whereas arithmetic operations encode a static CRF index. While arithmetic and configuration operations produce device-side instructions padded to the configured CRF capacity, general-purpose register movement is handled directly through host work.

5.5.2 FIMDRAM AAM Path

The FIMDRAM profile combines `fimdram_v1`, `fimdram_aam_v1`, and `hbm_aam_address_v1`, mapping semantic operations directly to AAM selected forms under published architectural encodings [11]. Figure 5.7 traces the lowering stages of this path in recipe execution order. Following scheduling and barrier insertion, control-flow finalization groups compatible operations into arithmetic replay blocks rather than emitting individual scalar CRF entries. Each replay block is bound by an explicit `JUMP` loop and terminated by an `EXIT` instruction. This pass strictly bounds program size against the 32-entry architectural CRF capacity before memory addressing begins.

The address policy first maps logical transactions to physical transactions, then decomposes them into Scheme8 intra-channel coordinates: bank, bank group, row, column, and zero offset. Direct transaction mapping preserves the logical transaction number at this first step. Consecutive logical transactions therefore remain consecutive before address decomposition. The alternative, AAM-aligned striping, distributes logical transactions using configurable lane and group strides. The lane stride sets the spacing between transactions within a stripe group, while the group stride separates successive groups. This changes the physical address fields without changing the workload's logical indices or the hardware decoder.

The distinction matters because AAM derives register identities from addresses. Striping can

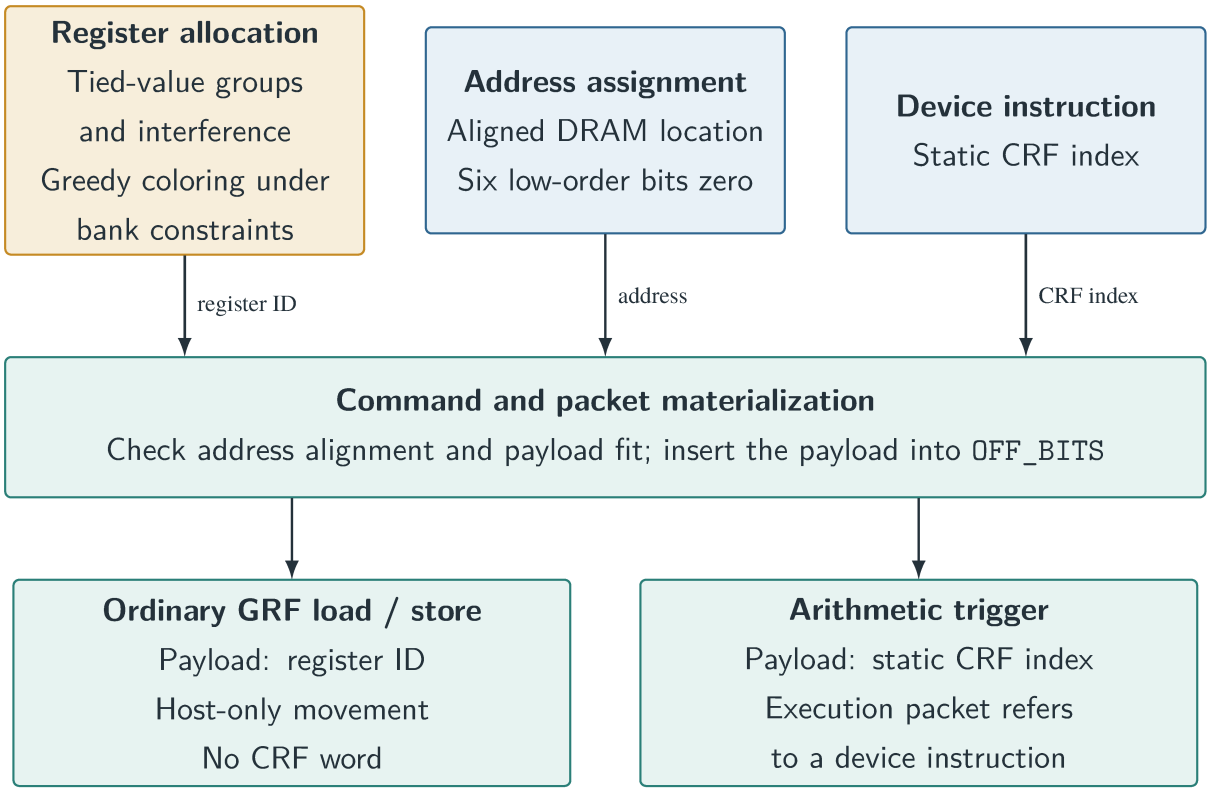


Figure 5.6: HD-FIMDRAM register allocation and low-order command-payload materialization.

arrange data so that operations fit a common replay group, whereas direct mapping may produce different register indices or conflicts between live values. Neither mapping changes the computation, and neither removes the need to check the resulting assignments. Section 6.3.2 measures how this policy choice affects address distribution, replay grouping, and compilation legality. Fixed AAM bit slices decode register indices directly from the row and column addresses; the address-aligned allocator enforces these hardware-derived indices, failing compilation if the required operand conflicts with the decoded location. Downstream lowering materializes execution triggers from this unified state. Each trigger expands into an abstract eight-command replay sequence capturing burst strides, broadcast scopes, and memory barriers. Placement, prologue generation, and binary emission finalize the lowering sequence, preserving clean isolation between CRF bounds verification, address-register binding, and host-command progression.

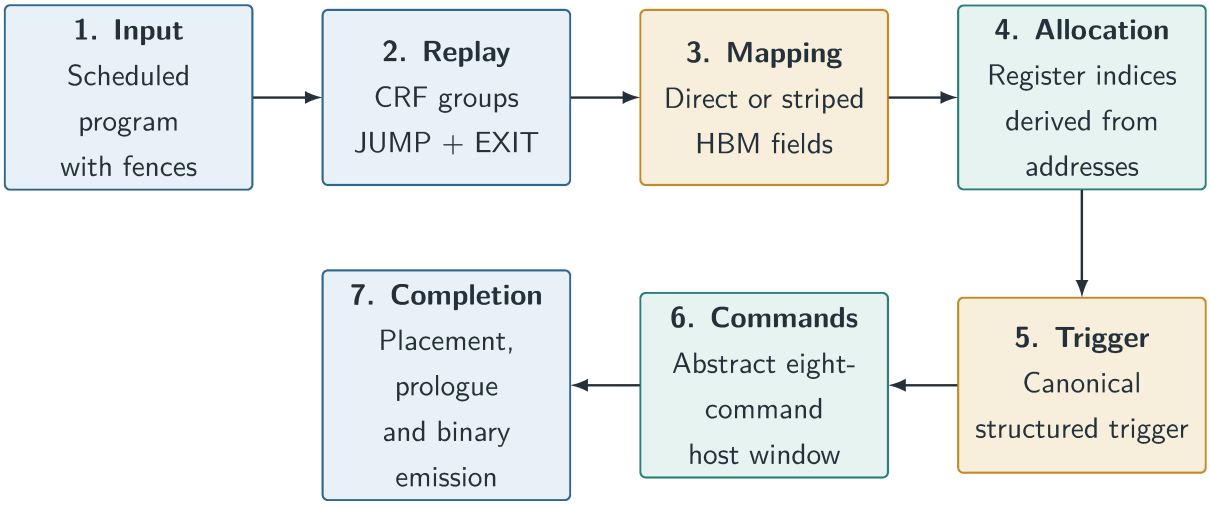


Figure 5.7: FIMDRAM AAM address, register, replay, and host-command materialization.

5.6 Final Artifact Materialization

Once addressing, allocation, placement, and target-specific hooks finalize the program representation, a prologue pass attaches setup metadata (target geometry, precision, data layout, and instruction mode) required for code emission. Binary emission then resolves each opcode against its target ISA rules, verifies operand forms, and executes the bound encoder functions while validating bit-field bounds. Figure 5.8 categorizes the resulting artifacts attached to the lowered `fimdram.launch` by role and target applicability. Common outputs comprise the emitted

machine words, ISA metadata, and binary coverage statistics. Target-specific structures branch naturally based on the active profile: HD-FIMDRAM generates padded device binary images alongside host execution packets and packet-rule models; FIMDRAM emits AAM trigger sequences and host-command descriptors derived from its earlier lowering passes. An optional assembly emitter provides a canonical, human-readable representation with explicit prologue, fence, and grouping annotations for inspection and verification. Both backends preserve original buffer identifiers and buffer-relative offsets across their final command artifacts. The backend ends at this compiler-artifact boundary. Runtime integration would bind the buffers to device allocations and use driver-established process mappings, following the responsibilities described in Section 3.1.4. This binding must preserve the alignment and address-field constraints assumed during lowering, particularly under AAM, where address fields also select registers [11]. The runtime would use the setup information and command artifacts to load the device program, perform the required mode transitions, and drive execution.

The static analysis pipeline runs after all lowering and verification contracts succeed. Rather than disassembling emitted binary words, the analyzer directly queries the structured models produced by upstream passes. This ensures that metrics such as register occupancy, memory layouts, and command counts remain traceable to their respective compilation stages. These consolidated artifacts form the basis of the comparative evaluation presented in Chapter 6.

5.7 Example: Following a Vector Addition Through Lowering

A vector addition kernel illustrates how these backend stages interact in practice. Consider the evaluation kernel `vector_add_n8`, which computes

$$c_i = a_i + b_i, \qquad 0 \leq i < 8,$$

within a target-neutral `cnm.launch`. As shown in Listing 1, the body loads two 16-bit integer operands, marks the first input as streaming and the second as stationary, and stores the result without specifying physical registers or memory addresses.

Conversion first lowers the launch construct and buffer types into their FIMDRAM equivalents, making memory accesses and arithmetic available as register-level operations. Instruction selection then maps the addition to a target-supported form: the single-use streaming load folds into an implicit `BANK` operand, while the stationary source remains an explicit register operand. Listing 2 shows the selected addition for the first element in each target's lowered body. In both cases, `%0` holds the stationary operand, while `implicit_memory_buffer_id` and `lindex` iden-

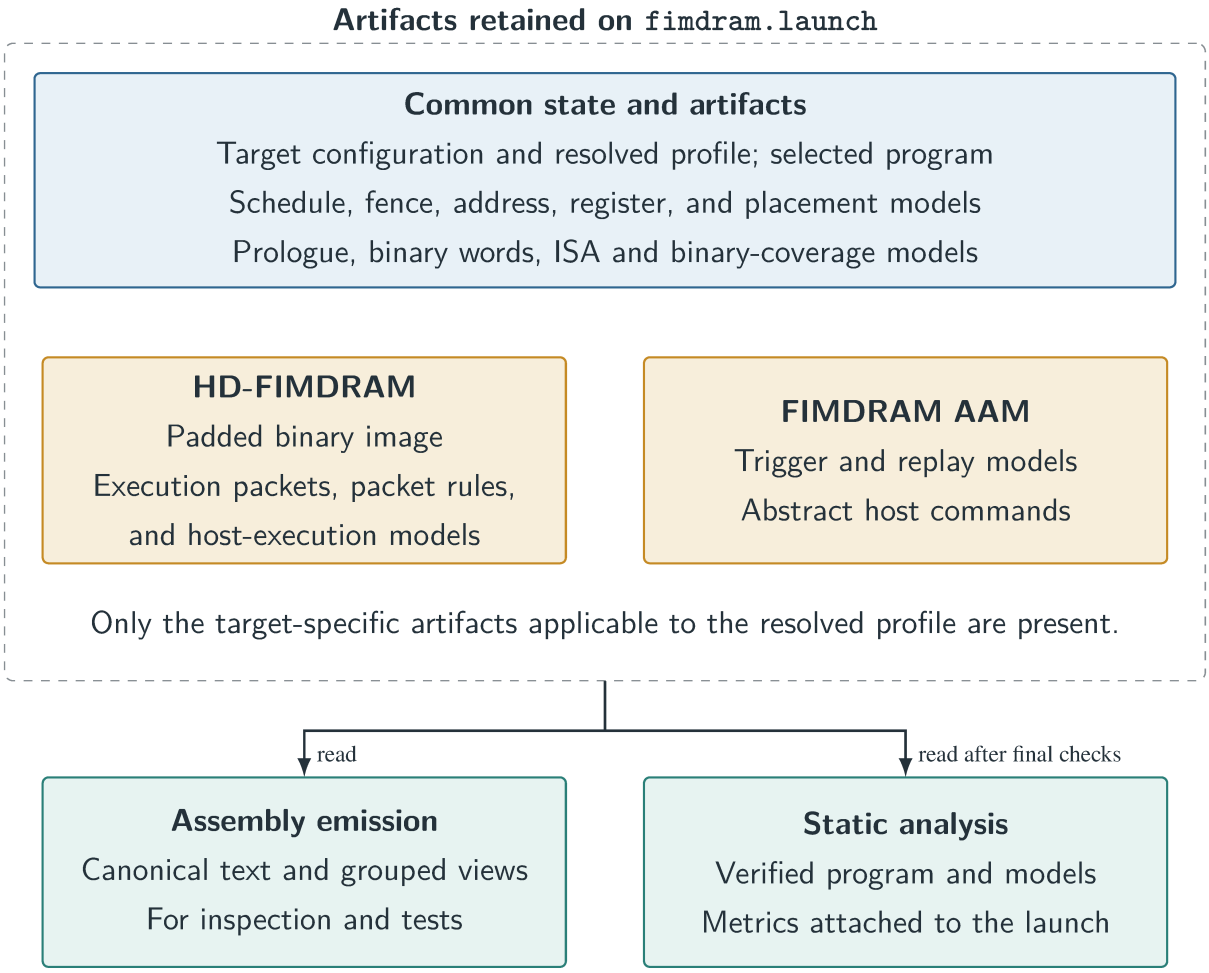


Figure 5.8: Self-describing compiler artifacts attached to a completed FIMDRAM launch.

```
affine.for %i = 0 to 8 {
  %a = affine.load %arg_lhs[%i] : memref<8xi16>
  %b = affine.load %arg_rhs[%i] : memref<8xi16>
  %sum = arith.addi %a, %b : i16
  affine.store %sum, %arg_out[%i] : memref<8xi16>
}
```

Listing 1: Loop body of the common input launch.

tify the streaming buffer and element index. The resulting operations explicitly distinguish the independent-destination form of HD-FIMDRAM from the address-aligned AAM form.

```
// HD-FIMDRAM: independent-destination form
%1 = fimdram.hd_fimdram_add_reg_immem %0 {
  implicit_memory_buffer_id = 0 : i64, lindex = 0 : i64
} : (!fimdram.grfa_reg<i16>) -> !fimdram.grfa_reg<i16>

// FIMDRAM AAM: address-aligned form
%1 = fimdram.add_reg_aam %0 {
  implicit_memory_buffer_id = 0 : i64, lindex = 0 : i64
} : !fimdram.grfa_reg<i16>
```

Listing 2: Selected additions in the lowered target bodies.

Table 5.2 compares the resulting lowering decisions across both targets. While HD-FIMDRAM reserves low-order address bits to encode command payloads, FIMDRAM decomposes structured HBM addresses to derive hardware-mandated register constraints.

Table 5.2: Lowering stages and target-specific decisions for the 8-element vector addition.

Stage	HD-FIMDRAM	FIMDRAM AAM
Arithmetic selection	Select independent-destination ADD with implicit `BANK` source.	Select AAM ADD bound to trigger-address register indices.
Program ordering	Schedule via dependencies and insert phase boundaries.	Schedule via dependencies, then finalize CRF replay loops after fencing.
Address assignment	Compute aligned row/column addresses with low-order bits clear.	Decompose HBM addresses and derive AAM register indices.
Register assignment	Allocate physical registers under liveness and bank constraints.	Bind values to address-derived registers and verify conflict-free liveness.
Materialization	Emit device binary words and pack host execution commands.	Construct trigger windows, apply placement, and emit binary records.

The two paths diverge visibly in how operations are scheduled and grouped into the final `program` attribute (Listing 3). HD-FIMDRAM flattens the kernel into eight explicit arithmetic records. Conversely, FIMDRAM compresses the operations into seven top-level records, where each load, addition, and store represents eight broadcast members executed via hardware replay under a local `jump` loop.

This divergence directly shapes final code generation. The HD-FIMDRAM program contains eight load records, eight ADD records, eight store records, and four fences, for 28 program

```
// HD-FIMDRAM: fully unrolled arithmetic sequence
program = [
  ...,
  {op = "hd_fimdram_add_reg_immem", lindex = 0 : i64, ...},
  ...,
  {op = "hd_fimdram_add_reg_immem", lindex = 7 : i64, ...},
  ...
]

// FIMDRAM AAM: compressed replay loop
program = [
  {op = "load_reg",    aam_broadcast_member_count = 8 : i64, ...},
  {op = "fence",       ...},
  {op = "add_reg_aam", aam_broadcast_member_count = 8 : i64, aam_replay_group =
  ↪  1 : i64, ...},
  {op = "jump",        aam_replay_group = 1 : i64, ...},
  {op = "fence",       ...},
  {op = "store_reg",   aam_broadcast_member_count = 8 : i64, ...},
  {op = "exit",        ...}
]
```

Listing 3: Structured `program` attributes for both targets.

records. Only the eight ADD records encode active device words; ordinary register movement belongs to host execution packets, and fences remain metadata. In FIMDRAM, the load, ADD, `JUMP`, store, and `EXIT` records encode five active words; the two fences account for the remaining records in its seven-record program. Both targets start from the same workload, but their program-record and binary-word counts describe different representations of its execution.

5.8 Verification and Test Implementation

Verification is distributed across compilation stages, matching the owner of each intermediate representation. At the front end, Operation Definition Syntax (ODS) rules and custom dialect verifiers enforce typed operation constraints, buffer bounds, and launch semantics. Static configuration checks then validate machine geometry and policy combinations before code lowering begins. To ensure consistency before processing any launch, a module-level contract pass statically validates the centralized target descriptions, checking opcode legality, encoder bindings, and execution capabilities. During compilation, a pipeline-contract pass re-evaluates the active target profile across six monitored checkpoints. These checkpoints track cumulative state invariants: profile freshness after initialization, operation legality after selection, address integrity after layout planning, and complete binary coverage following emission. Dedicated helper rou-

tines further validate packet bit-fields and FIMDRAM replay constraints. The regression test suite is structured around these lowering stages using `lit` and `FileCheck`. Test suites exercise each component across positive paths, malformed inputs, and hardware capacity boundaries, culminating in end-to-end recipe tests. Table 5.3 maps each verification concern to its owning compiler stage and test layer. As shown, invariants are enforced hierarchically: dialect verifiers guard well-formed IR, checkpoint passes ensure pipeline contract compliance, and target helpers validate packed binary layouts.

Table 5.3: Verification ownership and test coverage across the FIMDRAM implementation. Filled marks denote primary ownership and open marks denote supporting detection.

Owner	Typed IR	Geometry / policy	Stale model	Opcode / form	Address / register	Binary / packet	Experiment output	Principal lit-test family
Dialect verifier	●	–	–	○	–	–	–	Conversion, launch formation, typed operations
Target-config validator	–	●	–	○	○	–	–	Profiles and configurations
Program-schema stages	–	–	○	●	●	●	–	Program emission through materialization
Backend-table contract	–	●	○	●	○	●	–	Tables, tuples, recipes, rule coverage
Six pipeline checkpoints	○	●	●	●	●	●	–	Full recipes and cumulative states
Target-model helpers	–	○	○	○	●	●	–	Packets, FIMDRAM replay, packed fields
Experiment consistency gate	–	–	○	○	○	○	●	Resource and comparison integration

5.9 Extensibility and Implementation Trade-offs

Table 5.4 outlines three distinct mechanisms for extending the compiler backend. At the finest granularity, new algorithms can simply register an implementation behind existing abstract interfaces. Introducing new opcodes or ISAs requires updating the declarative X-macro tables, alongside only the selection or encoding logic with novel semantics. Lastly, supporting a new target involves defining a valid compatibility tuple and binding a coherent set of policies within a dedicated recipe, extending pipeline contracts whenever new invariants or required artifacts are introduced.

Table 5.4: Implementation, integration, and verification requirements across compiler extension scopes.

Scope	Implementation	Integration	Verification
Algorithm or policy	Implement the indexing, address, placement, or allocation interface.	Register the strategy and expose its identifier in configuration.	Unit validity and boundary rejection tests; comparative ablation checks.
Opcode or ISA	Add declarative X-macro rows, typed operations, selection rules, and encoders.	Specify legal operand forms and binary support per target profile.	Assembly generation tests, illegal-form rejections, and binary encoder checks.
Target architecture	Define target traits, custom policies, and specialized lowering hooks.	Register a validated profile tuple and bind policies within a recipe.	End-to-end recipe tests and contract checks for new invariants and artifacts.

This modularity keeps the experimental ablations local. The linear HD-FIMDRAM variant isolates register allocation by substituting only the assignment strategy, leaving the ISA, address mapping, and pass ordering untouched. The FIMDRAM mapping ablation modifies only the logical transaction layout within the HBM policy, preserving both the fixed AAM decoder and the surrounding lowering pipeline. These capabilities stem from deliberate design trade-offs across three areas. Dictionary attributes allow flexible, target-agnostic representations, leaving validation to staged contract passes rather than fixed IR types. Target-specific hooks within a shared sequence expose specialization points globally, but guarantee identical pass schedules across all comparative baselines. At the front end, the bounded static tiler lowers structured affine loops while avoiding the complex search heuristics of prior monolithic pipelines [21]. The resulting architecture addresses the representation and modularity goals of RQ1 and RQ4 while generating the contract artifacts required by RQ2 and RQ3. Chapter 6 leverages this infrastructure to evaluate both targets across representative workloads, analyzing the impact of architectural and policy decisions on the generated code.

Chapter 6

Experimental Evaluation

The implementation in Chapter 5 was designed as a compiler substrate for describing and exploring FIMDRAM-like targets, rather than as two independent target-specific code generators. This chapter evaluates that design at three levels. First, it establishes whether common target-neutral workloads traverse both source-backed profiles while preserving their declared semantic work and target invariants. Second, it measures the compiler-visible programs produced by the two profiles without treating unlike commands as interchangeable performance units. Third, it changes individual allocation and mapping policies, varies AAM tile shape, and probes resource boundaries to determine whether the backend exposes useful, controlled design-space observations. The measurements follow the source-work accounting in Section 4.5.2 and the static units and aggregation rules in Section 4.5.3. The discussion focuses on the compiler decisions responsible for the observed differences, within the evaluation boundary established in Section 4.6.

6.1 Experimental Corpus and Validation

To apply the evaluation method described in Section 4.5, the experimental evaluation exercises 168 configurations distributed across five investigation groups. Lowering 18 statically shaped kernels through both target profiles establishes the baseline with 36 primary configurations and 42 launch artifacts. The remaining scope evaluates design-space modularity and hardware envelopes: 72 configurations isolate register-allocation and memory-mapping policy substitutions, while 56 capacity configurations and four AAM tile variations probe resource exhaustion and tiling legality. The recorded test run passes all 245 backend regression tests. Separately, the corpus contains no pipeline-contract or model-consistency failures. The autoregressive inference discussed in Chapter 2 motivates near-memory execution through repeated matrix-vector operations. Here, small instances of these arithmetic patterns make instruction selection, register as-

signment, and address mapping easy to inspect and compare across targets. The capacity sweeps then increase output counts and reduction lengths to examine how these decisions respond to growing workloads. Moving towards inference-scale matrix-vector workloads would require extending the current output partitioning with reduction decomposition: separate launches would compute partial sums and a further step would combine them. This extension would introduce intermediate buffers, transfers of partial results, and ordering between dependent launches into the generated programs, as discussed in Section 7.4. The experiments in Section 6.4.3 identify the per-launch boundaries that motivate this next step.

Table 6.1 lists the evaluated problem sizes alongside their source arithmetic counts. The workloads separate independent output work from dependent accumulation, using the computations defined in Table 4.2. Vector addition produces n independent sums. SAXPY adds multiplication by a shared scalar, while element-wise multiply–accumulate reads a separate initial accumulator and two multiplicands for each result. The sizes $n = 2,4,8$ therefore increase the number of outputs without introducing dependencies between them. Dot product uses the same lengths differently: all n products contribute to one scalar, so increasing n lengthens its accumulation chain rather than adding outputs. Matvec and matmul combine both patterns. Matvec produces m results, each formed from k products; its primary cases first increase the output count from one to two at $k = 2$, then increase the reduction length to $k = 4$. Matmul produces an $m \times n$ output with k products contributing to each element. Its primary cases retain $k = 2$ while increasing the output matrix up to 4×4, allowing the comparison to examine two-dimensional output tiling as well as arithmetic selection. Source arithmetic counts assign one operation to each addition and multiplication, before fusion or replay grouping. Because each reduction adds every product to an initial accumulator, these counts are $2n$ for dot product, $2mk$ for matvec, and $2mnk$ for matmul.

Table 6.1: Primary workload dimensions and source arithmetic counts.

Workload	Dimensions	Source arithmetic
Vector addition	$n = 2,4,8$	n
SAXPY	$n = 2,4,8$	$2n$
Dot product	$n = 2,4,8$	$2n$
Matvec	$(m,k) = (1,2),(2,2),(2,4)$	$2mk$
Matmul	$(m,n,k) = (1,1,2),(2,2,2),(4,4,2)$	$2mnk$
Multiply–accumulate	$n = 2,4,8$	$2n$

The matmul generator assigns each output tile to a separate source launch, keeping the complete reduction for every element in that tile. A tile of t_m rows and t_n columns divides an $m \times n$ output into $\lceil m/t_m \rceil \lceil n/t_n \rceil$ independent launches. Thus, the $4 \times 4 \times 2$ primary kernel, generated

with 2×2 tiles, produces four compiled launches on both targets. The capacity sweeps in Table 6.2 then increase either the number of outputs or the work needed for each result. Increasing vector-add length n or matvec row count m adds independent work that the backend can partition across launches. The matmul output sweep instead uses explicit 1×2 source tiles, which remain legal through the sampled 8×8 output. Doubling both matrix dimensions quadruples the number of outputs and source tiles while keeping $k = 2$. Its launch growth therefore reflects source tiling, whereas vector addition tests backend partitioning of one source loop. The reduction sweeps hold the output count fixed: one scalar for dot product, two results for matvec, and a 4×4 matrix for matmul. Increasing n for dot product or k for the matrix kernels lengthens each accumulation. Since the backend does not split reductions into partial sums across launches, these cases test whether the longer computation still fits the target's register and program capacities.

Table 6.2: Workload dimensions varied in the capacity study under fixed hardware budgets.

Sweep	Sampled values	Fixed parameters
Vector-add outputs	$n = 8,16,32,64,128,256$	One source loop
Dot-product reduction	$n = 4,8,16,32,64,128$	Single reduction
Matvec outputs	$m = 1,2,4,8$	$k = 2$
Matvec reduction	$k = 4,8,16,32,64$	$m = 2$
Matmul outputs	$m = n = 2,4,8$	$k = 2$; 1×2 tiles
Matmul reduction	$k = 4,8,16,32$	$m = n = 4$; 2×2 tiles

Throughout these sweeps, each profile retains the resource settings in Table 6.3 (so only the workload grows, not the hardware budget). Both configurations employ a dependency-aware topological scheduler; FIMDRAM additionally enforces AAM bit-slice decoding and arithmetic replay loops. Table 6.4 outlines the controlled variables across all five experiment groups.

The configured lowering limit and the final device-program capacity apply at different stages. For HD-FIMDRAM, the tiler checks an early register-level operation estimate against the 64-operation limit, before instruction selection folds loads into implicit operands. This estimate does not count final execution packets. The FIMDRAM finalization pass instead checks the encoded device program, including replay control, against the 32-entry CRF. Early tiling decisions and final encoded-word counts therefore need not reach their limits at the same workload size.

Table 6.5 summarizes the compilation outcomes: every unsuccessful configuration failed with an explicit resource exhaustion or address-derived register collision diagnostic, while none produced a pipeline-contract or model-consistency violation. The status categories follow Table 4.3.

Table 6.3: Resource and transaction settings of the evaluated profiles.

Property	HD-FIMDRAM	FIMDRAM AAM
Processing units	4	8
GRF-A entries per unit	64	8
GRF-B entries per unit	64	8
SRF entries per unit	64	16
Configured lowering limit (`command_buffer_limit`)	64	32
Modeled device CRF capacity (words)	64	32
Transaction size (bytes)	64	32
Reported broadcast width	1	8

Table 6.4: Variables and controls across the five experiment groups.

Group	Varied quantity	Fixed baseline
Primary	Complete target profile (HD-FIMDRAM vs. FIMDRAM AAM)	Workload definitions and logical memory layouts.
Allocator	Graph-coloring vs. linear register allocation	HD-FIMDRAM target profile, workload, and pass order.
Mapping	Direct vs. striped logical-to-physical mapping	FIMDRAM target profile, AAM decoder, and scheduling.
Capacity	Workload dimensions (n, m, k)	Hardware capacities; matmul output sweep fixes 1×2 tiles.
AAM tile shape	1×1, 1×2, 2×1, or 2×2 tiles	Matmul $8 \times 8 \times 2$ under the FIMDRAM profile.

Table 6.5: Outcome and invariant summary for the five experiment groups.

Group	Outcome	Validated invariant
Primary	36 successes	Preserved source semantics; complete binary coverage with valid target-specific checks.
Allocator	36 successes	Identical aggregate counts for program records, addresses, commands, and binary words across paired runs.
Mapping	32 successes, 4 unsupported	Preserved semantics on common runs; matched all accepted AAM address-to-register bindings.
Capacity	40 successes, 5 unsupported, 11 compile failures	Verified strict resource boundary diagnostics upon capacity exhaustion.
AAM tile shape	2 successes, 2 unsupported	Confirmed exact expected register collision diagnostics on multi-row tiles.

6.2 Common Workloads Across Two Target Models

The primary evaluation establishes whether target-neutral workloads lower consistently across different execution policies. All 18 workload instances compile through both recipes without spills or unhandled opcodes, preserving their declared source arithmetic nodes and represented semantic instances. Each target also satisfies its respective invariants: HD-FIMDRAM passes all offset-alignment and payload-width checks, while FIMDRAM maintains agreement between address decodings and assigned registers.

Table 6.6 and Figure 6.1 report the resulting program-record and active-word counts for the largest instance of each family, normalized by source IR arithmetic operations. The accounting follows Sections 4.5.2 and 4.5.3, distinguishing source arithmetic, represented semantic instances, selected arithmetic records, and total program records. These differences show how the execution models affect the generated programs. For element-wise kernels (vector addition, SAXPY, and multiply–accumulate), FIMDRAM groups multiple semantic instances into single replay records, yielding program-record ratios below one. For vector addition, the ratio is 0.875, compared with 3.500 for HD-FIMDRAM, which retains separate arithmetic records and host-orchestrated register moves. In that case, HD-FIMDRAM emits 1.000 active device words per source arithmetic operation; the corresponding ratio is 0.500 for SAXPY and multiply–accumulate.

Table 6.6: Selected-program records and active binary words per source IR arithmetic operation at the largest primary sizes.

	Program records/source op		Active words/source op	
Workload	HD	FIMDRAM	HD	FIMDRAM
Vector add	3.500	0.875	1.000	0.625
SAXPY	2.250	0.500	0.500	0.375
Dot product	1.375	2.688	0.500	1.688
Matvec	1.688	1.438	0.625	0.938
Matmul	2.063	0.813	0.750	0.563
Multiply–accumulate	2.250	0.500	0.500	0.375

The dot-product case shows a different balance between arithmetic grouping and control records. At $n = 8$, FIMDRAM requires 43 program records, including 16 phase fences and nine control records, compared with 22 records and four fences in HD-FIMDRAM. Each accumulation consumes the preceding result, so the eight MAC steps remain separate arithmetic records rather than forming one independent broadcast group. The generated FIMDRAM program alternates operand loading and accumulation, retaining two fence records around each compute phase and a `JUMP` for each replay block; the final `EXIT` brings the control count to nine.

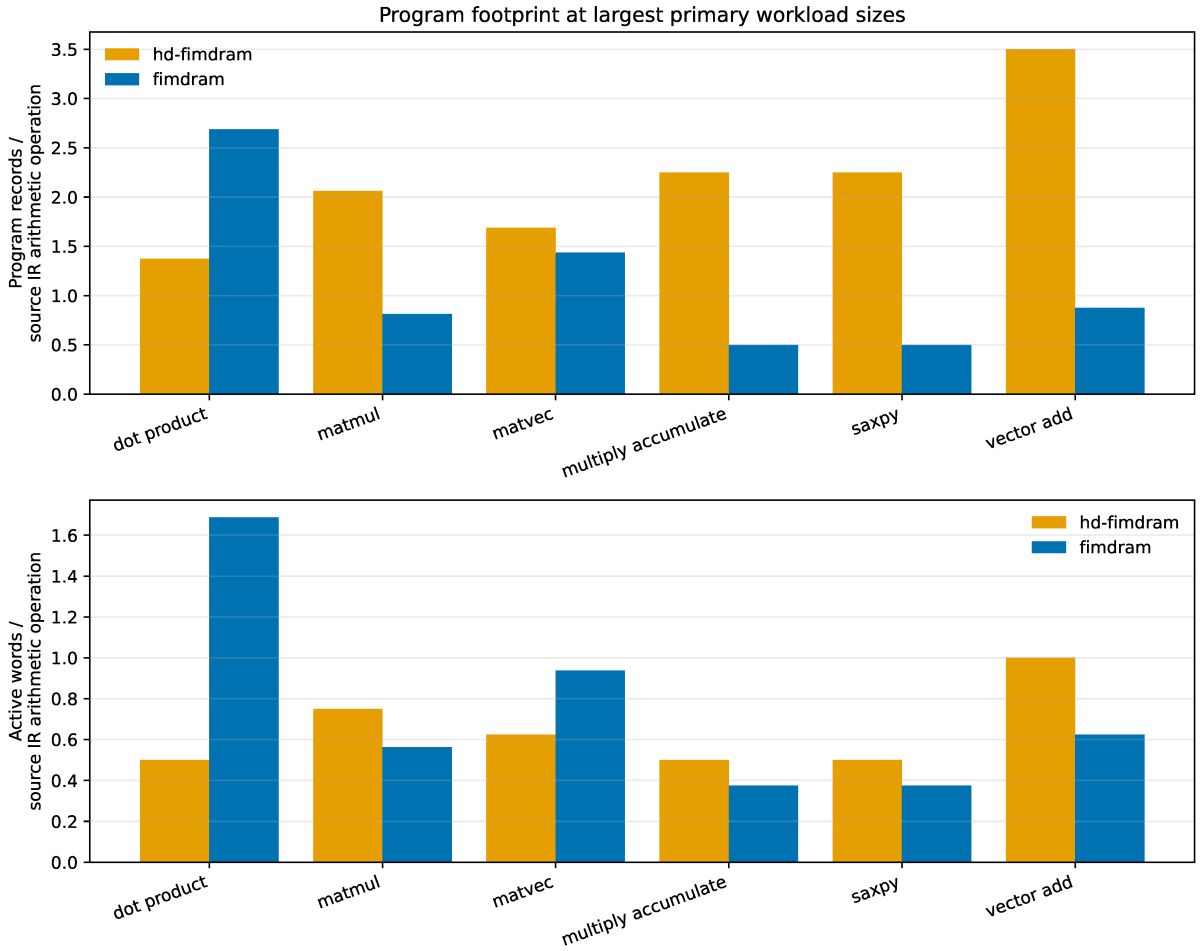


Figure 6.1: Selected-program records and active binary words per source IR arithmetic operation for the largest primary instances. Grouped FIMDRAM records may represent multiple semantic instances; the ratios describe compiler artifacts, not throughput.

Figure 6.2 shows these differences in program composition: FIMDRAM includes explicit `JUMP` and `EXIT` control records, whereas HD-FIMDRAM represents ordinary GRF loads and stores through host execution packets, without allocating CRF words to those transfers.

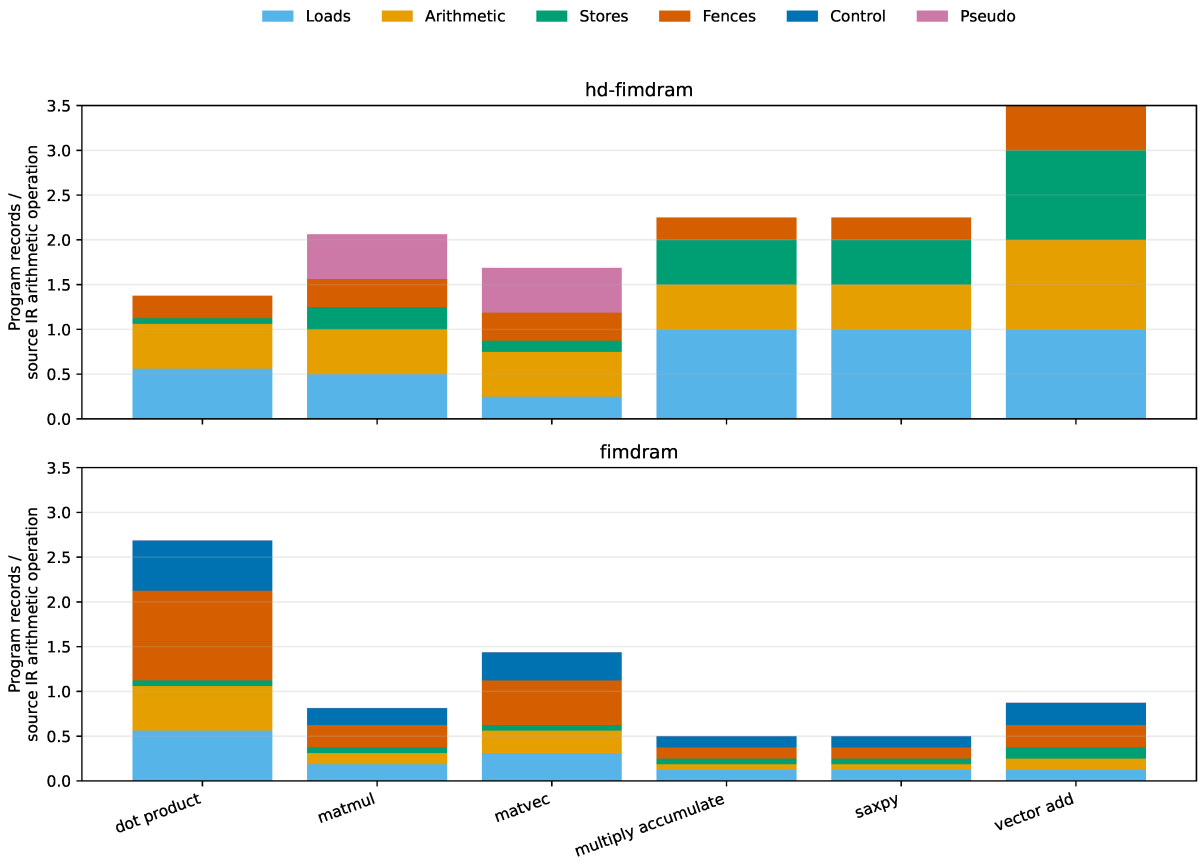


Figure 6.2: Target-specific selected-program composition at the largest primary sizes. FIMDRAM AAM broadcast grouping and HD-FIMDRAM host-driven movement produce different structures from the same source IR.

Host orchestration and register assignment also differ between the targets. For matmul $4 \times 4 \times 2$, the backend generates 100 HD-FIMDRAM execution packets and 192 expanded FIMDRAM HBM commands. These totals cover four launches generated with 2×2 source tiles. The output-scaling study uses 1×2 tiles and therefore eight launches for the same matrix dimensions, as specified in Section 6.1. The profiles use 64-byte and 32-byte transactions, respectively, and the packet and command counts remain distinct units. For dot product at $n = 8$, HD-FIMDRAM uses nine distinct physical register names, whereas FIMDRAM uses two under its address-derived allocation rules: `grfa0` for the loaded operand and `grfb0` for the tied accumulator. The latter remains in the same register across the dependent MAC chain, while successive operand loads reuse `grfa0`. This metric counts register names in the launch, not the total number of register cells across processing units; it does not multiply a name by the broad-

cast width. The compiler exposes these differences through separate measurements of programs, registers, and host commands, without treating target-specific commands as equivalent units of work.

6.3 Controlled Policy Substitution

Compiler abstractions for PIM support exploration by separating algorithmic policies from hardware constraints [39]. The two ablations isolate register allocation and logical-to-physical mapping while holding workloads, target ISAs, pass order, and contract checkpoints fixed.

6.3.1 HD-FIMDRAM Register Allocation

The first ablation substitutes HD-FIMDRAM's default interference graph allocator with a deterministic linear allocator across all 18 primary workloads. Section 5.4.3 describes how the two strategies assign and reuse registers. Both strategies compile without spills, preserving the eight checked aggregate counts: total program records, selected arithmetic records, source arithmetic operations, represented semantic instances, active binary words, addressed operations, fences, and host execution packets. Peak-live-slot and distinct physical-register counts also match. Both allocators receive the same scheduled IR and find legal assignments within the configured register budgets. Identical footprint metrics do not imply identical binary output, however. Table 6.7 illustrates this distinction for `vector_add_n2`: the linear allocator reverses the GRF-B bindings of the two loaded operands relative to the interference allocator. The register and program-record counts remain unchanged, but the operand encodings differ.

Table 6.7: Physical register assignments for vector addition at $n = 2$ under the two HD-FIMDRAM allocation strategies.

Assigned value	Interference	Linear
Loaded value at logical index 0	`grfb1`	`grfb0`
Loaded value at logical index 1	`grfb0`	`grfb1`
ADD result at logical index 0	`grfa0`	`grfa0`
ADD result at logical index 1	`grfa1`	`grfa1`

A focused mixed-bank test shows when constraint-aware coloring changes the outcome. With one GRF-A and two GRF-B slots, the interference allocator places a bank-flexible value in GRF-B, preserving GRF-A for a later A-only load. Order-dependent linear allocation consumes GRF-A first and is rejected. The primary workloads show no footprint advantage under their

configured budgets, while this test demonstrates a difference in legality. The shared backend allows both strategies to be substituted without changing the surrounding pipeline.

6.3.2 FIMDRAM Logical-to-Physical Mapping

The second ablation contrasts AAM-aligned striping with direct transaction mapping, holding the FIMDRAM ISA, address decoder, other policies, and pipeline fixed. Section 5.5.2 explains how these policies map logical transactions before address decomposition. In AAM, address fields determine physical register identities, so changing the mapping can alter replay grouping or cause conflicts between simultaneously live values. For dot product at $n = 8$, both policies produce 43 program records, 27 active binary words, and 144 abstract host commands, but distribute addresses differently. Figure 6.3 shows that striped mapping uses two distinct bank-field values and two row-field values, whereas direct mapping uses four bank-field values and one row-field value. Both use four bank-group values and one column value.

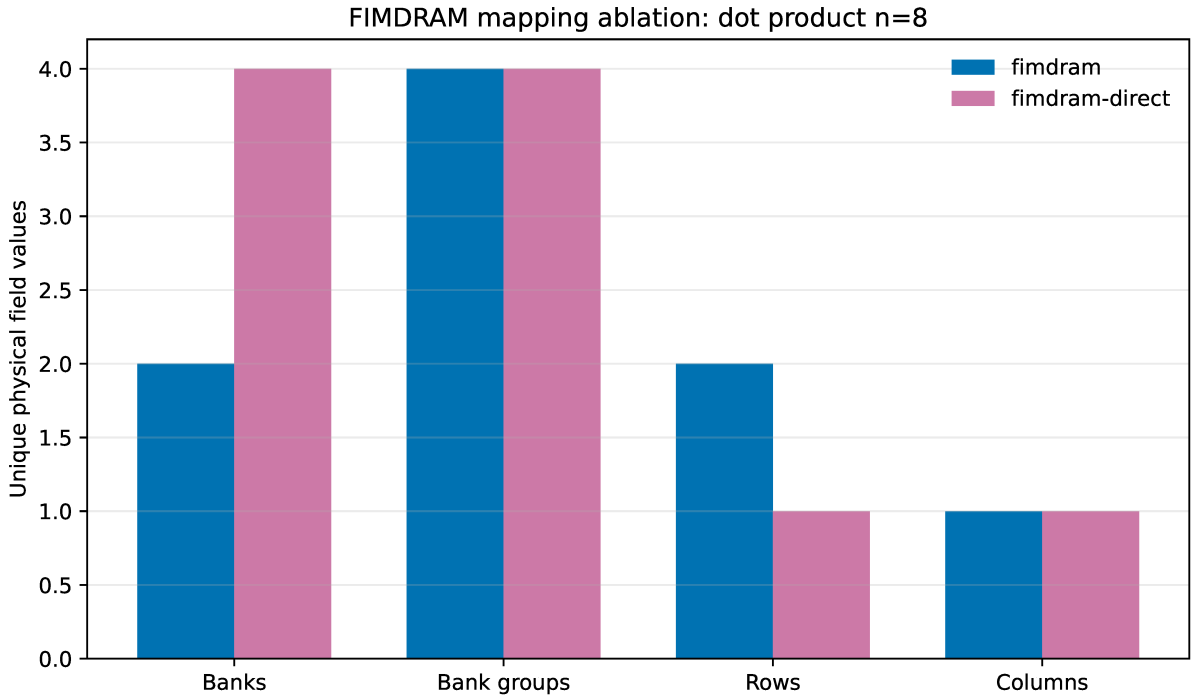


Figure 6.3: FIMDRAM mapping changes physical-field dispersion while preserving semantic work and the reported program, word, and command counts for dot product $n = 8$.

In element-wise workloads, mapping also changes program structure. For vector addition at $n = 8$, striped mapping forms one arithmetic replay group, producing seven program records and 24 commands. Direct mapping forms two arithmetic replay groups, producing 14 records and 48 commands for the same computation. Figure 6.4 also identifies four direct-mapping configurations classified as unsupported: `matvec_m2_k2`, `matvec_m2_k4`, `matmul_m2_n2_k2`,

and `matmul_m4_n4_k2`. In each case, simultaneously live values require the same address-derived register, `grfb0`.

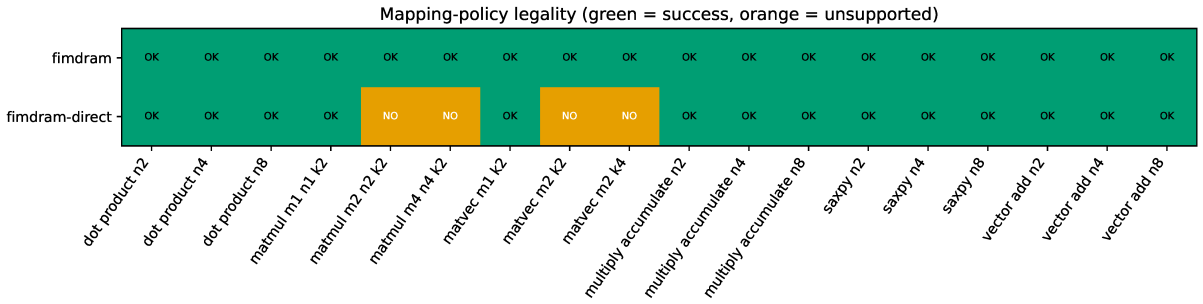


Figure 6.4: Legality of striped and direct FIMDRAM mappings across the primary corpus. AAM couples data layout to physical register identity, so policy substitution can alter legality as well as address dispersion.

The 32 successful configurations and four diagnosed rejections show that the same backend can compare alternative mappings and identify both changes in generated programs and configurations that violate register constraints.

6.4 Resource-Sensitive Scaling and Boundaries

The capacity sweep evaluates how compiler lowering scales when increasing independent output elements versus dependent reduction lengths under fixed target resource budgets. Because reductions cannot cross launch boundaries without multi-launch accumulation support, the backend can partition independent iterations into separate launches while reductions must fit within a single launch. These boundaries reflect the limits of the current static lowering under modeled target constraints rather than intrinsic hardware capacity. Vector addition serves as the representative element-wise workload for output scaling, while SAXPY and explicit multiply–accumulate are evaluated in the primary comparison. Dot product evaluates single-output reduction scaling, and matvec and matmul cover both output and reduction dimensions.

6.4.1 Output Scaling and Launch Decomposition

Table 6.8 compares automatic backend loop partitioning with explicit source-level tiling across scaled output dimensions. For vector addition at $n = 256$, FIMDRAM produces twelve launches of 20 elements and a final launch of 16, giving $12 \times 20 + 16 = 256$ elements across 13 launches. HD-FIMDRAM produces sixteen launches of 16 elements. These splits occur during bounded launch-body lowering, before the final program is selected and encoded. A vector-add element initially requires two loads, an addition, and a store; the HD-FIMDRAM 64-operation expansion

budget accommodates 16 such elements. After selection folds one load into the ADD's implicit operand, each of its launches contains 52 program records, including four fences, but only 16 active device words. The FIMDRAM launches contain between nine and 21 active words, all below the 32-entry CRF capacity. These final counts reflect selection and replay grouping after the tiler has already divided the source loop. Matvec at $m = 8, k = 2$ fits within a single launch for both targets, producing 61 HD-FIMDRAM records and 17 FIMDRAM records.

Table 6.8: Launch counts at the largest sampled vector-add and matvec output sizes and across the matmul output sweep.

		Launches		
Workload	Dimensions	HD-FIMDRAM	FIMDRAM	Decomposition
Vector add	$n = 256$	16	13	Automatic loop partitioning
Matvec	$m = 8, k = 2$	1	1	No split required
Matmul	$m = n = 2, k = 2$	2	2	Explicit 1×2 output tiles
Matmul	$m = n = 4, k = 2$	8	8	Explicit 1×2 output tiles
Matmul	$m = n = 8, k = 2$	32	32	Explicit 1×2 output tiles

For matmul, launch growth follows the predefined 1×2 output tiles rather than automatic backend partitioning: each source launch computes one tile, scaling launch count directly with output dimensions.

6.4.2 AAM Tile-Shape Legality

The fixed matmul $8 \times 8 \times 2$ experiment evaluates how 2D launch tiling interacts with AAM register constraints across four candidate tile shapes. As reported in Table 6.9 and illustrated in Figure 6.5, 1×1 and 1×2 tiles compile successfully, whereas 2×1 and 2×2 tiles trigger compilation rejections due to address-derived register collisions on `grfb0`. All four configurations represent identical source computation (256 arithmetic operations and 128 semantic instances).

Table 6.9: FIMDRAM AAM tile-shape outcomes for fixed matmul $8 \times 8 \times 2$.

Tile	Status	Launches	Program records	Active words	Commands
1×1	success	64	832	576	3,072
1×2	success	32	416	288	1,536
2×1	unsupported	–	–	–	–
2×2	unsupported	–	–	–	–

Widening the tile horizontally from 1×1 to 1×2 halves the number of launches, program

records, active words, and host commands by mapping two output columns into a single launch without altering live-range compatibility. Expanding the tile vertically across rows causes simultaneously live operands to decode to the same physical register index. While multi-row tiles compile legally in other geometries (such as 2×2 for $4 \times 4 \times 2$), scaling the leading dimensions alters the bit fields supplied to the AAM decoder. Legality is therefore an emergent property of tensor dimensions, tile geometry, and address mapping evaluated together.

The first 2×1 tile makes this conflict concrete. The first MAC for output $(0,0)$ uses logical index 0 in the left-hand input matrix; the first MAC for output $(1,0)$ uses index 2 because $k = 2$. Table 6.10 reports their trigger addresses from the rejected launch. The selected decoder forms the GRF-B index from column bits 3–4 and row bit 0, as specified by the compiler's interpretation of Samsung Figure 5(e). The two accesses occupy different bank groups, but both have row zero and column zero. Since those register-selection bits agree, both decode to index zero despite having different byte offsets. Different tensor rows therefore need not produce different register indices. The first accumulator remains live when the second is loaded, so their tied MAC chains require overlapping assignments to `grfb0` under the selected allocation model. The compiler rejects this overlap rather than overwriting one accumulator.

Table 6.10: First MAC trigger for each output in the first rejected 2×1 tile of matmul $8 \times 8 \times 2$. Offsets are relative to the left-hand input buffer; register indices follow the selected decoder.

Output	Input index	Byte offset	Bank group	(Row, column)	GRF-B index
$(0,0)$	0	0	0	$(0,0)$	0
$(1,0)$	2	128	1	$(0,0)$	0

6.4.3 Per-Launch Reduction Boundaries

Increasing reduction length adds accumulation steps to each output element rather than independent work. The sweep scales n for dot product and k for matvec and matmul at fixed output dimensions. Table 6.11 lists the first resource exhaustion encountered for each target, reporting the last successful and first rejected sampled lengths (these pairs do not establish the exact maximum legal reduction length). Figure 6.6 compares these capacity envelopes.

The diagnostics isolate register exhaustion from program storage constraints:

- **Register limits:** Stop the HD-FIMDRAM dot product at $n = 64$ and the FIMDRAM matmul at $k = 16$ (exhausting GRF-A).

- **Instruction storage:** FIMDRAM dot product and matvec fail at $n = 16$ and $k = 8$. The finalization diagnostics report requirements of 51 and 43 device CRF entries, respectively, against

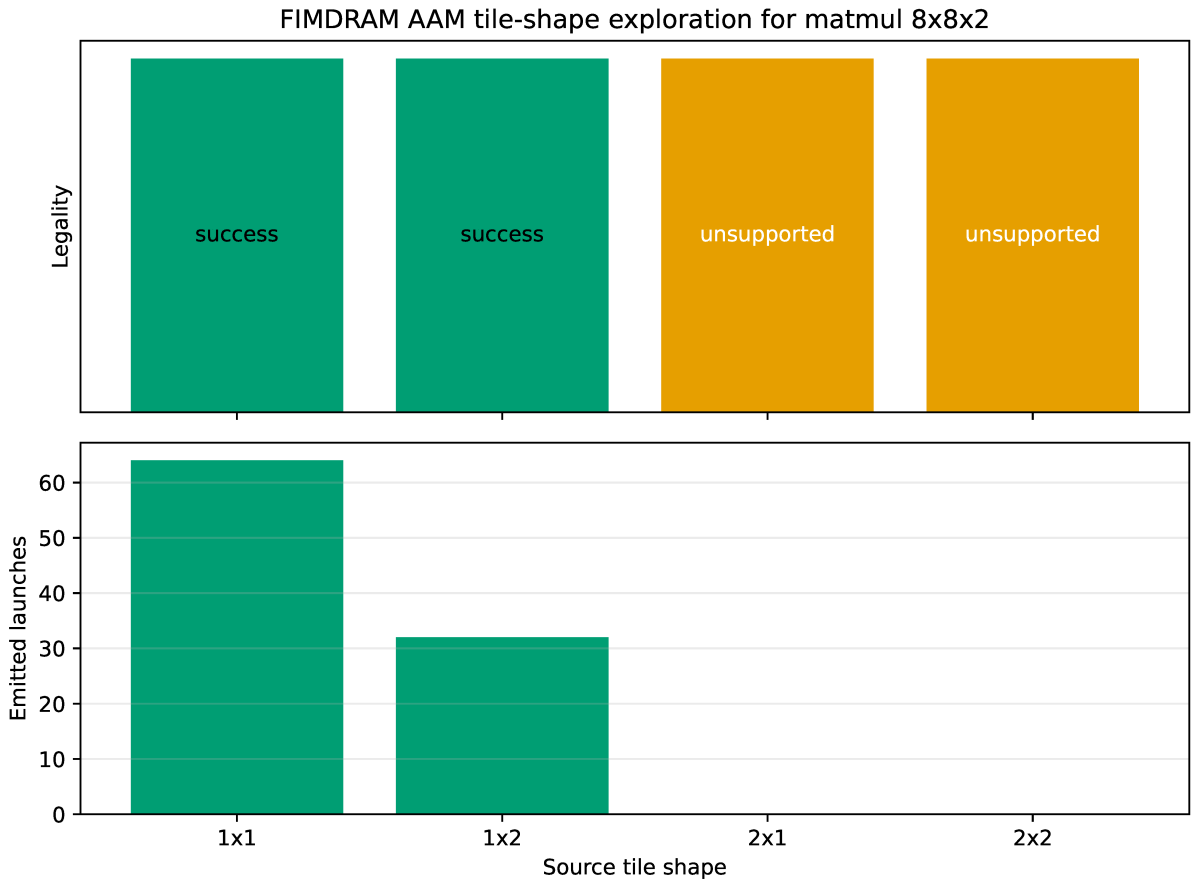


Figure 6.5: AAM tile-shape design-space probe for fixed matmul $8 \times 8 \times 2$. Wider legal tiles reduce static artifacts, while adding a second row conflicts with the selected address-to-register mapping.

Table 6.11: Observed per-launch reduction boundaries and first-failure diagnostics.

Workload	Target	Last success	First failure	Diagnostic category
Dot product	HD-FIMDRAM	32	64	Register file capacity exhausted
Dot product	FIMDRAM	8	16	CRF capacity (32 entries) exceeded
Matvec	HD-FIMDRAM	16	32	Early expansion budget (64 operations) exceeded
Matvec	FIMDRAM	4	8	CRF capacity (32 entries) exceeded
Matmul	HD-FIMDRAM	8	16	Early expansion budget (64 operations) exceeded
Matmul	FIMDRAM	8	16	GRF-A capacity exceeded

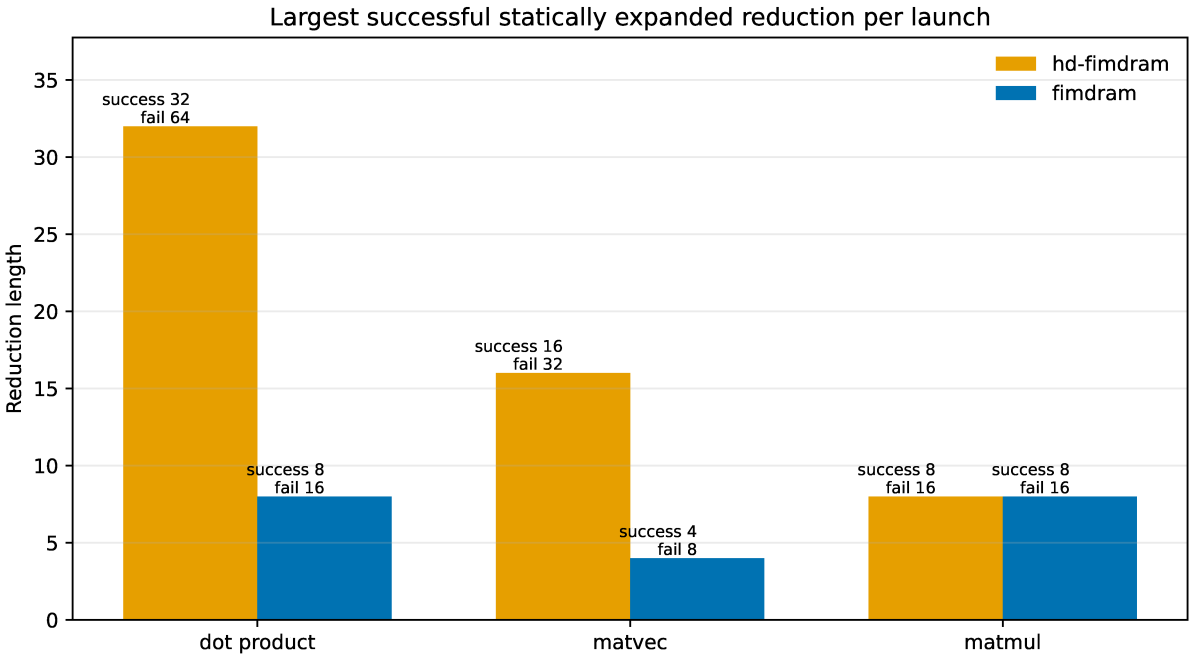


Figure 6.6: Current per-launch reduction envelope under resolved CRF, register, and command-buffer capacities. Independent work may be tiled; dependent reductions are not automatically decomposed.

the 32-entry capacity. These counts include encoded `JUMP` and `EXIT` control instructions, but exclude metadata-only fences; they are not total program-record counts. In particular, the 43-entry matvec requirement is a different measurement from the 43-record dot product at $n = 8$ in Section 6.2.

- **Early expansion limits:** For HD-FIMDRAM, the tiler estimates 98 register-level operations for one output row of matvec at $k = 32$: an initial-accumulator load, 32 pairs of operand loads with a MAC, and a final store give $1 + 3 \times 32 + 1 = 98$. For matmul at $k = 16$, one iteration of the outer row loop covers both columns of a 2×2 output tile, giving $2(1 + 3 \times 16 + 1) = 100$ operations. Each exceeds the configured 64-operation budget before instruction selection and packet generation. The latter count is unrelated to the 100 final execution packets summed across four launches of the smaller $4 \times 4 \times 2$ primary workload.

Decomposing reductions across multiple launches with partial accumulators would provide a way to extend the current lowering beyond these boundaries, as discussed in Section 7.4.

6.5 Evaluation Synthesis

The experiments establish four properties of the compiler backend. First, a single target-neutral workload corpus traverses both source-backed FIMDRAM recipes while satisfying all declared

source-work and model-consistency gates. Second, the shared pipeline preserves architectural distinctions: address-derived versus explicitly allocated registers, active program words, host artifacts, synchronization, and physical footprints remain measurable in the normalized output. Third, allocation and mapping policies can each be substituted while keeping the other policies fixed: paired validation isolates allocator differences, while mapping and tiling experiments expose the coupling between address layout, replay grouping, and register legality under AAM. Fourth, resource-aware tiling and staged verification translate target constraints into observable feasibility boundaries and diagnostics.

These findings demonstrate the viability of the modular FIMDRAM backend within Cinnamon for design-space exploration. A configurable compiler abstraction isolates hardware differences and exposes resource, mapping, and execution choices to unified analysis, consistent with broader PIM compiler frameworks [39]. In this context, FIMDRAM AAM and HD-FIMDRAM serve as concrete case studies that validate the backend architecture and enable controlled comparative ablations. While static compiler artifacts filter infeasible configurations, preserve semantic work, and evaluate policy trade-offs, they do not determine runtime execution time or energy consumption. The next chapter interprets these results against the research questions, examines threats to validity, and identifies the runtime, simulation, and compiler extensions required to advance from static exploration to system-level design decisions.

Chapter 7

Discussion and Future Work

The evaluation demonstrated that the FIMDRAM extension is more than a pair of working lowering paths. It provides a common compiler structure in which target semantics remain explicit, incompatible states are rejectable, selected policies can be changed independently, and resource consequences can be exported for analysis. This chapter interprets those findings against the research questions and the broader objective of enabling compiler-supported FIMDRAM design-space exploration. It then qualifies the evidence and identifies the work required to connect static feasibility with system-level evaluation.

7.1 Answers to the Research Questions

This section evaluates the implementation and experimental results against the four research questions, examining backend modularity, contract guarantees, target-specific artifact differences, and policy exchangeability.

RQ1: Can one Cinnamon backend compile the same workload for FIMDRAM AAM and HD-FIMDRAM, while keeping target-specific decisions separate from shared compiler stages? The implementation supports this across the evaluated target subset. Both profiles enter through a common `cnm.launch` boundary, share the target-facing dialect and selected-program representation, and follow an identical recipe structure. Target differences in ISA legality, addressing, logical mapping, register allocation, placement, binary encoding, and execution resolve through declarative descriptions and modular policy interfaces. All 36 primary configurations satisfy the source-work and model-consistency gates. The shared pipeline accommodates distinct execution models: grouped AAM replay and HD-FIMDRAM scalar materialization diverge in instruction selection, register semantics, active binary words, host commands, synchronization, and launch layouts. Hierarchical abstractions and progressive lowering isolate these differences at explicit stage boundaries, following the extensibility principles of Cinnamon and

MLIR [5], [19]. These guarantees apply to the modeled profiles and their implemented recipe subsets.

RQ2: What errors can the backend detect during compilation, and what can its checks guarantee about the generated program? Static backend checks validate target relations prior to recipe execution, while cumulative contracts and operation verifiers constrain each lowering stage. Across the test suite, all 245 unit and integration tests pass, alongside 11 end-to-end experiment checks verifying semantic accounting, legality outcomes, diagnostics, and AAM address-register consistency. The capacity and tiling experiments demonstrate conservative failure behavior: five capacity points are marked unsupported, 11 fail with explicit resource diagnostics, and two tile shapes are rejected due to live `grfb0` collisions without silent data truncation. No configuration reports an unexpected contract or model-consistency failure. These checks verify internal compiler consistency against the modeled target specifications; their limits regarding hardware execution fidelity are examined in Section 7.3.

RQ3: How do the generated instructions, register assignments, memory layouts, and host commands differ between the two target models for the same workload? The evaluation metric schema separates source IR operations, represented semantic instances, target instruction records, transaction geometry, and broadcast multiplicity. This decoupling distinguishes grouped FIMDRAM replay from reductions in underlying computational work. The resulting artifacts diverge in instruction counts, active binary words, fence placement, physical address layouts, host commands, and launch decomposition. In the largest matmul workload, the 100 abstract HD-FIMDRAM packets and 192 expanded FIMDRAM commands reflect distinct control structures and transaction widths rather than equivalent runtime events. Similarly, explicit register allocation and address-derived register indices exhibit target-specific pressure characteristics. The normalized schema exposes these structural differences without collapsing them into unified performance estimates.

RQ4: Can compiler strategies, such as register allocation and memory mapping, be changed without modifying the rest of the compilation flow? The allocator experiment confirms policy modularity for the register allocation interface: all 36 configurations compile successfully, and the eight checked aggregate counts remain invariant between linear and interference-based allocation. While overall metrics are identical on the primary corpus, a constrained-bank test demonstrates a case where interference coloring succeeds while linear allocation fails. The mapping experiment demonstrates exchangeability within architectural constraints. Substituting logical transaction mapping preserves the surrounding pipeline across 32 successful configurations, alters replay grouping for element-wise workloads, and rejects four multi-output reductions due to AAM live-register conflicts. The tile-shape sweep reflects similar coupling: 1×2 tiles compile legally and halve launch, record, active-word, and command counts

relative to 1×1, whereas two-row tiles are rejected due to register collisions for the $8 \times 8 \times 2$ workload. These results demonstrate modular policy replacement for allocation and mapping. The separate tile-shape experiment shows how source tiling interacts with address-derived register constraints, further identifying which configurations the backend can compile.

7.2 Compiler-Driven FIMDRAM Design-Space Exploration

The primary contribution of this thesis applies software engineering principles to system-level architecture: developing a modular, verified Cinnamon compiler backend for FIMDRAM design-space exploration. The backend transforms target-neutral CNM programs into source-traceable artifacts, exposing architectural and compiler trade-offs to systematic analysis. While sharing a common bank-level architecture, FIMDRAM AAM and HD-FIMDRAM diverge across address mapping, register models, instruction storage, and host-control semantics [11], [21]. Supporting both targets within a unified backend validates the decoupling between shared lowering mechanisms and target-specific policies. PIM design-space exploration couples hardware parameters with software mapping strategies. For instance, NicePIM treats hardware configurations, model mapping, DRAM capacity, and data layout as an interdependent search space [16]. Similarly, bank-level CNM research employs parametric FIMDRAM-derived templates to evaluate memory interfaces and compute resources [15]. In these exploration workflows, the compiler must determine whether a workload can be represented under candidate constraints, expose lowering decisions explicitly, and export structured artifacts for downstream evaluation. The implemented backend fulfills this role by structuring the exploration pipeline: target profiles and policies define candidate configurations, recipes generate target programs, compatibility checks, resource checks, and pipeline contracts reject incompatible configurations or invalid lowering states, and the analysis schema records instruction, address, register, binary, and command properties. As demonstrated by the capacity sweep, feasibility filtering is an integral exploration result, identifying where workload scaling alters launch decomposition or encounters resource limits under modeled target budgets. The framework provides an extensible foundation for integration with search heuristics, cost models, and architectural simulators, while runtime performance and energy ranking depend on the extensions outlined in Section 7.4.

7.3 Limitations and Threats to Validity

The results establish consistency within the implemented compiler models, but generation and verification are not independent. In the FIMDRAM AAM path, for example, address assignment and register checks use the same interpretation of Samsung's published address fields [11]. They

can therefore agree even if that interpretation is incomplete or incorrect. Negative tests and pipeline contracts help detect violations of the model; an independent executable reference or hardware validation would be needed to check the interpretation itself. Section 4.6 describes the source and evaluation boundaries underlying these results.

The experiments also provide a bounded test of modularity. The two targets exercise different register and command mechanisms within the same architectural family, and the workload corpus consists of small, statically expanded kernels. Their successful compilation and the controlled policy substitutions show that the shared interfaces support the variations tested here. They do not yet show whether those interfaces are sufficient for substantially different CIM or CNM devices or more complex applications. Additional targets and workloads would test whether new requirements fit the existing policy interfaces or require changes to the shared lowering structure.

7.4 Future Work

The evaluation identifies three directions for extending the backend: connecting generated artifacts to simulation, using compilation and simulation results to guide automated exploration, and supporting computations and targets beyond the current experiments. These extensions would build on the shared lowering pipeline while adding the execution models, search strategies, and transformations needed for broader architectural studies.

7.4.1 Compiler–Simulator Evaluation Loop

The primary extension of this work would connect external simulators to the verified artifact boundary of the compiler pipeline. While the backend isolates target-specific candidates and rejects resource-infeasible points, ranking accepted programs by runtime performance requires dynamic execution metrics such as latency and bandwidth. Energy comparisons would additionally require a validated energy model. Figure 7.1 positions the proposed simulator adapter outside the lowering pipeline, consuming outputs after final contract verification. The experiment driver would pass the selected program, resolved configuration, buffer-relative addresses, register assignments, binary metadata, and host descriptors across a stable simulation interface.

This architecture would support three progressive evaluation levels:

- **Functional simulation:** Would execute the declared opcode subset, validating numerical results and state transitions across replay loops, packets, and synchronization fences.
- **Timing-aware memory modeling:** Would combine compiler artifacts with DRAM geometry and timing parameters to model dynamic cycle counts, memory stall times, and effective

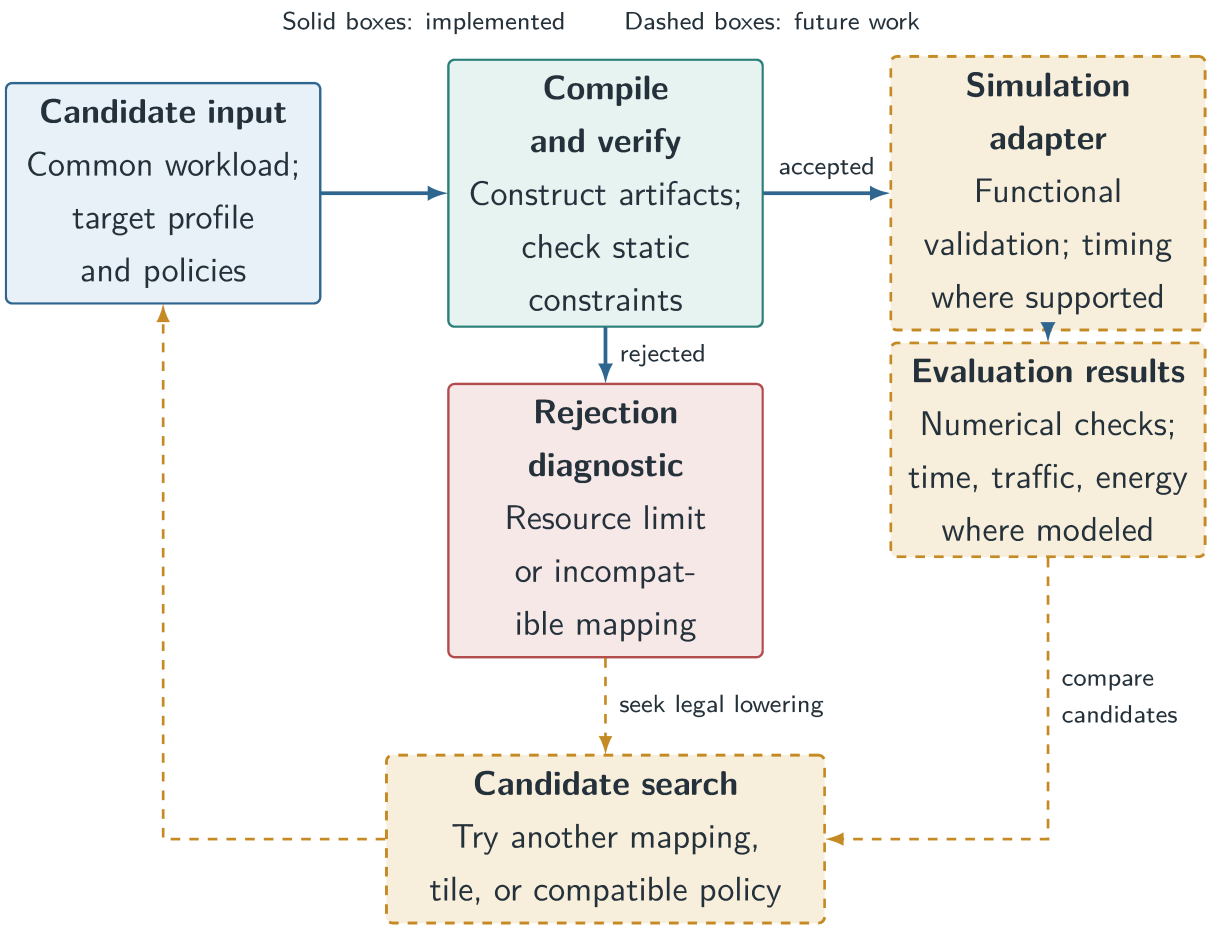


Figure 7.1: Proposed compiler–simulator and search loop. Solid boxes denote implemented components; dashed boxes denote future work. Diagnostic feedback guides legal compilation, while evaluation feedback supports ranking among accepted candidates.

bandwidth [8].

- **Full-system simulation:** Would evaluate host-device execution, cache hierarchies, and memory scheduling, following integrated environments such as PIMSys [26].

The experiment driver would integrate dynamic measurements into the existing provenance schema, recording compiler inputs, resolved memory configurations and simulator models.

7.4.2 Diagnostic-Guided and Automated Exploration

The feedback paths in Figure 7.1 would enable two distinct exploration mechanisms:

1. **Diagnostic feedback:** Would use compiler rejection diagnostics to adjust configurations before simulation. Detected constraints, such as AAM register collisions or CRF capacity exhaustion, could prompt alternative logical mappings or tile shapes and, once implemented, reduction decomposition.

2. **Evaluation feedback:** Would use simulation metrics, such as execution cycles, transferred bytes, and energy where supported by a validated model, to guide candidate selection across the configuration space.

To navigate spaces spanning register budgets, buffer limits, tile shapes, and placement policies without exhaustive enumeration, the exploration loop could combine static feasibility checks with iterative sampling and learned filtering. NicePIM provides an example of search over a joint hardware-and-mapping space, while parametric bank-level CNM studies expose variations in compute and memory resources [15], [16]. Within this workflow, candidate tile shapes would be generated, checked for static legality against address-mapping constraints, and passed to simulation only when verified.

7.4.3 Lowering Extensions and Broader Validation

At the compiler level, the capacity experiments identify reduction decomposition as the next lowering enhancement. While the current tiler partitions independent output spaces, a reduction-decomposition pass would split accumulation across multiple launches, manage intermediate partial-sum buffers, combine the partial results, and enforce inter-launch data dependencies. The input interface could also expand upward. Moving from statically shaped launch regions to higher-level tensor and linear-algebra operations within Cinnamon's dialect hierarchy [5] would expose application-level data distribution and kernel offloading to the exploration loop.

Supporting multiple applications would also require resource management in the execution stack. The runtime and driver arrangement described in Section 3.1.4 provides process mappings to device memory and configuration space [21, Section 3.4]. A multi-application extension

should isolate each process's buffers and coordinate access to processing units, loaded microprograms, and operating modes. In particular, it should prevent one application from accessing another's data or replacing device state while another kernel is executing. The compiler would still describe each kernel's layout and execution requirements; the runtime and driver would manage resource ownership and decide when that kernel can run.

Broadening the architectural scope would involve integrating targets with alternative scheduling, placement, or instruction-storage models, supported by dedicated compatibility tables and simulator adapters. Differential execution tests could compare numerical results against an independent reference. Where hardware or vendor runtime environments are available, they could provide additional checks of generated programs.

Chapter 8

Conclusion

This thesis investigated whether Cinnamon can serve as a modular compiler for exploring FIM-DRAM-like compute-near-HBM architectures. The central challenge was not merely to add another instruction emitter. A useful exploration compiler must retain target semantics across lowering, compose coherent architecture and compiler policies, reject invalid candidates, and expose the resulting programs in a form that supports controlled comparison. The work addressed this challenge through one FIMDRAM extension and two source-backed case studies whose address, register, instruction-storage, and host-control models differ substantially.

8.1 Thesis Outcome

This work extends Cinnamon below its target-neutral `cnm.launch` boundary with a common backend for FIMDRAM AAM and HD-FIMDRAM. A dedicated dialect preserves computation and buffer structure, while a selected-program representation records the decisions made during scheduling, addressing, register assignment, and emission. Declarative target descriptions and policy interfaces separate the modeled architectural constraints from the algorithms that lower a workload under those constraints. The two target paths demonstrate why that separation matters. HD-FIMDRAM requires explicit register allocation and low-order command payloads; FIMDRAM AAM couples register identity to the address mapping and uses CRF replay. The backend retains those differences within shared lowering stages, rather than requiring a separate compiler for each execution model. Compatibility checks and cumulative contracts make the resulting decisions inspectable and reject inconsistent intermediate states. The main contribution is therefore the compiler structure that makes these target variations manageable. Its output includes the selected program and the address, register, binary, and host-command models needed to explain how a candidate was formed. The accompanying evaluation flow uses those artifacts to compare common workloads and to isolate selected policy changes.

8.2 Main Findings

The answers developed in Section 7.1 connect the implementation to four observable results. The common representation supports both implemented target subsets; staged checks enforce their internal contracts; static analysis retains the differences between their generated programs; and allocation and mapping policies can be substituted without reconstructing the lowering sequence. The experiments also show why modularity does not imply that every policy combination is legal or beneficial. The allocator substitution preserves the checked non-allocation metrics without demonstrating a general allocation advantage. AAM mapping and tile shape, by contrast, can change replay grouping or introduce live-register conflicts. Workload scaling exposes a further distinction: the backend can partition supported independent work, but it cannot yet decompose long reductions into partial sums. Both successful compilations and diagnosed rejections therefore contribute to understanding the design space. These findings concern the implemented compiler models and the tested interfaces. They support using the backend to construct and inspect design candidates, but do not establish numerical correctness or the relative execution performance of the physical devices.

8.3 Closing Perspective

The next step is to connect the retained compiler artifacts to an independent functional model, followed by timing evaluation. The experiment-driver integration proposed in Section 7.4 would allow a common workload to be compiled and evaluated across target profiles while preserving the provenance of each result. Reduction decomposition, higher-level workloads, and additional targets would broaden the cases that this flow can examine. The thesis establishes the compiler part of that process. Architectural variation becomes an explicit input to lowering, policy choices remain observable in the generated program, and modeled constraints produce checks and diagnostics rather than undocumented assumptions. FIMDRAM AAM and HD-FIMDRAM provide two concrete cases of this approach; the contribution lies in the shared compiler organization through which their differences can be studied.

Bibliography

[1] A. Gholami, Z. Yao, S. Kim, C. Hooper, M. W. Mahoney, and K. Keutzer, "AI and memory wall," *IEEE Micro*, vol. 44, no. 3, pp. 33–39, 2024.

[2] A. Vaswani et al., "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[3] C. Duan et al., "HPIM: Heterogeneous processing-in-memory-based accelerator for large language models inference," *arXiv preprint arXiv:2509.12993*, 2025.

[4] S. Liu, Z. Huang, J. Yu, Q. Liu, and C. Chen, "McPAL: Scaling unstructured sparse inference with multi-chiplet HBM-PIM architecture for LLMs," in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, IEEE, 2025, pp. 1–7.

[5] A. A. Khan, H. Farzaneh, K. F. A. Friebel, C. Fournier, L. Chelini, and J. Castrillon, "CINM (Cinnamon): A compilation infrastructure for heterogeneous compute in-memory and compute near-memory paradigms," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, 2024, pp. 31–46.

[6] O. Mutlu, S. Ghose, J. Gómez-Luna, R. Ausavarungnirun, M. Sadrosadati, and G. F. Oliveira, "A modern primer on processing in memory," *arXiv preprint arXiv:2012.03112*, 2020. DOI: 10.48550/arXiv.2012.03112

[7] P. H. Tian, Z. Yousefijamarani, and A. Alameldeen, "DL-PIM: Improving data locality in processing-in-memory systems," *arXiv preprint arXiv:2510.07719*, 2025.

[8] M. Aghaei et al., "Modeling and simulation frameworks for processing-in-memory architectures," *arXiv preprint arXiv:2512.00096*, 2025.

[9] H. Jun et al., "HBM (high bandwidth memory) DRAM technology and architecture," in *2017 IEEE International Memory Workshop (IMW)*, IEEE, 2017, pp. 1–4.

[10] J. H. Kim et al., "Aquabolt-XL HBM2-PIM, LPDDR5-PIM with in-memory processing, and AXDIMM with acceleration buffer," *IEEE Micro*, vol. 42, no. 3, pp. 20–30, 2022.

[11] S. Lee et al., "Hardware architecture and software stack for PIM based on commercial DRAM technology: Industrial product," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, IEEE, 2021, pp. 43–56.

[12] A. A. Khan, J. P. C. De Lima, H. Farzaneh, and J. Castrillon, "The landscape of compute-near-memory and compute-in-memory: A research and commercial overview," *arXiv preprint arXiv:2401.14428*, 2024.

[13] M. Zhou, W. Xu, J. Kang, and T. Rosing, "TransPIM: A memory-based acceleration via software-hardware co-design for transformer," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, IEEE, 2022, pp. 1071–1085.

[14] P. Mannocci et al., "In-memory computing with emerging memory devices: Status and outlook," *APL Machine Learning*, vol. 1, no. 1, 2023.

[15] R. Medina et al., "Bank on compute-near-memory: Design space exploration of processing-near-bank architectures," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 43, no. 11, pp. 4117–4129, 2024.

[16] J. Wang, M. Ge, B. Ding, Q. Xu, S. Chen, and Y. Kang, "NicePIM: Design space exploration for processing-in-memory DNN accelerators with 3D-stacked-DRAM," *arXiv preprint arXiv:2305.19041*, 2023. DOI: 10.48550/arXiv.2305.19041

[17] B. Hyun, T. Kim, D. Lee, and M. Rhu, "Pathfinding future PIM architectures by demystifying a commercial PIM technology," in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, IEEE, 2024, pp. 263–279.

[18] E. Cukurtas, K. Ranawella, K. Skadron, and M. Stan, "IMPRINT: In-memory processing with indirect addressing techniques for GPU-hosted HBM-PIM," in *Proceedings of the International Symposium on Memory Systems*, 2025, pp. 165–176.

[19] C. Lattner et al., "MLIR: Scaling compiler infrastructure for domain specific computation," in *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, IEEE, 2021, pp. 2–14.

[20] A. Somaini, "A novel MLIR-based compilation flow for accelerating convolutional neural networks on process in memory architectures," 2024.

[21] P. Kukić, "A compiler framework for compute-near-memory processing units," M.S. thesis, Delft University of Technology, 2025.

[22] S. H. Seo et al., "FACIL: Flexible DRAM address mapping for SoC-PIM cooperative on-device LLM inference," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, IEEE, 2025, pp. 1720–1733.

[23] Y. Wu, Z. Wang, and W. D. Lu, "PIM-GPT: A hybrid process-in-memory accelerator for autoregressive transformers," *npj Unconventional Computing*, vol. 1, p. 4, 2024. DOI: 10.1038/s44335-024-00004-2

[24] G. Heo et al., "NeuPIMs: NPU-PIM heterogeneous acceleration for batched LLM inferencing," in *Proceedings of the 29th ACM International Conference on Architectural*

Support for Programming Languages and Operating Systems, Volume 3, Association for Computing Machinery, 2024, pp. 722–737. DOI: 10.1145/3620666.3651380

[25] Z. Li, Z. Bian, Z. Huang, G. Sun, Y. Liang, and Y. Zhuo, "TokenStack: A heterogeneous HBM-PIM architecture and runtime for efficient LLM inference," *arXiv preprint arXiv:2605.05639*, 2026. DOI: 10.48550/arXiv.2605.05639

[26] D. Christ, L. Steiner, M. Jung, and N. Wehn, "PIMSys: A virtual prototype for processing in memory," in *Proceedings of the International Symposium on Memory Systems*, Association for Computing Machinery, 2024. DOI: 10.1145/3695794.3695797

[27] Y. Zhao et al., "UM-PIM: DRAM-based PIM with uniform and shared memory space," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, IEEE, 2024, pp. 644–659. DOI: 10.1109/ISCA59077.2024.00053

[28] L. Steiner, U. Wasenmüller, and N. Wehn, "Optimization and implementation of DRAM-based interleavers for free-space optical communication beyond 100 Gbit/s," *IEEE Access*, 2025.

[29] T. Zheng, "Implementing an MLIR-based compiler for in-DRAM computing," 2025.

[30] V. Adhinarayanan et al., "Folded banks: 3D-stacked HBM design for fine-grained random-access bandwidth," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 1819–1833.

[31] JEDEC Solid State Technology Association. "JEDEC updates groundbreaking high bandwidth memory (HBM) standard." JEDEC press release reproduced by Design & Reuse, Accessed: Oct. 6, 2026. [Online]. Available: https://us.design-reuse.com/news/39018/jedec-high-bandwidth-memory-hbm-standard.html

[32] A. S. Mutschler. "Choosing the correct high-bandwidth memory," Semiconductor Engineering, Accessed: Sep. 11, 2026. [Online]. Available: https://semiengineering.com/choosing-the-correct-high-bandwidth-memory/

[33] E. Venieri, S. Manoni, A. Florian, J. Park, K. Sohn, and A. Bartolini, "AME-PIM: Can memory be your next tensor accelerator?" *arXiv preprint arXiv:2604.27808*, 2026. DOI: 10.48550/arXiv.2604.27808

[34] F. Reißmann, I. J. Castrillon, I. D. Göhringer, and I. A. A. Khan, "An MLIR-based compiler flow for memristive-crossbar accelerators," 2024.

[35] J. Ahn, S. Yoo, O. Mutlu, and K. Choi, "PIM-enabled instructions: A low-overhead, locality-aware processing-in-memory architecture," in *2015 ACM/IEEE 42nd Annual International Symposium on Computer Architecture (ISCA)*, IEEE, 2015, pp. 336–348. DOI: 10.1145/2749469.2750385

[36] J. Chen, J. Gómez-Luna, I. El Hajj, Y. Guo, and O. Mutlu, "SimplePIM: A software framework for productive and efficient processing-in-memory," in *2023 32nd Interna-*

tional Conference on Parallel Architectures and Compilation Techniques (PACT), IEEE, 2023, pp. 99–111. DOI: 10.1109/PACT58117.2023.00017

[37] S. Lee et al., "PIM-SHERPA: Software method for on-device LLM inference by resolving PIM memory attribute and layout inconsistencies," *arXiv preprint arXiv:2603.09216*, 2026.

[38] A. Boroumand et al., "LazyPIM: An efficient cache coherence mechanism for processing-in-memory," *IEEE Computer Architecture Letters*, vol. 16, no. 1, pp. 46–50, 2016. DOI: 10.1109/LCA.2016.2577557

[39] X. Sun, X. Wang, W. Li, Y. Han, and X. Chen, "PIMCOMP: An end-to-end DNN compiler for processing-in-memory accelerators," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 44, no. 5, pp. 1745–1759, 2024.

[40] P. Yang et al., "A tensor compiler for processing-in-memory architectures," *arXiv preprint arXiv:2511.15503*, 2025.

[41] Y. Shin, D. Kang, and H. Sung, "IMTP: Search-based code generation for in-memory tensor programs," *arXiv preprint arXiv:2412.19630v1*, 2024.

[42] Y. Shin, D. Kang, and H. Sung, "ATiM: Autotuning tensor programs for processing-in-DRAM," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 899–915.

[43] J. Lin et al., "SongC: A compiler for hybrid near-memory and in-memory many-core architecture," *IEEE Transactions on Computers*, vol. 73, no. 10, pp. 2420–2433, 2024. DOI: 10.1109/TC.2023.3311948

[44] H. Farzaneh, J. P. C. De Lima, M. Li, A. A. Khan, X. S. Hu, and J. Castrillon, "C4CAM: A compiler for CAM-based in-memory accelerators," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 164–177.

[45] L. Zhang and L. Shen, "PIM-HBMSim: A processing in memory simulator based on high bandwidth memory," in *International Conference on Cloud Computing, Internet of Things, and Computer Applications (CICA 2022)*, SPIE, vol. 12303, 2022, pp. 664–669.